

sCustomTkinter Reference Manual

Introduction

sCustomTkinter extends [CustomTkinter](#) with a set of widgets built for instrument panels and control applications — meters, dials, tables, file pickers — alongside themed variants of the standard controls. Every colour and font comes from one file, so an application can be restyled without touching its code.

This manual opens with four sections worth reading before the reference.

Theming is the one to read first. It covers `sCTkThemes.json`, how a local copy overrides the bundled one, and the several ways a hand-edited theme file can break. Widgets fail loudly on a missing key rather than substituting a guess, so knowing the format saves time.

Scrolling covers the container widgets and how wheel and trackpad input is handled across platforms. Read it if you are using `sCTkScrollableFrame` or `sCTkScrollArea`, or if scrolling feels wrong on your hardware.

List Properties is short. Several widgets take a list of strings — column headings, menu items, dial labels — and they all accept the same formats. Learn it once.

Designer Hints covers working in Pygubu Designer: how images resolve, why some widgets cannot be selected on the canvas, and what the green background means. Skip it if you build your interfaces in code.

The rest of the manual is reference, grouped into Containers, Controls and Display, Menus, and Additional Widgets. Each widget page has a screenshot in both light and dark modes, its constructor and properties, its public methods, and the theme block it reads — that last section is the place to start if you want to restyle something.

Most pages end with a runnable example. They are meant to be copied and run as they are.

Where a page explains why something works the way it does, or records a limitation, it is usually because the alternative was tried and did not work. Those notes are worth reading before assuming a behaviour is a bug.

Contents

- Theming
 - Where the file lives
 - Block structure
 - State maps
 - Light and dark
 - Changing values at runtime
 - Things that will break your theme
 - Adding a theme block for your own widget
- Scrolling
- List Properties
- Designer Hints
- Containers
 - sCTk
 - sCTkToplevel
 - sCTkFrame
 - sCTkScrollableFrame
- Controls and Display
 - sCTkButtonPrimary
 - sCTkButtonSecondary
 - sCTkButtonTertiary
 - sCTkCheckBox
 - sCTkEntryPrimary
 - sCTkEntrySecondary
 - sCTkLabelPrimary
 - sCTkLabelSecondary
 - sCTkLabelTertiary
 - sCTkProgressBar
 - sCTkRadioButton
 - sCTkScrollbar
 - sCTkSegmentedButton
 - sCTkSlider
 - sCTkSwitch
 - sCTkTabview
 - sCTkTextboxPrimary
 - sCTkTextboxSecondary
- Menus
 - sCTkComboBox
 - sCTkOptionMenuPrimary
 - sCTkOptionMenuSecondary
- Additional Widgets Provided by sCustomTkinter
 - sCTKDialBase
 - sCTkDialContinuous
 - sCTkDialRange
 - sCTkDialSelector
 - sCTkDialog
 - sCTkFileExplorer

- sCTkFrameLabeledPrimary
 - sCTkFrameLabeledSecondary
 - sCTkMessageBox
 - sCTkNotebook
 - sCTkPathChooser
 - sCTkScrollArea
 - ScrollBindingMixin
 - sCTkSelector
 - sCTkSeparator
 - sCTkSMeter
 - sCTkSMeterBar
 - sCTkSpinbox
 - sCTkTableview
-

Theming

Every colour, font, and several structural values in this library come from a single file: `sCTkThemes.json`. No widget hardcodes a colour. If you don't like the palette — and you may not, the author is somewhat red/brown colour blind — you change one file and the whole application follows.

- Where the file lives
- Block structure
- State maps
- Light and dark
- Changing values at runtime
- Things that will break your theme
- Adding a theme block for your own widget

Where the file lives

Two locations are checked, in this order:

1. `sCTkThemes.json` in your application's working directory — your override.
2. `scustomtkinter/assets/sCTkThemes.json` — the one shipped with the library.

The first one found wins, completely. There is **no merging**: if a local file exists, the bundled one is not consulted at all, for any widget.

That matters more than it sounds. Copy the bundled file, delete the blocks you don't care about, and every widget whose block you removed will fail to construct — not fall back to the library defaults. **Copy the whole file and edit in place.**

```
cp ../scustomtkinter/assets/sCTkThemes.json ./sCTkThemes.json
```

The file is read once, at import time. Changes require a restart.

Block structure

One block per widget class, keyed by the exact class name:

```
{
  "sCTkButtonPrimary": {
    "fg_color": ["#1A4375", "#1F6AA5"],
    "text_color": ["#FFFFFF", "#FFFFFF"],
    "font": ["Arial", 13, "bold"],
    "disabled_map": {
      "fg_color": ["#CBD5E1", "#334155"],
      "text_color": ["#94A3B8", "#64748B"]
    }
  }
}
```

The class name is the lookup key and it is **case-sensitive and exact**. A block named `sCTkButton` does nothing for a widget class named `sCTkButtonPrimary`. There is no inheritance between blocks — `sCTkDialSelector` does not inherit from a `sCTkDial` block, and each of the three dial variants carries its own full set of keys even where the values are identical.

Two kinds of key appear in a block:

- **Native CustomTkinter options** — `fg_color`, `corner_radius`, `border_width` and so on. These are passed through to the underlying widget.
- **Keys specific to this library** — `dial_color`, `led_on_color`, `scale_font`, `btn_hover`. These are read by the widget's own drawing code and never reach CustomTkinter.

You mostly don't need to know which is which. It matters in one place: see [adding a block for your own widget](#).

State maps

Nested inside a block, a state map overrides specific keys when the widget is in that state. Anything not listed keeps its normal value.

Map	Applies when
<code>disabled_map</code>	The widget is disabled via <code>state("disabled")</code> or <code>configure(state="disabled")</code> .
<code>pressed_map</code>	A button is being held down, or a latching dial is armed. Two different meanings in two different widget families: momentary for the buttons, latched for the dials.
<code>alarm_map</code>	A widget is in an alert condition.
<code>readonly_map</code>	An entry or spinbox is readonly — arrows still work, typing is blocked.

Most widgets use only `disabled_map`. The button family uses `pressed_map` and `alarm_map`; `sCTkEntryPrimary/Secondary` and `sCTkSpinbox` use `readonly_map` to support a genuine three-state model rather than collapsing everything non-disabled into "normal".

A state map is not a full block. Only list the keys that actually change. Widgets deliberately leave `fg_color` out of `disabled_map` in most cases — the background stays put and the border, text, or face carries the signal.

Some keys exist *only* inside a state map, because they have no normal-state equivalent. `sCTkSegmentedButton`'s `selected_text_color` is one: a selected segment's normal text colour comes from `text_color`, and the separate key exists so the selection is still identifiable when the control is greyed out.

Light and dark

Colours are written as a two-element list: `[light_mode, dark_mode]`.

```
"text_color": ["#1A4375", "#FF9100"]
```

Widgets pass these through as pairs rather than resolving them up front, so CustomTkinter's own appearance-mode tracking repaints them when the user switches. You don't need to do anything.

A single string is also accepted, and means the same colour in both modes. The literal `"transparent"` is a CustomTkinter pseudo-value meaning "show whatever is behind me". Note that `"transparent"` **cannot** be used for anything drawn on a raw `tkinter.Canvas` — that includes the dials, the S-meters, and `sCTkScrollArea`'s background, which need a real renderable colour.

Fonts are `[family, size]` or `[family, size, weight]`.

One place the theme does not reach

Dropdown menus ignore both your appearance mode and your colours, on `sCTkComboBox`, `sCTkOptionMenuPrimary` and `sCTkOptionMenuSecondary`.

Call `ctk.set_appearance_mode("dark")` while the operating system is set to light and the main widget goes dark while the menu that drops out of it stays light. Switch the system to dark and the menu follows — proving it tracks the OS rather than the application.

The same applies to the three keys that describe the menu:

- `dropdown_fg_color`
- `dropdown_text_color`
- `dropdown_hover_color`

On macOS they have no visible effect at all. Keep them: they are expected to work where Tk draws the menu itself rather than handing it to the operating system, so this is platform-specific rather than dead.

This is **not** something the theme file can fix, and not specific to this library — plain `customtkinter.CTkComboBox` behaves identically, confirmed by direct testing. The dropdown is a native menu that the operating system draws itself, largely ignoring the colours a widget configures on it.

The only real fix would be replacing the native menu with a CustomTkinter-drawn one — a `CTkTopLevel` holding themed buttons. That is a widget-level project, not a theme change, and it hasn't been done.

If your application sets an appearance mode explicitly rather than following the system, expect this mismatch on those three widgets.

Changing values at runtime

`configure()` accepts theme keys directly, and the override **sticks**:

```
frame.configure(fg_color="red")
```

Two consequences worth understanding:

A single colour replaces the light/dark pair. Passing one value for a key means that property stops following appearance mode — which is what asking for one specific colour means. Pass a two-element tuple if you want it to keep tracking:

```
frame.configure(fg_color=("#FFFFFF", "#111827"))
```

State maps still win in their state. Overriding `border_color` sets the *normal*-state colour. If the widget is disabled, or later becomes disabled, `disabled_map` supplies the border colour as usual. To change what a disabled widget looks like, change the theme file.

An override survives a state change. Set a colour, disable the widget, enable it again, and your colour is still there. That sounds obvious and was not true until recently: widgets repaint themselves from a snapshot of the theme taken at construction, and a runtime change was not written into that snapshot — so the next repaint quietly restored the theme value. Every widget now records the change first, and the repaint reproduces it.

Only keys the theme block defines are tracked. That is exactly the set a repaint reapplies, so it is the set that needs recording. `configure(width=200)` on a widget whose block says nothing about `width` is an ordinary property change and behaves as it always did.

A key absent from `disabled_map` does not dim — and neither does an override of it. Several widgets deliberately leave `fg_color` out, so a transparent widget does not gain a solid background when disabled. Override that property and it stays exactly as you set it through a state change, because there is nothing to change to. The library will not invent a dimmed version of your colour. If you want it to dim, add the key to `disabled_map`.

`text_color_disabled` and `disabled_map.text_color` are one value. The first is CustomTkinter's own option, the second this library's theme key, and a repaint feeds one into the other. Setting either is equivalent; they will not fight.

Things that will break your theme

This section is the important one. JSON is unforgiving and the failure modes are not always obvious.

Syntax errors take out the entire file

A missing comma, a stray trailing comma before a `}`, an unclosed brace, or a smart quote pasted in from a document — any one of these makes the whole file unparseable. The library catches the error, prints a warning, and **continues with an empty theme registry**. Every widget then fails to construct.

The warning looks like this:

```
sCustomTkinter System Warning -> Could not parse theme layout tracking: ...
```

If you see that line, the problem is a syntax error in your file, not in any widget. Validate before running:

```
python -m json.tool sCTkThemes.json > /dev/null
```

Silence means it parsed. Any editor with JSON support will also flag these as you type — worth using one.

Deleting a key is not the same as leaving it at default

There is no "default" to fall back to. Widgets validate their required keys at construction and raise immediately:

```
KeyError: "'sCTkTabview' theme block is missing 'text_color' at the top level of sCTkThemes.json."
```

That message names the exact key and whether it belongs at the top level or in a state map. This is deliberate. An earlier design substituted a plausible hardcoded colour when a key was missing, and the result was worse than a crash: five separate widgets shipped for a long time rendering in hardcoded colours while their configured theme values were silently ignored, and nobody noticed because the substituted colours *looked* fine. A loud failure that names the key is far better than a widget that quietly ignores you.

If you genuinely don't want a widget's block, don't delete it — you'll break that widget. Change its values instead.

Misspelling a key is worse than deleting it

A misspelled key is not an error. It's an unrecognised key that gets ignored, while the *correct* key is now missing:

```
"text_colour": ["#1A4375", "#FF9100"]
```

That produces a `KeyError` about `text_color` being missing — which is confusing until you spot that your line is right there, spelled British. Check the spelling in this library's own documentation for the widget; it uses American spellings throughout, following CustomTkinter.

A misspelling inside a state map is quieter still: state maps aren't validated as strictly, so a typo there usually means "that property just doesn't change when disabled," with no error at all.

A widget with no block at all is themed by CustomTkinter

Not a crash, and easy to miss. A widget whose block is absent gets no values from this file, so it renders in CustomTkinter's own defaults and its state maps are empty — meaning it never dims when disabled.

`sCTkSlider` was in exactly this position, and the way it got there is the warning. Its block existed and was documented — the values are still in `sCTkSlider.md` — but it went missing from the file at some point. Nothing failed. The slider simply looked like a plain CTK slider and did not change when disabled, and the absence was only noticed when someone tried to clear a colour in the Designer.

A block can disappear from a hand-edited file without anything announcing it, which is exactly why the check below is worth running after a large edit.

A quick check that every widget has one:

```
python3 -c "
import json
t = json.load(open('sCTkThemes.json'))
print(sorted(t))
"
```

Compare that list against the widget pages. Six absences are correct: `sCTk` itself, the abstract `sCTkDialog` base, `sCTkDialogTopLevel` (which shares `sCTkDialog`'s block), the two mixins, and the plugin registration module.

Renaming a block orphans it

Rename `sCTkSlider` to `sCTkSliders` and the block becomes dead data while every slider fails to construct. Block names must match class names exactly.

The reverse also happens: a block for a widget that no longer exists, or was renamed, sits in the file doing nothing. Harmless, but it accumulates.

Adding a key that isn't read does nothing

Adding `"hover_glow_color"` to a block will not make anything glow. Widgets read a fixed set of keys; extra ones are ignored silently. If you want a new visual property, the widget's drawing code has to read it.

Adding a theme block for your own widget

If you subclass `ThemeableWidget`, your block is found automatically by class name. Three things to know:

Custom keys must not reach the native constructor. CustomTkinter widgets reject keyword arguments they don't recognise — `CTkToplevel` in particular raises on *any* unknown key. Filter your resolved keywords down to what the native class actually accepts before calling `super().__init__()`. Every widget in this library that adds custom keys does this; copy the pattern from one close to yours.

Some custom keys are stripped for you. `ThemeableWidget` maintains an internal list of names it removes from the resolved keywords for canvas-drawing widgets — `dial_color`, `text_color`, `pointer_color` and others. If your widget uses one of those names, it will not be in `final_kw`, and you must read it from the raw theme registry instead. This is not obvious and has caused real bugs: an entire widget family rendered in fallback colours for its whole existence because it looked for those keys in the wrong place. If a colour you configured isn't appearing, this is the first thing to check.

Validate your required keys at construction. Follow the pattern used throughout:

```
_REQUIRED_THEME_KEYS = ("fg_color", "text_color")
_REQUIRED_DISABLED_KEYS = ("text_color",)
```

and raise `KeyError` naming the missing key. It costs a dozen lines and turns a class of silent visual bug into an immediate, self-explaining failure.

Scrolling

Scrolling in this library is handled in one place. Whichever widget you use, the wheel and trackpad behaviour comes from a single shared implementation — `ScrollBindingMixin` — so it feels the same everywhere and a fix applies everywhere.

- Which widget to use
- How scroll input is handled
- Tuning scroll speed
- Disabling scrolling
- Nested scrolling regions

Which widget to use

Widget	Use when
<code>sCtkScrollableFrame</code>	You want a scrolling container and don't care where the scrollbar lives. This is the default choice.
<code>sCtkScrollArea</code> + <code>sCtkScrollbar</code>	You need the scrollbar somewhere the built-in one can't go, or you want to control child event binding explicitly.
<code>sCtkFileExplorer</code> , <code>sCtkTableview</code> , <code>sCtkSelector</code>	These scroll internally. You don't wire anything up.

`sCtkScrollableFrame` builds and manages its own scrollbar. `sCtkScrollArea` deliberately doesn't — you create an `sCtkScrollbar` separately and connect the two with `hook_scrollbar()`. That's the whole reason the pair exists: it lets the bar sit outside the scrolling region, share space with other widgets, or be styled independently.

```
scroll_view = sCtkScrollArea(container)
scroll_view.pack(side="left", fill="both", expand=True)

scrollbar = sCtkScrollbar(container, orientation="vertical")
scrollbar.pack(side="right", fill="y")

scroll_view.hook_scrollbar(scrollbar)

# Content goes into scroll_content, not into the area itself.
for row in data:
    sCtkLabelSecondary(scroll_view.scroll_content, text=row).pack(anchor="w")
```

Content added to `scroll_content` is bound for scrolling automatically, including anything added later. You do not need to call `propagate_scroll_events()` on each item — that method now exists only for widgets placed *outside* the content tree.

How scroll input is handled

Three platforms behave differently, and all three are handled:

Platform	Mechanism
Windows	<MouseWheel> with a delta scaled in units of 120
Linux	Discrete <Button-4> / <Button-5> events — there is no continuous delta

Platform	Mechanism
macOS	Its own <MouseWheel> scaling, plus a separate higher-precision <TouchpadScroll> event

macOS trackpads deliver far more events, with far finer values, than a wheel does. Acting on each one is unusably fast, so trackpad deltas accumulate and move the view only once a threshold is crossed. The accumulator resets when you reverse direction, so a change of direction responds immediately rather than having to cancel out what built up going the other way.

Bindings activate on their own and maintain themselves. You never call an activation method, and content added after a widget is placed — the normal case, since you construct, place, then populate — is picked up automatically.

Full detail, including why this takes four separate mechanisms, is on the [ScrollBindingMixin](#) page.

Tuning scroll speed

Three constants control the feel. They live on `ScrollBindingMixin` as class attributes, so they can be changed globally, per widget class, or per instance.

Constant	Default	Effect
<code>MAC_SCROLL_SENSITIVITY</code>	3	Amplification for macOS wheel deltas, which are much smaller than Windows' steps.
<code>MAC_SCROLL_MAX_STEP</code>	5	Ceiling on rows travelled per macOS wheel event.
<code>TOUCHPAD_ACCUMULATION_THRESHOLD</code>	12.0	Accumulated trackpad movement required before the view moves.

`MAC_SCROLL_MAX_STEP` is the one you're most likely to want to change. macOS reports wildly different delta magnitudes depending on hardware: an Apple Magic Mouse sends fine values near 1, while a conventional wheel mouse sends a large value per detent — around 38 in testing. Without a ceiling, the amplification turns one wheel click into 114 rows of travel, which throws a hundred-row list end to end. The clamp lets small deltas scale normally and saturates large ones.

Set it to 3 for the conventional three-rows-per-notch that matches macOS defaults and most applications. Values below 3 also slow the Magic Mouse, since its fine deltas already scale to 3 before the clamp applies.

To change it everywhere, set it once at startup:

```
from scustomtkinter.sctk_scroll_mixin import ScrollBindingMixin
ScrollBindingMixin.MAC_SCROLL_MAX_STEP = 3
```

Or per instance, if one widget wants a different feel:

```
log_view = sCTkScrollableFrame(root)
log_view.MAC_SCROLL_MAX_STEP = 8
```

These values are tuned on macOS. If you're shipping to Windows or Linux and the feel is wrong, these are the knobs.

Disabling scrolling

`sCTkScrollableFrame` and `sCTkFileExplorer` both stop scrolling entirely when disabled — wheel, trackpad, and scrollbar dragging. The bar stays visible but inert; CustomTkinter's scrollbar has no greyed-out appearance to switch to.

`sCTkScrollableFrame` additionally separates two ideas that are easy to confuse:

- **state** is the user-facing enabled/disabled presentation.
- **scroll_enabled** is your own intent about whether this frame should scroll at all.

Scrolling happens only when both allow it, and neither overwrites the other. A frame you deliberately set non-scrolling stays non-scrolling after a disable/enable round trip:

```
frame = sCTkScrollableFrame(master, scroll_enabled=False)
frame.configure(state="disabled")
frame.configure(state="normal")
frame.is_scrolling()      # still False -- your intent survived
```

`disable_scroll()` and `enable_scroll()` are the runtime equivalents, useful when adding a lot of content at once:

```
frame.disable_scroll()
for item in many_items:
    sCTkLabelSecondary(frame, text=item).pack()
frame.enable_scroll()
```

`sCTkScrollArea` has no disabled state.

Nested scrolling regions

Putting one scrolling widget inside another works: the inner one keeps its own bindings and the outer one stops at its boundary, so the wheel scrolls whichever region the pointer is actually over rather than both at once.

The guard recognises `CTkScrollableFrame` and anything built on it — `sCTkScrollableFrame`, `sCTkTableview`, `sCTkSelector`. A scrolling region you build yourself directly on a plain `tkinter.Canvas` is **not** recognised, and would get bound to the outer widget as well as your own handler. If you need that, put it in an `sCTkScrollArea` instead.

List Properties

Several widgets take a list of strings — column headings, selectable items, dial labels, file extensions. All of them accept the same formats and parse them through one shared function, so the format you learn for one works everywhere.

- Accepted formats
- Which properties
- In Pygubu Designer
- In Python code
- Why this is shared

Accepted formats

Three forms, all equivalent:

```
["Name", "Freq", "Mode"]    # bracketed list -- preferred
'Name, Freq, Mode'          # bare comma-separated
["Name", "Freq", "Mode"]    # a real Python list, in code
```

Details that hold across all of them:

- **Either quote style**, and mixing them is fine: `['A', "B"]` parses correctly.
- **Whitespace is stripped** from every value, so `AM, FM, LSB` gives `["AM", "FM", "LSB"]` with no leading spaces.
- **Quotes are optional** in the bracketed form: `[A, B]` works, even though it isn't valid Python.
- **Empty values are dropped**, so `a,,b` gives two items, not three.
- **Empty input** gives an empty list — or the property's own default where it has one — never `None`.

Use the bracketed form when a value contains a comma. That's the one thing the bare form can't express:

```
["Smith, John", "Doe, Jane"] # two values
'Smith, John, Doe, Jane'     # four values
```

Space is **not** a separator. `Meat Loaf` is one value.

Which properties

Widget	Property
<code>sCTkTableview</code>	<code>columns</code>
<code>sCTkSelector</code>	<code>items</code>
<code>sCTKDialSelector</code>	<code>labels</code>
<code>sCTKSpinbox</code>	<code>values</code>
<code>sCTKFileExplorer</code>	<code>filetypes</code>
<code>sCTKPathChooser</code>	<code>filetypes</code>

All are settable at construction and through `configure()`, and both paths parse identically.

In Pygubu Designer

Type the bracketed form into the property field:

```
["AM", "FM", "LSB"]
```

The bare comma-separated form also works and is quicker to type, but the bracketed form is what the inspector's defaults and help text show, for two reasons: it can express a value containing a comma, and it matches what the generated Python code will contain.

Generated code always emits a real Python list regardless of which form you typed.

In Python code

Pass a real list. There's no reason to pass a string:

```
table = sCTkTableview(parent, columns=["Channel", "Frequency", "Mode"])
selector = sCTkSelector(parent, items=["Ch 1", "Ch 2", "Ch 3"])
mode = sCTkDialSelector(parent, labels=["AM", "FM", "LSB", "USB", "CW"])
```

Tuples work too, and are returned as lists.

Why this is shared

These properties previously had **seven different parsers** and no two behaved alike:

Widget	Old behaviour
sCTkTableview.columns	comma split, stripped
sCTkSelector.items	ast.literal_eval only — the bare form failed
sCTkDialSelector.labels	literal_eval at construction, but a plain comma split with no quote stripping in configure() — the same value parsed differently depending on when it was set
sCTkFileExplorer.filetypes	three separate implementations, in one file
sCTkPathChooser.filetypes	literal_eval only
sCTkSpinbox.values	shlex.split when no comma was present, making space a separator there and nowhere else

The user-visible result was that each widget wanted a different format for the same kind of property, with nothing in the inspector to say which. Entering `AM`, `FM`, `LSB` into a dial produced labels with leading spaces; the identical string in a Tableview worked correctly. Entering `["A", "B"]` into a dial produced two garbage values.

They now all call `parse_list_property()` in `themeable_widget.py`. Space separation was dropped rather than propagated — `Meat Loaf` being two values in a spinbox and one value everywhere else was more surprising than useful.

If you add a widget with a list property, use that function. It's the difference between a format users can learn once and a format they have to look up per widget.

Designer Hints

Things that are true across the library when working in Pygubu Designer. Widget-specific quirks stay on each widget's own page.

Rough notes for now — expect this to grow.

Images resolve relative to the running directory

Set an image in the inspector and it appears on the canvas. Run the generated code from a different directory and it won't.

The image property editor stores only the **basename**. The full path lives in the designer's own registry for the length of the session and does not survive into generated code, which emits the bare name:

```
self.img_logo = image_loader(master, "logo.png")
```

The generated `safe_image_loader` is an explicit stub — its docstring says "Setup image_loader in derived class file" — and does a plain `tk.PhotoImage(file=name)`. So the name resolves against the process's working directory.

Put the image beside the script you run, or supply your own loader. This is pygubu's design rather than a defect: the name is a key, and resolving it is the application's job.

Worth knowing that the two widget families fail differently when it isn't found. The tk stub wraps its load in `except tk.TclError: pass` and returns `None`, so `configure(image=None)` succeeds and you get no image and no message. The CustomTkinter path calls `Image.open()` unwrapped and raises `FileNotFoundError`. Same cause, opposite symptoms — a silent blank in one case, a traceback in the other.

Images fail in "Preview in toplevel"

Separate from the above, and not a path problem. A widget with an image shows correctly on the design canvas, then previewing it produces:

```
Failed to set property 'image' ... Error: image "pyimage60" does not exist
```

That is Tk saying there is no image by that name in **this** interpreter, not that a file is missing. The preview builds a new root, and images created against the designer's own interpreter do not exist in it.

Reproducible with a plain `customtkinter.CTkLabel`, so it is not something this library introduces. CustomTkinter's designer plugin rebuilds its image cache when a new root appears, for exactly this reason -- the widgets that go through that path are fine, and others are not.

No workaround beyond checking images in the design view and in the generated code rather than in the preview.

Overriding a theme value in the inspector

Set a colour, a font or a size in the inspector and it **replaces** the theme's value for that widget, for as long as that widget exists. Leave the field blank and the theme applies.

An override survives state changes. Set `fg_color` red, disable the widget, re-enable it, and it is still red. That sounds obvious, and it was not true until recently: widgets repaint themselves from a snapshot of the theme taken at construction, and nothing wrote runtime changes into that snapshot — so any state change quietly restored the theme colour. In the Designer it happened immediately, because properties are applied in an order that sets `state` after the colours.

Clearing a field returns to the theme, not to the last value. Blanking a property does not blank the property — it reverts to what the theme says, which is what the generated code produces anyway, since the property is simply omitted and the constructor default applies. Clearing a button's `apply_text` shows "Apply" again rather than an empty button.

The default reported to the Designer comes from an untouched copy of the theme block, so clearing a field you have already overridden gives you the theme's value rather than the override you just replaced.

Properties the theme says nothing about keep their current value instead. A no-op is better than a guessed default, because the guess would be applied.

One field means one thing. A widget usually has two sets of colours — the normal ones and a `disabled_map` — but the inspector shows a single field per property. That field always edits the **normal** value, whatever state the widget is currently in. Setting a colour on a disabled widget therefore shows nothing until you re-enable it.

The one exception is `text_color_disabled`, which has its own field precisely because it names the disabled value directly. Setting it is equivalent to setting `disabled_map.text_color` in the theme, and the two are kept as a single value rather than fighting each other.

A property the theme says nothing about is not overridden — it is just set. Only keys present in a widget's theme block participate in any of this, because those are the only ones a repaint touches.

Some properties are absent from `disabled_map` on purpose, and keep their current value when the widget is disabled rather than changing. The labels and `sCtkSelector` are the clearest cases: their `fg_color` is `transparent`, so a disabled fill would give them a solid background they do not otherwise have — appearing when you disable the widget and vanishing when you enable it. If a property does not appear to change on disable, that is usually why.

An override of such a property will not dim either. Set a colour the theme has no disabled value for, and it stays exactly as you set it through a state change. That is the same rule, not a separate one — there is nothing to change to, and the library will not invent a dimmed version of your colour.

If you want it to dim, add the key to that widget's `disabled_map` in the theme. Your override then dims to whatever you put there.

An override can look stale on the canvas after a state change. Set two colours on a slider, switch it to disabled, and one may keep the colour you gave it while the other dims — the canvas shows a value the widget has already moved on from.

The data is correct: preview and generated code both dim properly, and adding any other widget forces the canvas to catch up. It comes from the Designer applying properties one at a time as you edit, in an order that can land a colour after the repaint that was meant to replace it. Nothing is wrong with what gets saved.

The Bindings tab is empty

Deliberately, and not by us — CustomTkinter's plugin disables it on twelve builder objects individually. Most CTk widgets are composites that draw on an internal canvas, and a binding attached to the outer widget frequently never fires; the canvas or a child receives the event instead. An enabled tab producing dead bindings would be worse than no tab.

Bind in your derived class instead:

```
class MyApp(baseui.MyAppUI):
    def __init__(self, master=None):
        super().__init__(master)
        self.my_entry.bind("<KeyRelease>", self.on_key)
```

Some widgets are not in the palette, deliberately

`sTkMessageBox` is raised at runtime in response to an error or an informational event, not placed on a form — there is nothing to design.

`sTkDialogTopLevel` is created by `sTkDialog`, never placed by a user. The class ships; only its Designer registration does not.

Both are omissions by choice, not oversights.

Some widgets are selected from the tree, not the canvas

Most widgets can be selected by clicking them on the canvas. Two cannot: **tabview tabs**, and the three scrolling frames -- `sTkScrollableFrame`, `sTkFrameLabeledPrimary` and `sTkFrameLabeledSecondary`.

Clicking a tab switches the page but leaves the inspector showing whatever was selected before; clicking a widget inside the new page corrects it. Clicking one of the three frames does nothing at all.

Select them in the widget tree. Selecting a tab there switches the canvas to it, and selecting a widget inside a tab switches to whichever tab holds it. A widget dropped *inside* any of these is selectable by clicking as normal, so this affects only the container itself.

Both are dead ends rather than outstanding bugs, for reasons in how CustomTkinter builds those two widgets. If you are curious, or tempted to fix it, `dev/docs/Developing.md` records the mechanism and the three approaches that were tried and failed.

Segmented buttons, dials and the file explorer were on this list until recently and are now selectable.

sCTkTabview does not grow with its contents

A tabview is a fixed size. Drop widgets into a tab and the tabview stays as it was, clipping anything that does not fit -- it does not expand the way a frame does, and no amount of packing or `expand=True` on the children changes that.

Set `width` and `height` on the tabview itself.

This is native `CTkTabview` behaviour rather than something this library adds: the widget takes explicit dimensions and does not propagate its children's requested size. Its theme block sets neither, so the size you get without asking is CustomTkinter's own default.

If a tab's contents look cut off, that is why. Reach for `width` and `height` before suspecting the layout.

Transparent widgets look wrong on the canvas

A widget whose theme sets `"transparent"` shows whatever is behind it. At runtime that's a themed parent, so it follows light and dark correctly. The design canvas does not participate in appearance mode — it is a fixed light grey — so in dark mode a transparent widget keeps a light background while its text follows the dark palette, and the text can become almost unreadable.

Several widgets in this library carry a concrete background on their preview class for exactly this reason. If you meet it on one that doesn't, that's why.

The **light green** you sometimes see is pygubu's own preview background, showing through where a widget doesn't fill the space the canvas allotted it. Not your theme, and not a fault.

Widgets that build their own contents are not containers

You cannot drop children into `sCTkSelector`, `sCTkPathChooser`, `sCTkFileExplorer` or the dials. Each builds and manages what it holds, and a child dropped in would land in an unmanaged position and be destroyed by the next rebuild.

`sCTkDialog` is the exception that looks like one of these but isn't: widgets dropped onto it land in its content area automatically, which is the point.

Dropdown colours are ignored on macOS

The dropdown lists on `sCTkComboBox`, `sCTkOptionMenuPrimary` and `sCTkOptionMenuSecondary` are native menus the operating system draws itself. On macOS that means the OS decides how they look, and the theme keys that describe them have no effect:

- `dropdown_fg_color`
- `dropdown_text_color`
- `dropdown_hover_color`

The same applies to appearance mode: call `ctk.set_appearance_mode("dark")` while the system is in light mode and the dropdown stays light, whatever the rest of the application does.

All of it is reproducible with plain `customtkinter.CTkComboBox`, so it is not something this library introduces and it cannot be fixed from the theme file.

Platform-specific, not dead. These keys are expected to work where Tk draws the menu itself rather than handing it to the OS. Do not remove them from your theme because they do nothing on a Mac.

See [Theming](#).

Containers

The following widgets are the containers that will hold your user interface. There are some additional constainers that might be of interest that are listed later in the section where we document additional widgets added with `sCustomTkinter`.

sCTk

The `sCTk` is the primary main window container class wrapper for the `sCustomTkinter` workstation library ecosystem. It acts as a clean, direct pass-through equivalent to its foundational parent container layout class, `customtkinter.CTk`.

Table of Contents

- [Overview](#)
 - [Constructor](#)
-

Overview

The application base frame serves as the core master anchor for your interface tree:

1. **Decoupled User Space:** It eliminates the architectural requirement to maintain raw `import customtkinter` bindings inside your station cockpit panel code.
 2. **Framework Alignment:** It standardizes the root initialization sequence pass to match the repository's native object naming conventions (`sCTkFrame`, `sCTkButtonPrimary`, etc.).
-

Constructor

It maps perfectly onto all native window properties, event loop callbacks, lifecycle handlers, and geometries tracking parameters out-of-the-box.

```
from sCTk import sCTk
from sCTkThemes import apply_sCTkThemes

1. Initialize centralized framework look records natively on system boot
apply_sCTkThemes()

2. Instantiate your primary root application backplane directly
app = sCTk()
app.geometry("800x600")
app.title("Main Control Rig Backplane")

app.mainloop()
```

[Return to Table of Contents](#)

sCTkToplevel

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkToplevel is a themeable subclass of `customtkinter.CTkToplevel`, for secondary windows, modal dialogs, and popups. It adds automatic light/dark theme resolution from `sCTkThemes.json`. This is the simplest widget in the library — no disabled state, no `state()` / `get_state()` at all, and no per-state color-swapping logic, since a top-level window has no interactive "enabled/disabled" concept the way a control does.

Constructor

```
sCTkToplevel(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent window.
**kwargs	—	—	Any native CTkToplevel argument, or an <code>fg_color</code> override — the theme block for this widget currently defines only <code>fg_color</code> .

```
settings_window = sCTkToplevel(root)
settings_window.title("Settings")
settings_window.geometry("300x200")
```

Methods

Method	Returns	Description
<code>configure(**kwargs)</code> / <code>config(**kwargs)</code>	varies	Standard widget configuration, with positional-dict support (e.g. <code>configure({"fg_color": "red"})</code>). There's no single-argument property-query support here — unlike every other widget in this library, a bare positional string (e.g. <code>configure("fg_color")</code>) currently has no effect at all, since the only positional-argument handling implemented is the dict-merge case.

Theming (sCTkThemes.json)

Everything is applied once, at construction — there's no `disabled_map` and no runtime color-swapping logic at all.

```
{
  "sCTkToplevel": {
    "fg_color": ["#F8FAFC", "#0F172A"]
  }
}
```

Safe to use as a base class for your own composite widgets. If you build a composite widget by inheriting `sCTkToplevel` directly, construction is protected on two fronts: a run-once guard in `ThemeableWidget.__init__` stops your composite's own `final_kw` from being silently overwritten if your widget explicitly calls `ThemeableWidget.__init__` before `super().__init__()`; and this widget's own constructor only forwards the specific keys `CTkToplevel` actually accepts. This matters more here than for most widgets — confirmed directly against CustomTkinter's own source, `CTkToplevel.__init__` explicitly validates that no unrecognized keyword survives after its own known-valid keys are popped, and raises immediately if one does. This only matters for the base-class composition pattern — constructing a plain `sCTkToplevel` directly is unaffected either way.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkToplevel, sCTkLabelPrimary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("Toplevel Example")

    def open_settings():
        settings_window = sCTkToplevel(root)
        settings_window.title("Settings")
        settings_window.geometry("300x200")
        sCTkLabelPrimary(settings_window, text="Settings go here").pack(expand=True)

    open_button = sCTkButtonPrimary(root, text="Open Settings", command=open_settings)
    open_button.pack(pady=20)

    root.mainloop()
```

Known Limitations

- No single-argument property-query support (e.g. `configure("fg_color")` does nothing) — different from every other widget in this library. `ThemeableWidget._configure_query()` would supply it in one line if a use for it appears; a window has no inspector to answer to, which is why it has not been added.
- No `state()` / `get_state()` / disabled concept at all — this widget has no visual state to toggle.

[Return to Table of Contents](#)

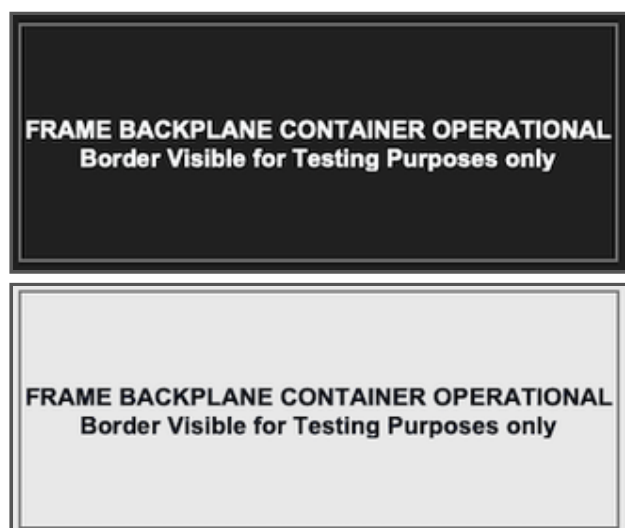
sCTkFrame

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkFrame is a themeable subclass of `customtkinter.CTkFrame`. It adds automatic light/dark theme resolution from `sCTkThemes.json`. Unlike every other widget in this library, it has no disabled state and no per-state color swapping — frames are containers, not interactive controls, so there's nothing to dim or lock.



Constructor

```
sCTkFrame(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
**kwargs	—	—	Any native CTkFrame argument, or an override for one of the theme keys listed under Theming .

```
panel = sCTkFrame(control_root)
panel.pack(expand=True, fill="both", padx=20, pady=20)
```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	No-op. Always returns "normal" regardless of what's passed in — deliberate, not a bug, so generic code written against every widget's <code>state()/get_state()/configure(state=...)</code> API doesn't need a special case for frames.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument. Always "normal".
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> is silently absorbed (a no-op) rather than forwarded to the native widget, which has no real state concept; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (<code>name</code> , <code>name</code> , <code>name</code> , <code>default</code> , <code>current</code>) tuple for <code>state</code> , <code>fg_color</code> , and <code>border_color</code> — since neither varies by state here, <code>default</code> and <code>current</code> are always identical. Queries for any other property name fall through to the native <code>CTkFrame.configure</code> .

Theming (sCTkThemes.json)

Everything is applied once, at construction — there's no `disabled_map` for this widget, and no runtime color-swapping logic at all.

```
{
  "sCTkFrame": {
    "border_width": 0,
    "corner_radius": 0,
    "border_color": ["gray", "gray"],
    "fg_color": "transparent"
  }
}
```

With `border_width` at `0`, `border_color` never actually renders visibly regardless of its value — the two are set to the neutral Tkinter color name "gray" for both light and dark mode here, but that's moot while the border has no width.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are passed through as raw (`light`, `dark`) tuples at construction and never touched again, so CustomTkinter's own native appearance-mode tracking handles light/dark repaints on its own — there's no `_set_appearance_mode()` override here, since there's nothing for one to re-trigger. This is the same underlying mechanism validated more deliberately on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family.

Safe to use as a base class for your own composite widgets. If you build a composite widget by inheriting `sCTkFrame` directly (rather than placing it as a child), construction is protected on two fronts: a run-once guard in `ThemeableWidget.__init__` stops your composite's own `final_kw` from being silently overwritten if your widget explicitly calls `ThemeableWidget.__init__` before `super().__init__()`; and this widget's own constructor only forwards the specific keys native `CTkFrame` actually accepts (confirmed directly against CustomTkinter's source) to

its own native constructor call, so any of your composite's own theme keys that `CTkFrame` wouldn't recognize are filtered out rather than causing a `TypeError`. This only matters for that composition pattern — constructing a plain `sCTkFrame` directly is unaffected either way.

Example

```
#!/usr/bin/python3

from scustomtkinter import sCTkButtonPrimary, sCTkLabelPrimary, sCTk, sCTkFrame

if __name__ == "__main__":

    root = sCTk()
    root.geometry("500x300")
    root.title("sCTkFrame Container Validation Bench")

    # Instantiate your custom theme-compliant frame element chassis
    base_container = sCTkFrame(root, border_width=2)
    base_container.pack(expand=True, fill="both", padx=30, pady=30)

    #
    # # Add a simple sub-element child widget to verify structural clipping layouts
    lbl_marker = sCTkLabelPrimary(base_container, text="FRAME BACKPLANE CONTAINER OPERATIONAL\n"+
                                   "Border Visible for Testing Purposes only")
    lbl_marker.pack(expand=True)

    root.mainloop()
```

Known Limitations

- `state()` / `get_state()` / `configure(state=...)` are all no-ops by design — there's no way to visually disable a frame through this API, since the widget has no disabled state at all.
- **Fixed:** single-argument queries used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state` / `fg_color` / `border_color`, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

sCTkScrollableFrame

Table of Contents

- Overview
- Constructor
- Scrolling and State
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

`sCTkScrollableFrame` is a themeable subclass of `customtkinter.CTkScrollableFrame`. It adds automatic light/dark theme resolution from `sCTkThemes.json`, plus carefully-tuned cross-platform mouse wheel and macOS trackpad scroll handling that native `CustomTkinter` doesn't reliably provide on its own.



Unlike `sCTkFrame`, this widget **does** have a disabled state. That's justified here where it isn't for a plain frame: this widget owns real behavior to disable, not just colors. Disabling dims the border and scrollbar and stops all scrolling — wheel, trackpad, and scrollbar drag alike.

Disabling does **not** cascade to child widgets. That remains the caller's responsibility, exactly as with the labeled frame variants — loop over `get_children()` and call `.configure(state=...)` on each one.

Constructor

```
sCTkScrollableFrame(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
state	str	"normal"	"normal" or "disabled". See Scrolling and State .
scroll_enabled	bool	True	Whether this frame should respond to scroll input at all.
**kwargs	—	—	Any native CTKScrollableFrame argument (e.g. <code>label_text</code> , <code>orientation</code>), or an override for one of the theme keys listed under Theming .

```
log_viewport = sCTkScrollableFrame(dashboard, width=380, height=250, label_text="Telemetry Log")
log_viewport.pack(padx=20, pady=20, fill="both", expand=True)
# Scrolling works immediately -- no activation call needed.
```

Scrolling and State

Scroll bindings are **automatic** and self-maintaining. No activation call is needed, and content added after the widget is placed is picked up on its own.

Scroll handling comes from [ScrollBindingMixin](#), the library's single shared implementation — that page covers the four activation mechanisms, the debounced content rebind, the platform models, the nested-frame guard, and the tuning constants (`MAC_SCROLL_SENSITIVITY`, `MAC_SCROLL_MAX_STEP`, `TOUCHPAD_ACCUMULATION_THRESHOLD`). This widget supplies four hooks: `_scroll_target()` resolves the parent canvas via `wininfo_parent()`, since it's wrapped by a native `CTkScrollableFrame` that owns the canvas; `_scroll_layers()` assembles the frame, canvas, that canvas's parent, the scrollbar, and the content tree; `_scroll_permitted()` returns `is_scrolling()`; and `_scroll_drag_targets()` returns the internal scrollbar.

It also passes `_parent_frame` as the mixin's `extra_map_widget`, because this widget is a canvas-window child and may never receive `<Map>` itself.

state and **scroll_enabled** are two independent axes, deliberately not collapsed into one. `state` is the user-facing enabled/disabled presentation; `scroll_enabled` is the developer's own intent about whether this frame should scroll at all. Effective scrolling is the AND of the two:

scroll_enabled	state	Scrolls?
True	"normal"	yes
True	"disabled"	no
False	"normal"	no
False	"disabled"	no

Because state changes never write to `scroll_enabled`, intent survives a round trip. A frame explicitly set non-scrolling stays non-scrolling after `state="disabled" → state="normal"`, rather than being silently switched on by the state change.

This is also why `cget("scroll_enabled")` reports stored **intent** while `is_scrolling()` reports the live **effective** result. A frame with `scroll_enabled=True` that has been disabled returns `True` from the former and `False` from the latter.

Temporarily suspending scroll is a supported pattern, useful during bulk content updates where rebinding on every widget added would be wasted work:

```
frame = sCTkScrollableFrame(master)
frame.disable_scroll()
frame.pack(fill="both", expand=True)
for item in many_items:
    sCTkLabelSecondary(frame, text=item).pack()
frame.enable_scroll()
```

Calling `disable_scroll()` before placement correctly suppresses automatic activation rather than being overridden by it — every activation path routes through the same effective-state check. Passing `scroll_enabled=False` to the constructor achieves the same starting state without the separate call.

Methods

Method	Returns	Description
<code>configure(**kwargs) / config(**kwargs)</code>	None	Standard configuration. Accepts state and <code>scroll_enabled</code> alongside any native option. Both are this library's own properties and are removed before reaching native <code>CTkScrollableFrame.configure()</code> , which rejects unrecognized keywords.
<code>configure(name)</code>	tuple	Pygubu-style single-argument query for <code>fg_color</code> , <code>label_fg_color</code> , <code>scrollbar_button_color</code> , <code>border_color</code> , <code>state</code> , and <code>scroll_enabled</code> . For the color keys the default and current positions are identical; for state and <code>scroll_enabled</code> they can differ, since those carry live runtime values.
<code>cget(name)</code>	Any	Extended to know about state and <code>scroll_enabled</code> ; everything else passes through to the native widget.
<code>enable_scroll()</code>	None	Turns scroll handling back on. Equivalent to <code>configure(scroll_enabled=True)</code> . Safe to call repeatedly.
<code>disable_scroll()</code>	None	Turns scroll handling off — wheel, trackpad, and scrollbar drag. Equivalent to <code>configure(scroll_enabled=False)</code> .
<code>is_scrolling()</code>	bool	The live effective scroll state — the AND of <code>scroll_enabled</code> and state. Distinct from <code>cget("scroll_enabled")</code> ; see above.
<code>get_state()</code>	str	Current state, "normal" or "disabled". Mirrors the same accessor on <code>sCTkFrameLabeledPrimary/Secondary</code> .

Method	Returns	Description
<code>winfo_children(include_private=False)</code>	list	By default, filters out children whose exact class name is "CTkScrollbar", "CTkCanvas", or "Canvas" — internal furniture this widget creates for its own scrolling machinery. Confirmed correct by direct, live testing — printing <code>get_children()</code> alongside the widget's internal <code>_parent_frame.winfo_children()</code> confirmed the real content widgets are found correctly by this method, and are <i>not</i> reachable via <code>_parent_frame</code> at all (they're nested deeper, inside the internal scrolling canvas). Pass <code>include_private=True</code> for the raw, unfiltered list.
<code>get_children()</code>	list	Equivalent to <code>winfo_children(include_private=False)</code> .
<code>get_all_children()</code>	list	Equivalent to <code>winfo_children(include_private=True)</code> .
<code>_finalize_split_bindings()</code>	None	Retained for compatibility; no longer required. Calling this after placement was once mandatory, and calling it after rebuilding content was the way to bind newly-created rows. The debounced <Configure> rebind now handles both automatically. Existing callers are harmless — the underlying toggle is idempotent — but new code shouldn't need it. It respects the current effective state rather than forcing scrolling on.

Platform handling, the nested-frame guard, and how disabling actually blocks scrolling are all documented on the [ScrollBindingMixin](#) page. Read it before changing anything about scroll behavior here — several of the mechanisms look like needless complications and are not.

Theming (sCTkThemes.json)

```
{
  "sCTkScrollableFrame": {
    "border_width": 1.5,
    "border_color": ["#64748B", "#94A3B8"],
    "corner_radius": 8,
    "fg_color": ["#FFFFFF", "#111827"],
    "label_fg_color": "transparent",
    "scrollbar_fg_color": ["#FFFFFF", "#111827"],
    "scrollbar_button_color": ["#64748B", "#4B5563"],
    "scrollbar_button_hover_color": ["#1A4375", "#2471A3"],
    "label_text_color": ["#111827", "#F9FADF"],
    "disabled_map": {
      "border_color": ["#CBD5E1", "#374151"],
      "scrollbar_button_color": ["#CBD5E1", "#1F2937"],
      "scrollbar_button_hover_color": ["#CBD5E1", "#1F2937"],
      "label_text_color": ["#94A3B8", "#64748B"]
    }
  }
}
```

`label_fg_color` is deliberately `"transparent"`, so the internal title-row label blends with the frame's own `fg_color` via CustomTkinter's native parent-to-child color propagation, rather than showing its own distinct background.

disabled_map is required, not optional. Construction raises `KeyError` immediately if `border_color`, `scrollbar_button_color`, or `scrollbar_button_hover_color` is missing from either the top-level block or `disabled_map`. This is the same fail-loud principle used across this project — a theme gap surfaces at construction with a message naming exactly what's missing, rather than being papered over with a guessed color.

The hover color needs a disabled entry because a disabled scrollbar is inert (dragging is blocked), and one that still lit up on hover would falsely advertise itself as draggable. Setting it to the same value as the disabled `scrollbar_button_color`, as above, means it simply doesn't react.

label_text_color dims when disabled. This widget can carry a label, and its heading used to stay at full contrast on a disabled panel — the repaint applied `fg_color`, `border_color` and `label_fg_color` but not the text colour, so `disabled_map.label_text_color` had no effect. Both labelled-frame variants already dimmed theirs; this one now matches.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Note that a disabled scrolling frame is subtle by design — only the border, the scrollbar and the label change. `fg_color` is deliberately absent from `disabled_map`, so the panel background stays put and the contents carry the signal.

Only the keys that genuinely change when disabled are required in `disabled_map`. `fg_color` is deliberately **not** among them: the content background stays put when disabled, and the border and the now-inert scrollbar carry the visual signal on their own.

Validation is scoped to direct construction. A subclass inheriting this class (such as `sCTkTableview`) reaches this constructor with `final_kw` built from *its own* theme block — `ThemeableWidget`'s run-once guard means the parent never rebuilds it. Validating this widget's keys against a subclass's block would demand scrollbar colors from, say, the `sCTkTableview` block and raise on every construction. Subclasses own their own theme contract and validate it themselves, so this check runs only for the concrete class.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family.

Runtime color overrides persist. `configure()` records any of the tracked theme keys — `fg_color`, `border_color`, `label_fg_color`, `scrollbar_button_color`, `scrollbar_button_hover_color` — into the widget's stored defaults *before* repainting, so an override survives the repaint, later state changes, and appearance-mode switches. This matches CustomTkinter's own semantics, where `configure(fg_color=...)` sticks.

Two consequences worth knowing. Passing a single color replaces the theme's `(light, dark)` tuple for that key, so **that property stops following light/dark** — which is what asking for one specific color means. And `disabled_map` still wins while disabled: an override sets the *normal*-state color.

`scroll_enabled` is deliberately excluded from this write-back, so the Pygubu query can report construction-time default and live value separately.

Safe to use as a base class for your own composite widgets. If you build a composite widget by inheriting `sCTkScrollableFrame` directly, construction is protected on two fronts: a run-once guard in `ThemeableWidget.__init__` stops your composite's own `final_kw` from being silently overwritten if your widget explicitly calls `ThemeableWidget.__init__` before `super().__init__()`; and this widget's own constructor only

forwards the specific keys native `CTkScrollableFrame` actually accepts (confirmed directly against CustomTkinter's source, which has no fallback `**kwargs` at all — every parameter is explicitly named, so this matters more here than for most widgets).

Example

```
from scustomtkinter import sCTk, sCTkButtonPrimary, sCTkEntryPrimary, sCTkScrollableFrame

if __name__ == "__main__":
    root = sCTk()
    root.title("ScrollableFrame Example")
    root.geometry("450x420")

    log_viewport = sCTkScrollableFrame(root, width=380, height=250, label_text="Telemetry Log")
    log_viewport.pack(padx=20, pady=20, fill="both", expand=True)

    for i in range(12):
        entry = sCTkEntryPrimary(log_viewport, placeholder_text=f"Channel {i + 1}")
        entry.pack(padx=10, pady=5, fill="x")

    # No activation call needed -- scrolling is live as soon as the widget
    # is placed.

    def toggle_lock():
        target = "disabled" if log_viewport.get_state() == "normal" else "normal"
        log_viewport.configure(state=target)
        toggle_btn.configure(text="Enable All" if target == "disabled" else "Disable All")

        # Disabling the frame dims it and stops its scrolling, but does NOT
        # cascade to children -- do that explicitly.
        for child in log_viewport.get_children():
            if hasattr(child, "configure"):
                try:
                    child.configure(state=target)
                except Exception:
                    pass

    toggle_btn = sCTkButtonPrimary(root, text="Disable All", command=toggle_lock)
    toggle_btn.pack(side="bottom", pady=15)

    root.mainloop()
```

Known Limitations

- **Disabling does not cascade to children.** The frame dims and stops scrolling, but child widgets are unaffected — disabling their content is the caller's responsibility, as shown in the example above.
- **The scrollbar is made inert, not hidden.** When disabled it can't be dragged and doesn't respond to hover, but it stays visible. CustomTkinter's scrollbar has no native disabled state to lock, so there's no greyed-out appearance to switch to either. Hiding it entirely is a separate technique, used elsewhere in this project (`sCTkFrameLabeledPrimary / Secondary`) via color-matching and zero width.
- **`wininfo_children()`'s default filtering is a class-name check, not an identity check** — a plain, un-themed `customtkinter.CTkCanvas / CTkScrollbar / Canvas` added directly as a real child (not internal furniture) would be incorrectly filtered out too, since its class name matches. Themed `sCTk`-prefixed widgets are unaffected.
- **`_parent_frame`'s `width / height` don't reflect the real configured size** — confirmed by direct testing: reading `width / height` through the outer widget correctly returns the real value, but the same properties read through the internal `_parent_frame` attribute always report `0`, regardless of the widget's actual size. `fg_color`, `border_color`, and `border_width` are reliable through either path; `width / height` are not. There's no current code path in this widget that relies on `_parent_frame` for sizing, so this is a trap for future changes, not an active bug.

- **The debounced rebind also runs on genuine resizes.** `<Configure>` doesn't distinguish "a child was added" from "the window was dragged", so resizing rebinds too. It's one coalesced pass rather than one per event, but on a very large content tree it is not free.
- **The nested-frame boundary guard is reasoned, not yet live-tested.** The logic mirrors native CustomTkinter's own guard and is straightforward, but an actual nested scrollable frame (or an `sCTkSelector` / `sCTkTableview` placed inside another scrollable frame) hasn't been exercised against it yet.
- **A separate `Canvas` + scrollbar placed inside this frame is not guarded.** The nested-frame boundary check keys on `CTkScrollableFrame` specifically. An independent scrolling region built directly on a plain `Canvas` would still be walked into and bound to this frame's handler, stacking an unwanted scroll behavior on top of its own. Guarding this would need an explicit opt-out convention, since a plain `Canvas` has no generic way to declare itself an independent scroll region.
- **Single-argument color queries return `str(value)`**, where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. A known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.

[Return to Table of Contents](#)

Controls and Display

These are the basic everyday widgets that you will use frequently. There are some additional selections below where we document the extra widgets included in sCustomTkinter.

sCTkButtonPrimary

Table of Contents

- [Overview](#)
- [Constructor](#)
- [Methods](#)
- [Theming \(sCTkThemes.json\)](#)
- [Example](#)
- [Known Limitations](#)

Overview

sCTkButtonPrimary is a themeable subclass of customtkinter.CTkButton — the most prominent of the library's three button tiers (see also sCTkButtonSecondary, sCTkButtonTertiary). It adds automatic light/dark theme resolution from sCTkThemes.json, a four-state visual model (not just enabled/disabled, but also pressed and alarm), and Pygubu Designer property introspection.



Constructor

```
sCTkButtonPrimary(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.

Parameter	Type	Default	Description
**kwargs	—	—	Any native CTKButton argument (e.g. text, command, width, height, font, corner_radius), or an override for one of the theme keys listed under Theming . Anything not supplied falls back to the sCTkButtonPrimary block of sCTkThemes.json. Unlike sCTkComboBox/sCTkSegmentedButton, there's no special extraction step here — command and every other native argument flow straight through to construction.

```

save_button = sCTkButtonPrimary(
    master=control_panel,
    text="Save Changes",
    command=on_save_clicked,
)
save_button.pack(fill="x", padx=40, pady=10)

```

Methods

Method	Returns	Description
state(mode=None)	str	Gets or sets the widget's enabled/disabled state. Only the literal string "disabled" (case-insensitive) disables it; "normal", "enabled", or "active" all enable it. Any other value matches neither branch and leaves the state unchanged (no error raised). Disabling correctly blocks both clicks and hover color changes — confirmed by direct, repeated testing.
get_state()	str	Equivalent to calling state() with no argument.
set_pressed(pressed)	None	Forces the visual "pressed" look on or off. No-op while disabled or while in alarm state.
set_alarm_state(active)	None	Forces a high-visibility warning/alarm look on or off. No-op while disabled. Turning alarm on clears any active "pressed" state, since alarm takes visual precedence — see Theming for the full precedence order.

Method	Returns	Description
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> routes to <code>state()</code> rather than the native option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for <code>state</code> , <code>fg_color</code> , <code>border_color</code> , <code>text_color</code> , and <code>hover_color</code> — with current reflecting whichever state (disabled/alarm/pressed/normal) is presently active. Queries for any other single property name fall through to the native <code>CTkButton.configure</code> .

Theming (sCTkThemes.json)

Four visual states, not two, with a fixed precedence when more than one could apply: **disabled > alarm > pressed > normal**. Only the highest-precedence active state's colors are ever shown — e.g. a button that's both "pressed" and in "alarm" shows alarm colors, and setting alarm while pressed automatically clears the pressed flag.

- **Applied once, at construction** — every key in the widget's theme block, including `width`, `height`, `font`, and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every state change** — `fg_color`, `hover_color`, `text_color`, and `border_color` are recomputed from whichever map matches the current precedence every time you call `state()`, `set_pressed()`, or `set_alarm_state()`. `border_width`, `corner_radius`, and `font` are **not** re-applied on state changes — they don't vary between states, so they're set once at construction and left alone.

```
{
  "sCTkButtonPrimary": {
    "width": 140,
    "height": 34,
    "font": ["Arial", 15, "normal"],
    "fg_color": ["#1A4375", "#2471A3"],
    "hover_color": ["#112A4B", "#1F618D"],
    "text_color": ["#FFFFFF", "#FFFFFF"],
    "corner_radius": 6,
    "disabled_map": {
      "fg_color": ["#E5E7EB", "#374151"],
      "hover_color": ["#E5E7EB", "#374151"],
      "text_color": ["#94A3B8", "#64748B"]
    },
    "pressed_map": {
      "fg_color": ["#3B5984", "#2E4A75"],
      "hover_color": ["#3B5984", "#2E4A75"],
      "text_color": ["#FFFFFF", "#FFFFFF"]
    },
    "alarm_map": {
      "fg_color": ["#990000", "#E74C3C"],
      "hover_color": ["#990000", "#E74C3C"],
      "text_color": ["#FFFFFF", "#FFFFFF"]
    }
  }
}
```

Note there's no `border_color` anywhere in this block — this button style has no themed border by design (it's a solid-fill button). The widget checks for `border_color` in every state's color swap for consistency with the other themed widgets, but that lookup always resolves to nothing here and is simply skipped.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw (light, dark) tuples rather than resolved to a single value ahead of time, the same approach already confirmed working on `sCTkComboBox` and `sCTkSegmentedButton` — so they should correctly follow system/app appearance-mode changes automatically. That specific behavior hasn't been separately re-confirmed for this widget's light/dark toggle, only for its disable/enable cycle.

Disabling this button uses CustomTkinter's native `state="disabled"`, not a manual workaround — that distinction matters here specifically because an earlier version of this widget instead manually unbound mouse events while leaving the native state at `"normal"`, and that approach was directly tested and found to **not** actually block clicks. Native `state="disabled"` is what's required.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("ButtonPrimary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    save_button = sCTkButtonPrimary(base, text="Save", command=lambda: print("Saved!"))
    save_button.pack(pady=10)

    def toggle_alarm():
        save_button.set_alarm_state(not save_button.is_alarm)

    alarm_toggle = sCTkButtonPrimary(base, text="Toggle Alarm Look", command=toggle_alarm)
    alarm_toggle.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if save_button.get_state() == "normal" else "normal"
        save_button.state(target)
        disable_toggle.configure(text="Enable Save" if target == "disabled" else "Disable Save")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Save", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"/"enabled"/"active"`; any other value (including typos) matches neither branch and silently leaves the state unchanged. No exception is raised.
- **Fixed:** single-argument queries used to return `str(value)` of a (light, dark) tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` is supported and merges into the update; a positional property-name string returns the Tkinter-style query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

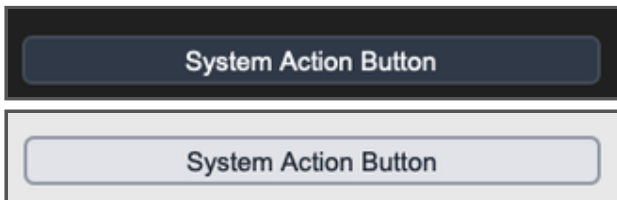
sCTkButtonSecondary

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkButtonSecondary is a themeable subclass of `customtkinter.CTkButton` — a lower-emphasis sibling of `sCTkButtonPrimary` (see also `sCTkButtonTertiary`). It adds automatic light/dark theme resolution from `sCTkThemes.json`, a three-state visual model (normal, disabled, and pressed — no "alarm" state, unlike Primary), and Pygubu Designer property introspection.



Constructor

```
sCTkButtonSecondary(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
**kwargs	—	—	Any native CTkButton argument (e.g. text, command, width, height, font, corner_radius), or an override for one of the theme keys listed under Theming . As with sCTkButtonPrimary, there's no special extraction step — command and every other native argument flow straight through to construction.

```
cancel_button = sCTkButtonSecondary(
    master=control_panel,
    text="Cancel",
    command=on_cancel_clicked,
)
cancel_button.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's enabled/disabled state. Only the literal string "disabled" (case-insensitive) disables it; "normal", "enabled", or "active" all enable it. Any other value matches neither branch and leaves the state unchanged. Disabling uses CTK's native <code>state="disabled"</code> , confirmed by direct testing to correctly block both clicks and hover color changes.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>set_pressed(pressed)</code>	<code>None</code>	Forces the visual "pressed" look on or off. No-op while disabled.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> routes to <code>state()</code> rather than the native option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for <code>state</code> , <code>fg_color</code> , <code>border_color</code> , <code>text_color</code> , and <code>hover_color</code> , with <code>current</code> reflecting whichever state (disabled/pressed/normal) is presently active. Queries for any other property name fall through to the native <code>CTkButton.configure</code> .

Theming (sCTkThemes.json)

Three visual states, with precedence **disabled** > **pressed** > **normal** when both could apply.

- **Applied once, at construction** — every key in the widget's theme block, including `font` and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every state change** — `fg_color`, `hover_color`, `border_color`, and `text_color` are recomputed from whichever map matches the current state every time you call `state()` or `set_pressed()`. `border_width`, `corner_radius`, and `font` are **not** re-applied on state changes — they don't vary between states.

```
{
  "sCTkButtonSecondary": {
    "font": ["Arial", 15, "normal"],
    "fg_color": ["#E5E7EB", "#374151"],
    "hover_color": ["#D1D5DB", "#4B5563"],
    "text_color": ["#1F2937", "#F9FAFB"],
    "border_width": 2,
    "border_color": ["#9CA3AF", "#4B5563"],
    "corner_radius": 6,
    "disabled_map": {
      "fg_color": ["#F3F4F6", "#1F2937"],
      "hover_color": ["#F3F4F6", "#1F2937"],
      "border_color": ["#E5E7EB", "#374151"],
      "text_color": ["#94A3B8", "#64748B"]
    },
    "pressed_map": {
      "fg_color": ["#CBD5E1", "#1F2937"],
      "hover_color": ["#CBD5E1", "#1F2937"],
      "border_color": ["#475569", "#94A3B8"],
      "text_color": ["#0F172A", "#FFFFFF"]
    }
  }
}
```

Unlike `sCTkButtonPrimary` (which has no themed border at all, being a solid-fill button), this style does define `border_color` at every tier — normal, pressed, and disabled all have their own distinct border color.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw (light, dark) tuples rather than resolved to a single value ahead of time, so they correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and `sCTkButtonPrimary`.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonSecondary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("ButtonSecondary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    cancel_button = sCTkButtonSecondary(base, text="Cancel", command=lambda: print("Cancelled"))
    cancel_button.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if cancel_button.get_state() == "normal" else "normal"
        cancel_button.state(target)
        disable_toggle.configure(text="Enable Cancel" if target == "disabled" else "Disable Cancel")

    disable_toggle = sCTkButtonSecondary(base, text="Disable Cancel", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes "disabled" and "normal"/"enabled"/"active"; any other value (including typos) matches neither branch and silently leaves the state unchanged.
- **Fixed:** single-argument queries used to return `str(value)` of a (light, dark) tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for four specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

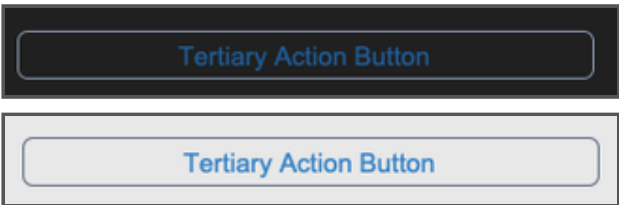
sCTkButtonTertiary

Table of Contents

- Overview
 - Constructor
 - Methods
 - Theming (sCTkThemes.json)
 - Example
 - Known Limitations
-

Overview

`sCTkButtonTertiary` is a themeable subclass of `customtkinter.CTkButton` — the lowest-emphasis of the library's three button tiers (see also `sCTkButtonPrimary`, `sCTkButtonSecondary`), styled as an outline button: border and text only, no filled background. It adds automatic light/dark theme resolution from `sCTkThemes.json`, a three-state visual model (normal, disabled, pressed), and Pygubu Designer property introspection.



Constructor

```
sCTkButtonTertiary(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
**kwargs	—	—	Any native CTKButton argument, or an override for one of the theme keys listed under Theming . No special extraction step — command and every other native argument flow straight through to construction.

```
learn_more_button = sCTkButtonTertiary(  
    master=control_panel,  
    text="Learn More",  
    command=on_learn_more_clicked,  
)  
learn_more_button.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
state(mode=None)	str	Gets or sets the widget's enabled/disabled state. Only "disabled" (case-insensitive) disables it; "normal", "enabled", or "active" all enable it. Any other value leaves the state unchanged. Uses CTK's native state="disabled", confirmed by direct testing to correctly block clicks and hover color changes.
get_state()	str	Equivalent to calling state() with no argument.
set_pressed(pressed)	None	Forces the visual "pressed" look on or off. No-op while disabled.
configure(**kwargs) / config(**kwargs)	varies	Standard widget configuration, plus: passing state=... routes to state() rather than the native option; calling configure("propname") with a single property name returns a Tkinter-style query tuple for state, fg_color, border_color, text_color, and hover_color. Queries for any other property name fall through to the native CTKButton.configure.

Theming (sCtkThemes.json)

Three visual states, with precedence **disabled > pressed > normal**.

```
{
  "sCtkButtonTertiary": {
    "font": ["Arial", 15, "normal"],
    "fg_color": "transparent",
    "text_color": ["#3B8ED0", "#1F6AA5"],
    "corner_radius": 6,
    "border_width": 1.25,
    "border_color": ["#64748B", "#94A3B8"],
    "hover_color": ["#E2E8F0", "#1E293B"],
    "disabled_map": {
      "border_color": ["#E5E7EB", "#374151"],
      "text_color": ["#94A3B8", "#64748B"]
    },
    "pressed_map": {
      "fg_color": ["#E2E8F0", "#1E293B"],
      "border_color": ["#112A4B", "#1F618D"],
      "text_color": ["#112A4B", "#1F618D"]
    }
  }
}
```

A few design decisions specific to this outline style, worth knowing before editing this block:

- **fg_color** is the literal string **"transparent"**, not a color pair — this is a border-and-text-only button by design.
- **disabled_map** has no **fg_color** or **hover_color** entries, **deliberately**. Since only keys present in a map get swapped, omitting these means the button stays transparent when disabled instead of gaining an unwanted solid gray fill — a filled button (Primary/Secondary) wants that fill; this one doesn't.
- **pressed_map** has no **hover_color** entry either. Rather than leaving hover color unset while pressed, the widget explicitly falls back to the normal-state **hover_color** in that case.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw (light, dark) tuples rather than resolved to a single value ahead of time, so they correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCtkComboBox`, `sCtkSegmentedButton`, and `sCtkButtonPrimary`.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonTertiary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("ButtonTertiary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    learn_more_button = sCTkButtonTertiary(base, text="Learn More", command=lambda: print("Clicked"))
    learn_more_button.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if learn_more_button.get_state() == "normal" else "normal"
        learn_more_button.state(target)
        disable_toggle.configure(text="Enable" if target == "disabled" else "Disable")

    disable_toggle = sCTkButtonTertiary(base, text="Disable Learn More", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"/"enabled"/"active"`; any other value (including typos) silently leaves the state unchanged.
- **Fixed:** single-argument queries used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for four specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

sCTkCheckBox

Table of Contents

- Overview
 - Constructor
 - Methods
 - Theming (sCTkThemes.json)
 - Example
 - Known Limitations
-

Overview

sCTkCheckBox is a themeable subclass of `customtkinter.CTkCheckBox`. It adds automatic light/dark theme resolution from `sCTkThemes.json`, including the checkmark color itself, and a distinct enabled/disabled visual state.

Constructor

`sCTkCheckBox(master=None, **kwargs)`

Parameter	Type	Default	Description
master	widget	None	Parent container.
**kwargs	—	—	Any native CTKCheckBox argument (e.g. text, command, variable, onvalue, offvalue), or an override for one of the theme keys listed under Theming . No special extraction step — everything flows straight through to construction.

```
agree_checkbox = sCTkCheckBox(
    master=form_panel,
    text="I agree to the terms",
    command=on_agreement_toggled,
)
agree_checkbox.pack(anchor="w", padx=20, pady=10)
```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	str	Gets or sets the widget's enabled/disabled state. Only "disabled" (case-insensitive) disables it; "normal", "enabled", or "active" all enable it. Any other value leaves the state unchanged.
<code>get_state()</code>	str	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> routes to <code>state()</code> rather than the native option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style query tuple for state, fg_color, border_color, text_color, hover_color, and checkmark_color. Queries for any other property name fall through to the native <code>CTKCheckBox.configure</code> .

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.

- **Re-applied on every state() change** — `fg_color`, `border_color`, `hover_color`, `text_color`, `checkmark_color`, `border_width`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
  "sCTkCheckBox": {
    "font": ["Arial", 15, "normal"],
    "border_width": 3,
    "border_color": ["#64748B", "#94A3B8"],
    "fg_color": ["#1A4375", "#2471A3"],
    "hover_color": ["#112A4B", "#1F618D"],
    "text_color": ["#1F2937", "#D1D5DB"],
    "checkmark_color": ["#FFFFFF", "#FFFFFF"],
    "disabled_map": {
      "text_color": ["#94A3B8", "#64748B"],
      "fg_color": ["#E5E7EB", "#374151"],
      "border_color": ["#CBD5E1", "#475569"],
      "checkmark_color": ["#94A3B8", "#64748B"]
    }
  }
}
```

`checkmark_color` was added as a real theme key during this project's audit — previously the theme block didn't define it at all, so the checkmark silently always used `CTkCheckBox`'s native default color regardless of what theme was active, even though the widget's own code was already set up to read it. When disabled, the checkmark dims to match `disabled_map.text_color`'s gray rather than staying bright against a grayed-out box.

Note there's no `hover_color` entry in `disabled_map` — the widget's native disabled state is expected to suppress hover interaction entirely (consistent with the same behavior confirmed on other themed widgets in this project), so a disabled-specific hover color was judged unnecessary; this hasn't been independently re-confirmed for this specific widget.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and `sCTkButtonPrimary`, though not separately re-confirmed for this widget's light/dark toggle specifically.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkCheckBox, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("CheckBox Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    agree_checkbox = sCTkCheckBox(
        base, text="I agree to the terms", command=lambda: print("Toggled")
    )
    agree_checkbox.pack(anchor="w", pady=10)

    def toggle_disabled():
        target = "disabled" if agree_checkbox.get_state() == "normal" else "normal"
        agree_checkbox.state(target)
        disable_toggle.configure(text="Enable" if target == "disabled" else "Disable")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Checkbox", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes "disabled" and "normal"/"enabled"/"active"; any other value (including typos) silently leaves the state unchanged.
- **Fixed:** single-argument queries used to return `str(value)` of a (light, dark) tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.
- Color reapplication after a `state()` change is deferred by one event-loop tick (`after_idle`), as a precaution carried over from a confirmed race condition on `sCTkButtonPrimary`. In virtually all normal usage this is imperceptible, but code that inspects colors in the same tick as a `state()` call may see the pre-change values.

[Return to Table of Contents](#)

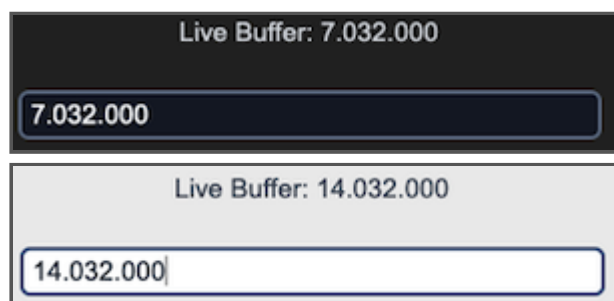
sCTkEntryPrimary

Table of Contents

- [Overview](#)
- [Constructor](#)
- [Methods](#)
- [Theming \(sCTkThemes.json\)](#)
- [Example](#)
- [Known Limitations](#)

Overview

`sCTkEntryPrimary` is a themeable subclass of `customtkinter.CTkEntry` — the higher-emphasis of the library's two entry-field tiers (see also `sCTkEntrySecondary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a genuine three-state visual model: normal, readonly, and disabled.



All three states use CTK's native `state` option (`"normal"`, `"readonly"`, `"disabled"`). `normal/disabled` are confirmed correct by direct testing, consistent with every other widget in this library. `readonly` was added specifically to support `sCTkSpinbox`'s own readonly mode correctly (its entry can't be typed into directly, but the increment/decrement arrows stay clickable) — matching real `ttk.Spinbox` semantics, which distinguish readonly (arrows still work) from disabled (nothing works). Confirmed directly against CustomTkinter's own source: native `CTkEntry` already has full, deliberate support for a `"readonly"` state distinct from `"disabled"` — including a placeholder-text rule worth knowing about (see [Known Limitations](#)).

Constructor

```
sCTkEntryPrimary(master=None, **kwargs)
```

Parameter	Type	Default	Description
<code>master</code>	widget	None	Parent container.
<code>**kwargs</code>	—	—	Any native <code>CTkEntry</code> argument (e.g. <code>placeholder_text</code> , <code>width</code>), or an override for one of the theme keys listed under Theming . <code>state</code> is extracted and applied after construction rather than passed to the native constructor.

```
freq_entry = sCTkEntryPrimary(
    master=control_panel,
    placeholder_text="Enter frequency (MHz)",
)
freq_entry.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
<code>state(state_string=None)</code>	str	Gets or sets the widget's visual state. "normal"/"enabled"/"active" all map to "normal"; "readonly" maps to "readonly"; "disabled" maps to "disabled". All three use CTK's native state option.
<code>get_state()</code>	str	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> routes to <code>state()</code> rather than the native option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for <code>state</code> , <code>fg_color</code> , <code>text_color</code> , <code>border_color</code> , and <code>placeholder_text_color</code> . Queries for any other property name fall through to the native <code>CTkEntry.configure</code> .

insert() and delete() work while disabled. A disabled Tk entry silently ignores both — no error, no text. That matters because generated code sets the state before the content:

```
entry1.configure(state="disabled")
entry1.insert(0, "read only value")
```

which produced a permanently empty field. The native state is lifted for the write and restored afterwards, leaving the widget's own state flag untouched so the colours do not flicker.

The reasoning: state describes what the **user** may edit, not what the program may write. Setting text on a disabled field is ordinary.

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block, including `font` and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every state() change** — `fg_color`, `border_color`, `text_color`, and `placeholder_text_color` are recomputed from the theme's normal values, its `disabled_map`, or its `readonly_map` every time you call `state()`.

```

{
  "sCTkEntryPrimary": {
    "font": ["Arial", 15, "normal"],
    "border_width": 1.5,
    "border_color": ["#1A4375", "#64748B"],
    "fg_color": ["#FFFFFF", "#111827"],
    "text_color": ["#1F2937", "#F9FAFB"],
    "placeholder_text_color": ["#94A3B8", "#64748B"],
    "corner_radius": 6,
    "disabled_map": {
      "fg_color": ["#F3F4F6", "#1F2937"],
      "border_color": ["#CBD5E1", "#475569"],
      "text_color": ["#94A3B8", "#64748B"]
    },
    "readonly_map": {
      "fg_color": ["#F8FAFC", "#1F2937"],
      "border_color": ["#64748B", "#94A3B8"],
      "text_color": ["#1F2937", "#F9FAFB"],
      "placeholder_text_color": ["#94A3B8", "#64748B"]
    }
  }
}

```

readonly_map requires all four keys (`fg_color`, `border_color`, `text_color`, `placeholder_text_color`) whenever `readonly` is actually requested — if any are missing, `state("readonly")` raises immediately rather than falling back to a guessed color. This check only runs when `readonly` is used, so existing code that never requests it is unaffected regardless of whether `readonly_map` is present.

The design intent behind the values above: `text_color` in `readonly_map` deliberately matches `normal`'s `text_color` exactly — `readonly` means "you can still read this clearly, you just can't edit it," a different message from `disabled`'s "this is inactive." `border_color` is the primary visual cue distinguishing `readonly` from `normal`, using a muted "locked" tone distinct from both `normal`'s vivid border and `disabled`'s washed-out one.

`placeholder_text_color` is a genuinely distinct, themed value — not a fallback to `text_color`. This follows CustomTkinter's own convention: in the library's stock dark-blue theme, `text_color` is `["gray14", "gray84"]` while `placeholder_text_color` is a visibly more muted `["gray52", "gray62"]`. The value here reuses the muted gray already established throughout this theme file for `disabled` states — deliberate, but worth knowing if you'd rather placeholder text and `disabled` text look distinguishable from each other.

Note: CTKEntry has no separate font for placeholder text — it always shares the single `font` property with typed text. This is a real limitation of the underlying widget, not a gap in this theme file; there's no way to make placeholder text use a different font.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw (`light`, `dark`) tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
#!/usr/bin/python3

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkLabelPrimary, sCTkEntryPrimary

if __name__ == "__main__":

    root = sCTk()
    root.geometry("450x260")
    root.title("sCTkEntryPrimary Testing Deck")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    # Label notice layer to monitor buffer array activity
    lbl_monitor = sCTkLabelPrimary(base, text="Console monitor active...")
    lbl_monitor.pack(pady=10)

    # Instantiate your custom Primary helper field
    input_field = sCTkEntryPrimary(base, placeholder_text="Enter configuration metadata...")
    input_field.pack(expand=False, fill="x", padx=40, pady=10)

    # Monitor keystrokes live
    input_field.bind("<KeyRelease>", lambda e: lbl_monitor.configure(text=f"Live Buffer: {input_field.get()}"))

    def toggle_operational_state():
        """Toggles the helper input field between normal active and dimmed disabled profiles."""
        current_mode = input_field.get_state()
        target = "disabled" if current_mode == "normal" else "normal"

        # Explicitly testing the dual-routing capability via configure()
        input_field.configure(state=target)
        btn_toggle.configure(
            text="Lock Helper Input (Set 'disabled')" if target == "normal" else "Unlock Helper Input (Set 'normal')")
        print(f"Logged Verification Hook -> input_field.get_state() = {input_field.get_state()}")

    btn_toggle = sCTkButtonPrimary(base, text="Lock Helper Input (Set 'disabled')", command=toggle_operational_state)
    btn_toggle.pack(side="bottom", pady=15)

    # Run the interactive boot tracking logs
    print("--- BOOT INITIALIZATION PASSTHROUGH ---")
    input_field.state("disabled")
    print("state (Disabled Pass) =", input_field.get_state()) # Output: disabled

    input_field.state("normal")
    print("state (Normal Pass)  =", input_field.get_state()) # Output: normal
    print("=====\\n")

    root.mainloop()
```

Known Limitations

- `state()` only recognizes "disabled", "readonly", and "normal"/"enabled"/"active"; any other value (including typos) leaves the internal state flag unchanged, though colors are still harmlessly re-applied.

- **readonly never deactivates placeholder text, even on focus** — confirmed directly against CustomTkinter's own source: native `CTkEntry`'s internal placeholder logic explicitly skips clearing the placeholder whenever state is `"readonly"`. This makes sense (there's no reason to clear a placeholder for typing on a field that can't be typed into), but it means a readonly field showing placeholder text will keep showing it indefinitely, regardless of focus.
- The disable/enable-cycle cursor-position fix (`_reset_cursor_if_showing_placeholder`) is also applied on transitions into `readonly`, as a precaution — but this specific transition (unlike normal `disabled`, which is directly confirmed by testing) has not been independently verified. Given the point above, this is likely lower-risk than it might otherwise seem, since a readonly field showing placeholder text stays in that state continuously rather than toggling.
- **Fixed:** `configure("fg_color")` used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

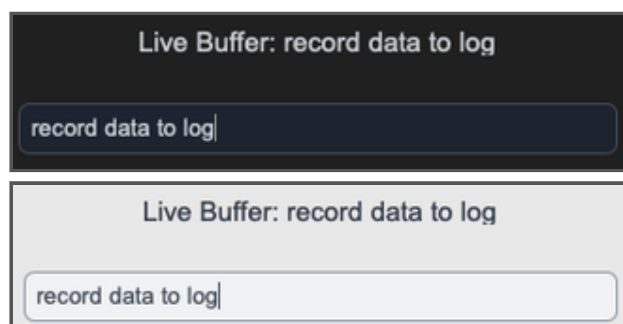
sCTkEntrySecondary

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkEntrySecondary is a themeable subclass of `customtkinter.CTkEntry` — the lower-emphasis of the library's two entry-field tiers (see also `sCTkEntryPrimary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a genuine three-state visual model: normal, readonly, and disabled.



All three states use CTk's native `state` option (`"normal"`, `"readonly"`, `"disabled"`) — see `sCTkEntryPrimary`'s documentation for the full rationale and the readonly-specific placeholder behavior worth knowing about.

Constructor

```
sCTkEntrySecondary(master=None, **kwargs)
```

Parameter	Type	Default	Description
<code>master</code>	<code>widget</code>	<code>None</code>	Parent container.
<code>**kwargs</code>	—	—	Any native <code>CTkEntry</code> argument, or an override for one of the theme keys listed under Theming . <code>state</code> is extracted and applied after construction rather than passed to the native constructor.

```
notes_entry = sCTkEntrySecondary(
    master=control_panel,
    placeholder_text="Optional notes",
)
notes_entry.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
<code>state(state_string=None)</code>	<code>str</code>	Gets or sets the widget's visual state. "normal"/"enabled"/"active" all map to "normal"; "readonly" maps to "readonly"; "disabled" maps to "disabled". All three use CTK's native state option.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> routes to <code>state()</code> rather than the native option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for state, fg_color, text_color, border_color, and placeholder_text_color. Queries for any other property name fall through to the native <code>CTkEntry.configure</code> .

`insert()` and `delete()` work while disabled. A disabled Tk entry silently ignores both — no error, no text. That matters because generated code sets the state before the content:

```
entry1.configure(state="disabled")
entry1.insert(0, "read only value")
```

which produced a permanently empty field. The native state is lifted for the write and restored afterwards, leaving the widget's own state flag untouched so the colours do not flicker.

The reasoning: state describes what the **user** may edit, not what the program may write. Setting text on a disabled field is ordinary.

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block, including `font` and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `border_color`, `text_color`, and `placeholder_text_color` are recomputed from the theme's normal values, its `disabled_map`, or its `readonly_map` every time you call `state()`.

```

{
  "sCTkEntrySecondary": {
    "font": ["Arial", 13, "normal"],
    "border_width": 1,
    "border_color": ["#9CA3AF", "#4B5563"],
    "fg_color": ["#F3F4F6", "#1F2937"],
    "text_color": ["#4B5563", "#D1D5DB"],
    "placeholder_text_color": ["#94A3B8", "#64748B"],
    "corner_radius": 6,
    "disabled_map": {
      "fg_color": ["#F3F4F6", "#0B0F19"],
      "border_color": ["#CBD5E1", "#374151"],
      "text_color": ["#94A3B8", "#64748B"]
    },
    "readonly_map": {
      "fg_color": ["#F3F4F6", "#1F2937"],
      "border_color": ["#64748B", "#6B7280"],
      "text_color": ["#4B5563", "#D1D5DB"],
      "placeholder_text_color": ["#94A3B8", "#64748B"]
    }
  }
}

```

readonly_map requires all four keys whenever `readonly` is actually requested — see `sCTkEntryPrimary`'s docs for the full requirement and design rationale (`text_color` deliberately matches normal exactly; `fg_color` stays close to normal too, since Secondary's normal state is already fairly subtle and there wasn't much room to differentiate further without it starting to look disabled instead).

Same rationale as `sCTkEntryPrimary`: `placeholder_text_color` is a genuinely distinct, themed value, following CustomTkinter's own convention of giving placeholder text a visibly more muted color than typed text. `CTkEntry` has no separate font for placeholder text — it always shares the single `font` property with typed text; that's a real limitation of the underlying widget, not a gap in this theme file.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw (`light`, `dark`) tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkEntrySecondary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("EntrySecondary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    notes_entry = sCTkEntrySecondary(base, placeholder_text="Optional notes")
    notes_entry.pack(fill="x", pady=10)

    def toggle_disabled():
        target = "disabled" if notes_entry.get_state() == "normal" else "normal"
        notes_entry.state(target)
        disable_toggle.configure(text="Enable Field" if target == "disabled" else "Disable Field")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Field", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes "disabled", "readonly", and "normal"/"enabled"/"active"; any other value leaves the internal state flag unchanged, though colors are still harmlessly re-applied.
- **readonly never deactivates placeholder text, even on focus** — confirmed directly against CustomTkinter's own source; see `sCTkEntryPrimary`'s docs for the full explanation.
- The disable/enable-cycle cursor-position fix is also applied on transitions into `readonly`, as a precaution not independently verified the way `normal`→`disabled` was — see `sCTkEntryPrimary`'s docs for why this is likely lower-risk than it sounds.
- **Fixed:** `configure("fg_color")` used to return `str(value)` of a (light, dark) tuple rather than a resolved colour. The shared query helper now resolves the pair.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

sCTkLabelPrimary

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkLabelPrimary is a themeable subclass of `customtkinter.CTkLabel` — the most prominent of the library's three label tiers (see also `sCTkLabelSecondary`, `sCTkLabelTertiary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state. Since labels have no native interactivity to block, "disabled" here is purely a text-color dim — there's no click-blocking concern the way there is for buttons or checkboxes.




Constructor

```
sCTkLabelPrimary(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
**kwargs	—	—	state is pulled out and applied after construction rather than passed to the native widget. Everything else is any native <code>CTkLabel</code> argument (e.g. <code>text</code> , <code>font</code>), or an override for one of the theme keys listed under Theming .

```
console_header = sCTkLabelPrimary(
    master=control_panel,
    text="Main Console",
)
console_header.pack(expand=True, pady=10)
```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's enabled/disabled state. Only the literal string "disabled" (case-insensitive) disables it; "normal", "enabled", or "active" all enable it. Any other value matches neither branch and leaves the state unchanged.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>cget("state")</code>	<code>str</code>	Returns the current state, same as <code>get_state()</code> . Intercepted specially because native <code>CTkLabel</code> has no real "state" option to query — without this override, <code>cget("state")</code> would raise.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> routes to <code>state()</code> rather than the native option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for <code>state</code> , <code>fg_color</code> , and <code>text_color</code> . Queries for any other property name fall through to the native <code>CTkLabel.configure</code> .

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
  "sCTkLabelPrimary": {
    "font": ["Arial", 18, "bold"],
    "fg_color": "transparent",
    "text_color": ["#111827", "#F9FAFB"],
    "disabled_map": {
      "text_color": ["#64748B", "#94A3B8"]
    }
  }
}
```

`text_color` is required in whichever map is active — if it's missing, `_update_current_visual_state()` raises immediately rather than falling back to a default. This is deliberate: this project's design is for ThemeableWidget-based widgets to fail hard on incomplete theme data, not paper over it with a fallback color.

Primary's disabled color is intentionally the least muted of the three label tiers — by design, a disabled `sCTkLabelPrimary` should still read as more prominent than a disabled `sCTkLabelSecondary` or `sCTkLabelTertiary`, echoing the hierarchy the three tiers already have when enabled.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkLabelPrimary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("LabelPrimary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    header = sCTkLabelPrimary(base, text="Main Console")
    header.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if header.get_state() == "normal" else "normal"
        header.state(target)
        disable_toggle.configure(text="Enable Header" if target == "disabled" else "Disable Header")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Header", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"/"enabled"/"active"`; any other value (including typos) matches neither branch and silently leaves the state unchanged.
- **Fixed:** single-argument queries used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state/fg_color/text_color`, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

sCTkLabelSecondary

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

`sCTkLabelSecondary` is a themeable subclass of `customtkinter.CTkLabel` — the mid-emphasis tier of the library's three label styles, between `sCTkLabelPrimary` and `sCTkLabelTertiary`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state. Since labels have no native interactivity to block, "disabled" here is purely a text-color dim.

Active Teleceiver Signal Frequency Lane [94.1 MHz]

Active Teleceiver Signal Frequency Lane [94.1 MHz]

Constructor

```
sCTkLabelSecondary(master=None, **kwargs)
```

Parameter	Type	Default	Description
<code>master</code>	widget	None	Parent container.
<code>**kwargs</code>	—	—	state is pulled out and applied after construction rather than passed to the native widget. Everything else is any native <code>CTkLabel</code> argument (e.g. <code>text</code> , <code>font</code>), or an override for one of the theme keys listed under Theming .

```
panel_sub_label = sCTkLabelSecondary(
    master=control_panel,
    text="Status: Active",
)
panel_sub_label.pack(expand=True, pady=10)
```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's enabled/disabled state. Only the literal string "disabled" (case-insensitive) disables it; "normal", "enabled", or "active" all enable it. Any other value matches neither branch and leaves the state unchanged.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>cget("state")</code>	<code>str</code>	Returns the current state, same as <code>get_state()</code> . Intercepted specially because native <code>CTkLabel</code> has no real "state" option to query — without this override, <code>cget("state")</code> would raise.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> routes to <code>state()</code> rather than the native option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for <code>state</code> , <code>fg_color</code> , and <code>text_color</code> . Queries for any other property name fall through to the native <code>CTkLabel.configure</code> .

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
  "sCTkLabelSecondary": {
    "font": ["Arial", 15, "normal"],
    "fg_color": "transparent",
    "text_color": ["#374151", "#D1D5DB"],
    "disabled_map": {
      "text_color": ["#94A3B8", "#64748B"]
    }
  }
}
```

Unlike `sCTkLabelPrimary` / `sCTkLabelTertiary`'s history, this widget's `_update_current_visual_state()` was already never falling back to a default — but it also wasn't raising, either; a missing `text_color` would just silently never get set. As of this project's audit, all three label widgets now raise explicitly if `text_color` is missing from whichever map is active, per this project's design of failing hard on incomplete theme data rather than working around it.

Secondary's disabled color sits deliberately in the middle of the three label tiers' disabled states — less muted than Tertiary's, more muted than Primary's — mirroring the emphasis hierarchy the three tiers already have when enabled.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw (light, dark) tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkLabelSecondary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("LabelSecondary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    status_label = sCTkLabelSecondary(base, text="Status: Active")
    status_label.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if status_label.get_state() == "normal" else "normal"
        status_label.state(target)
        disable_toggle.configure(text="Enable Label" if target == "disabled" else "Disable Label")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Label", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes "disabled" and "normal"/"enabled"/"active"; any other value (including typos) matches neither branch and silently leaves the state unchanged.
- **Fixed:** single-argument queries used to return `str(value)` of a (light, dark) tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state` / `fg_color` / `text_color`, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

sCTkLabelTertiary

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkLabelTertiary is a themeable subclass of customtkinter.CTkLabel — the lowest-emphasis of the library's three label tiers (see also sCTkLabelPrimary, sCTkLabelSecondary), intended for inline descriptions, sub-legends, or auxiliary notices. It adds automatic light/dark theme resolution from sCTkThemes.json and a distinct enabled/disabled visual state. Since labels have no native interactivity to block, "disabled" here is purely a text-color dim.

Inline notice: tuning resolution bounded to 100Hz.

Inline notice: tuning resolution bounded to 100Hz.

Constructor

```
sCTkLabelTertiary(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
**kwargs	—	—	state is pulled out and applied after construction rather than passed to the native widget. Everything else is any native CTkLabel argument (e.g. text, font), or an override for one of the theme keys listed under Theming .

```
panel_legend = sCTkLabelTertiary(
    master=control_panel,
    text="Note: values update every 5 seconds.",
)
panel_legend.pack(expand=True, pady=10)
```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's enabled/disabled state. Only the literal string "disabled" (case-insensitive) disables it; "normal", "enabled", or "active" all enable it. Any other value matches neither branch and leaves the state unchanged.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>cget("state")</code>	<code>str</code>	Returns the current state, same as <code>get_state()</code> . Intercepted specially because native <code>CTkLabel</code> has no real "state" option to query — without this override, <code>cget("state")</code> would raise.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> routes to <code>state()</code> rather than the native option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for <code>state</code> , <code>fg_color</code> , and <code>text_color</code> . Queries for any other property name fall through to the native <code>CTkLabel.configure</code> .

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
  "sCTkLabelTertiary": {
    "font": ["Arial", 13, "normal"],
    "fg_color": "transparent",
    "text_color": ["#4B5563", "#9CA3AF"],
    "disabled_map": {
      "text_color": ["#CBD5E1", "#475569"]
    }
  }
}
```

`text_color` is required in whichever map is active — if it's missing, `_update_current_visual_state()` raises immediately rather than falling back to a default. This is deliberate: this project's design is for `ThemeableWidget`-based widgets to fail hard on incomplete theme data, not paper over it with a fallback color.

Tertiary's disabled color is intentionally the most muted of the three label tiers — by design, a disabled `sCTkLabelTertiary` should read as the least prominent of the three even while disabled, echoing the hierarchy the three tiers already have when enabled.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkLabelTertiary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("LabelTertiary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    legend = sCTkLabelTertiary(base, text="Note: values update every 5 seconds.")
    legend.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if legend.get_state() == "normal" else "normal"
        legend.state(target)
        disable_toggle.configure(text="Enable Legend" if target == "disabled" else "Disable Legend")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Legend", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"/"enabled"/"active"`; any other value (including typos) matches neither branch and silently leaves the state unchanged.
- **Fixed:** single-argument queries used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state/fg_color/text_color`, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

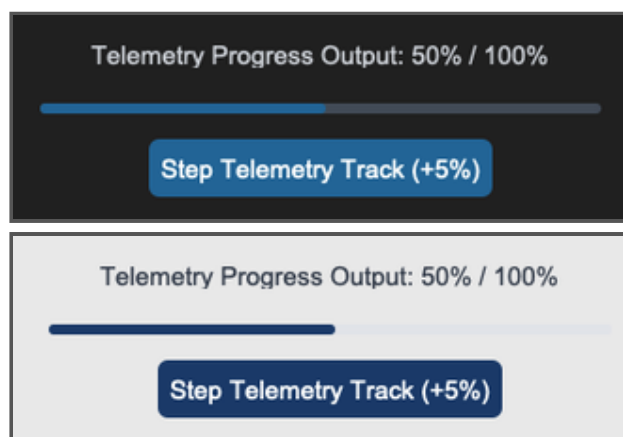
sCTkProgressBar

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkProgressBar is a themeable subclass of `customtkinter.CTkProgressBar`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a purely visual "disabled" state — progress bars have no click behavior to block, so disabling one only dims its colors.



Constructor

```
sCTkProgressBar(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
**kwargs	—	—	state is pulled out and applied after construction. Everything else is any native CTkProgressBar argument, or an override for one of the theme keys listed under Theming .

```
signal_meter = sCTkProgressBar(master=control_panel)
signal_meter.pack(fill="x", padx=40, pady=10)
signal_meter.set(0.65)
```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's visual "disabled" state. Unlike most widgets in this library, any string is accepted and stored as-is (lowercased) — there's no validation against a fixed set of values. Only the literal "disabled" actually changes colors; anything else is treated as "not disabled".
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>cget("state")</code>	<code>str</code>	Returns the current state, same as <code>get_state()</code> . Intercepted specially because native <code>CTkProgressBar</code> has no real "state" option to query — without this override, <code>cget("state")</code> would raise.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> routes to <code>state()</code> rather than the native option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for <code>state</code> , <code>fg_color</code> , <code>progress_color</code> , and <code>border_color</code> . Queries for any other property name fall through to the native <code>CTkProgressBar.configure</code> .

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block, including `width`, `height`, and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `progress_color`, `border_width`, and `corner_radius` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
  "sCTkProgressBar": {
    "width": 200,
    "height": 6,
    "fg_color": ["#E5E7EB", "#4B5563"],
    "progress_color": ["#1A4375", "#2471A3"],
    "corner_radius": 100,
    "disabled_map": {
      "fg_color": ["#CBD5E1", "#374151"],
      "progress_color": ["#94A3B8", "#4B5563"]
    }
  }
}
```

There's no `border_color` anywhere in this theme block, even though the repaint loop checks for one — this style simply has no themed border, the same situation as `sCTkButtonPrimary`'s `border_color`.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkProgressBar, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("ProgressBar Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    meter = sCTkProgressBar(base)
    meter.pack(fill="x", pady=10)
    meter.set(0.65)

    def toggle_disabled():
        target = "disabled" if meter.get_state() == "normal" else "normal"
        meter.state(target)
        disable_toggle.configure(text="Enable Meter" if target == "disabled" else "Disable Meter")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Meter", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` performs no validation at all — any string you pass is stored verbatim; only `"disabled"` actually changes the rendered colors.
- **Fixed:** single-argument queries used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for four specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

sCTkRadioButton

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkRadioButton is a themeable subclass of `customtkinter.CTkRadioButton`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state, using CustomTkinter's native `state="disabled"` — confirmed by direct testing to correctly block clicks.



Constructor

```
sCTkRadioButton(master=None, variable=None, value=None, command=None, **kw)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
variable	tkinter.Variable	None	Shared variable used for mutual exclusion within a group.
value	any	None	This button's value — the button shows as selected when variable equals this.
command	callable	None	Called when the button is selected.
**kw	—	—	Any native CTkRadioButton argument, or an override for one of the theme keys listed under Theming .

```
mode_var = tkinter.StringVar(value="AM")
am_radio = sCTkRadioButton(control_panel, text="AM", variable=mode_var, value="AM")
fm_radio = sCTkRadioButton(control_panel, text="FM", variable=mode_var, value="FM")
am_radio.pack(anchor="w")
fm_radio.pack(anchor="w")
```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's enabled/disabled state. Only "disabled" (case-insensitive) disables it; "normal", "enabled", or "active" all enable it.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: variable/value/command/state are each routed individually. Rebinding variable/value to a new group after construction is confirmed correct by direct testing — it properly tears down the old variable's binding and establishes real mutual exclusion in the new group, with no leftover coupling to the original group. Calling <code>configure("propname")</code> with a single property name returns a Tkinter-style query tuple for <code>state</code> , <code>fg_color</code> , <code>border_color</code> , <code>text_color</code> , and <code>hover_color</code> .

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `border_color`, `hover_color`, `text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
  "sCTkRadioButton": {
    "font": ["Arial", 15, "normal"],
    "text_color": ["#374151", "#D1D5DB"],
    "border_width_unchecked": 4,
    "border_width_checked": 6,
    "border_color": ["#64748B", "#94A3B8"],
    "fg_color": ["#1A4375", "#2471A3"],
    "hover_color": ["#112A4B", "#1F618D"],
    "disabled_map": {
      "text_color": ["#94A3B8", "#64748B"],
      "fg_color": ["#CBD5E1", "#374151"],
      "border_color": ["#CBD5E1", "#475569"]
    }
  }
}
```

`border_width_unchecked` and `border_width_checked` are real, top-level-only theme keys (not in `disabled_map`) that control the button's border thickness based on whether it's currently the selected button in its group — thicker when checked, to show the filled dot. They're applied once at construction and left alone afterward; the native widget switches between them internally based on the checked/unchecked state, so no repaint-time logic is needed here.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import tkinter
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkRadioButton, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("RadioButton Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    mode_var = tkinter.StringVar(value="AM")
    am_radio = sCTkRadioButton(base, text="AM", variable=mode_var, value="AM")
    am_radio.pack(anchor="w", pady=5)
    fm_radio = sCTkRadioButton(base, text="FM", variable=mode_var, value="FM")
    fm_radio.pack(anchor="w", pady=5)

    def toggle_disabled():
        target = "disabled" if am_radio.get_state() == "normal" else "normal"
        am_radio.state(target)
        fm_radio.state(target)
        disable_toggle.configure(text="Enable Group" if target == "disabled" else "Disable Group")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Group", command=toggle_disabled)
    disable_toggle.pack(pady=15)

    root.mainloop()
```

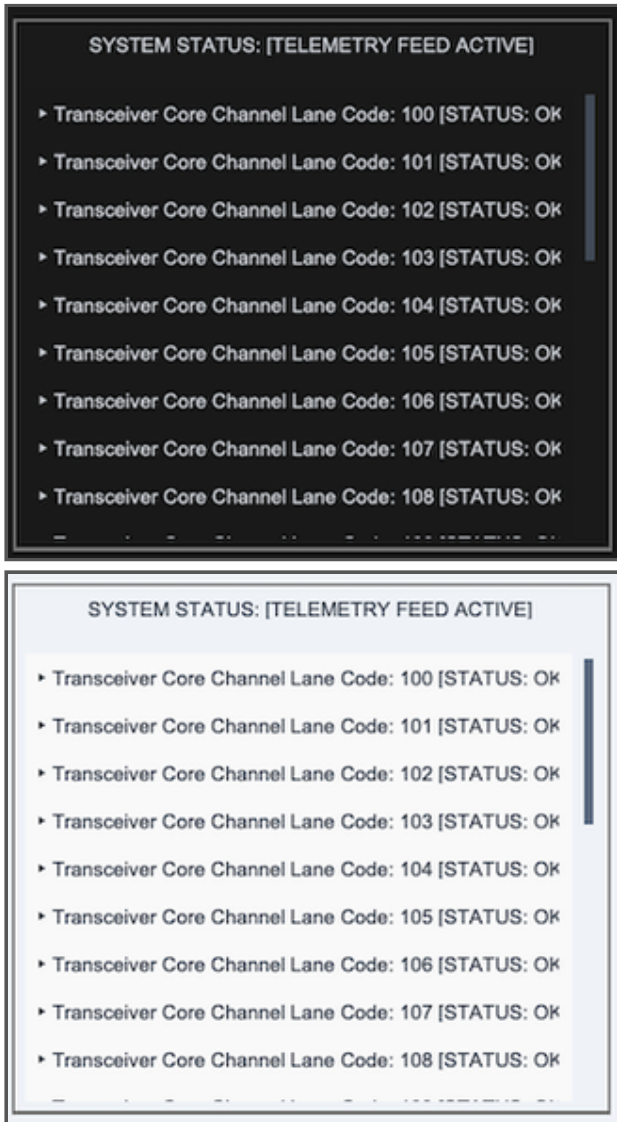
Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"/"enabled"/"active"`; any other value matches neither branch, though colors are still harmlessly re-applied.
- **Fixed:** single-argument queries used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state/fg_color/border_color/text_color/hover_color`, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

sCTkScrollbar

`sCTkScrollbar` is a themeable scrollbar — a subclass of `ctk.CTkScrollbar` with color resolution from `sCTkThemes.json` and orientation-aware default sizing. It's designed to pair with `sCTkScrollArea`, which needs an external scrollbar, but works anywhere a `CTkScrollbar` would.



This widget contains no scroll-handling logic. It's a scrollbar: it renders a draggable bar and reports its position. Wheel and trackpad handling belongs to the scrolling container — see `ScrollBindingMixin`. An earlier version of this page credited the scrollbar with an "inertial micro-delta aggregator"; that logic lives in `sCTkScrollArea`, not here.

Table of Contents

- [Constructor](#)
 - [Methods](#)
 - [Theming](#)
 - [Example](#)
 - [Known Limitations](#)
-

Constructor

```
scrollbar = sCTkScrollbar(master=None, orientation="vertical", **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
orientation	"vertical" / "horizontal"	"vertical"	Layout direction. Sets a default width of 14 when vertical, or height of 14 when horizontal.
**kwargs	—	—	Any native CTkScrollbar argument, or a theme-key override (see Theming).

Methods

Method	Returns	Description
configure(**kwargs) / config(**kwargs)	None	Standard configuration. Overrides of <code>button_color</code> and <code>button_hover_color</code> persist — see below.
configure(name)	tuple	Pygubu-style single-argument query for <code>button_color</code> and <code>button_hover_color</code> . Any other name passes through to the native widget.

Runtime color overrides persist. `configure()` records the tracked theme keys into the widget's stored defaults *before* repainting, so an override survives the repaint and later appearance-mode switches. This matches CustomTkinter's own semantics, where `configure(button_color=...)` sticks.

This was previously broken. `_apply_custom_theme_colors()` runs on every `configure()` call and re-pushes both colors from the stored defaults — and since `configure()` never wrote to those defaults, `configure(button_color="red")` applied red and then had it overwritten on the very next line. Runtime color overrides silently did nothing.

Two consequences worth knowing: passing a single color replaces the theme's (`light`, `dark`) tuple for that key, so **that property stops following light/dark** — which is what asking for one specific color means.

The single-argument query was also previously broken. The implementation tested `if args` and `isinstance(args, dict)`, but `args` is always a tuple, so that branch was dead and there was no query branch at all — `configure("button_color")` silently returned `None` instead of a property tuple.

Theming (sCTkThemes.json)

```
{
  "sCTkScrollbar": {
    "corner_radius": 4,
    "fg_color": "transparent",
    "button_color": ["#64748B", "#4B5563"],
    "button_hover_color": ["#1A4375", "#2471A3"]
  }
}
```

`button_color` is the bar itself; `button_hover_color` is the bar under the cursor. `fg_color` is the track behind it.

button_color and button_hover_color are required. Construction raises `KeyError` naming the missing one. These previously carried hardcoded fallbacks, so a theme block missing either would silently substitute a plausible guess rather than failing loudly.

`orientation` may also be supplied from the theme block. It's read from the resolved keywords rather than the raw constructor dict, so it's picked up whichever way it arrives — an earlier version read the raw dict *after* `ThemeableWidget` had processed it, which risked a horizontal scrollbar silently getting a default `width` instead of `height`.

Colors are passed through as raw `(light, dark)` tuples rather than resolved ahead of time, so they follow appearance-mode changes automatically.

There is no `disabled_map`, and no disabled state. CustomTkinter's scrollbar has none to lock. Containers that need an inert scrollbar block dragging at the binding level instead and dim the bar themselves — see [ScrollBindingMixin](#).

Example

```
#!/usr/bin/python3
import customtkinter as ctk
from scustomtkinter import (sCTk, sCTkFrame, sCTkLabelSecondary,
                             sCTkScrollbar, sCTkScrollArea)

if __name__ == "__main__":
    root = sCTk()
    root.geometry("480x420")
    root.title("sCTkScrollbar Validation Bench")

    main_layout = sCTkFrame(root, border_width=2)
    main_layout.pack(expand=True, fill="both", padx=15, pady=15)

    scrollbar = sCTkScrollbar(main_layout, orientation="vertical")
    scrollbar.pack(side="right", fill="y", padx=(5, 10), pady=10)

    content_chassis = sCTkFrame(main_layout, border_width=0, fg_color="transparent")
    content_chassis.pack(side="left", fill="both", expand=True, padx=(10, 0), pady=10)

    scroll_view = sCTkScrollArea(content_chassis)
    scroll_view.pack(fill="both", expand=True)

    for i in range(25):
        sCTkLabelSecondary(
            scroll_view.scroll_content,
            text=f"Transceiver channel {100 + i} [OK]"
        ).pack(anchor="w", padx=10, pady=4)

    scroll_view.hook_scrollbar(scrollbar)

    root.mainloop()
```

Known Limitations

- **No disabled state** — see [Theming](#).
- **Only `button_color` and `button_hover_color` are tracked** for the `persist-on-configure()` behavior. Other properties still repaint from the theme.

[Return to Table of Contents](#)

sCTkSegmentedButton

Table of Contents

- Overview
 - Constructor
 - Methods
 - Theming (sCTkThemes.json)
 - Example
 - Known Limitations
-

Overview

sCTkSegmentedButton is a themeable subclass of `customtkinter.CTkSegmentedButton` — a horizontal strip of connected text buttons where selecting one automatically unselects the others, similar to a row of radio buttons. It adds automatic light/dark theme resolution from `sCTkThemes.json`, a distinct enabled/disabled visual state, and per-segment text-color handling for the currently selected segment.



Constructor

```
sCTkSegmentedButton(master=None, values=None, variable=None, command=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
values	list[str]	native default (empty)	The segment labels.
variable	tkinter.StringVar	None	Optional variable bound to the current selection.
command	callable	None	Called with the selected value when the user picks a segment.
**kwargs	—	—	Any native CTKSegmentedButton argument, or an override for fg_color / selected_color.

Not settable via constructor: `unselected_color`, `unselected_hover_color`, `border_width`, `border_color`, and `selected_color_padding` are intentionally stripped out before the native widget is built, regardless of what you pass in. They're only ever pulled from `sCTkThemes.json` — see [Known Limitations](#) for why setting them later doesn't stick either.

```
mode_selector = sCTkSegmentedButton(
    master=control_panel,
    values=["Alpha", "Beta", "Gamma"],
    command=on_mode_changed,
)
mode_selector.pack(fill="x", padx=40, pady=10)
```

Colors are first applied roughly 15ms after construction, not immediately — the widget's individual segment buttons don't exist yet at the moment `sCTkSegmentedButton.__init__` returns, so the initial color pass is deferred with `self.after(15, ...)`. In virtually all normal usage this is unnoticeable, but code that inspects a segment's color in the same tick as construction may see it before this pass runs.

Methods

Method	Returns	Description
<code>get()</code>	str	Currently selected value (native CTKSegmentedButton behavior).
<code>set(value)</code>	None	Selects a segment programmatically and repaints (native <code>set()</code> , plus a theme-color refresh).

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code> <code>None</code>	Gets or sets the widget's enabled/disabled visual state. Only the literal string "disabled" (case-insensitive) is treated as disabled — any other value, including typos, is treated as "normal" with no error raised. Called with no argument, returns the current state as a lowercase string.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) /</code> <code>config(**kwargs)</code>	—	Standard widget configuration. Passing <code>state=...</code> routes through <code>state()</code> rather than the native state option. A single positional argument (string or dict) is accepted, but — unlike <code>sCTkComboBox/sCTkCheckBox</code> — a single property-name string does not return a Pygubu-style query tuple here; it's forwarded directly to the native widget instead. (Broader Pygubu query-behavior gaps across the library are a known follow-up item, not specific to this widget.)

Theming (`sCTkThemes.json`)

Same two-tier model as the other themed widgets:

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built, *except* the five keys listed under [Constructor](#), which are deliberately excluded and applied only through the mechanism below.
- **Re-applied on `state()` change, on selection, and on click** — `fg_color`, `selected_color`, `unselected_color`, `unselected_hover_color`, and each segment's `text_color` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`, `set()`, or click a segment.

Colors are stored and passed through as raw (`light`, `dark`) tuples rather than being resolved to a single value ahead of time, which means they correctly follow system/app appearance-mode changes automatically — including while the widget is disabled. Confirmed by direct testing: toggling light/dark mode on a disabled, pre-selected widget repaints it immediately, with no manual intervention needed.

```
{
  "sCTkSegmentedButton": {
    "fg_color": ["#4F75A2", "#2B4C7E"],
    "font": ["Arial", 13, "normal"],
    "font": ["Arial", 13, "normal"],
    "selected_color": ["#1A4375", "#3A6FA2"],
    "unselected_hover_color": ["#3A5C85", "#3A5F8C"],
    "text_color": ["#FFFFFF", "#FFFFFF"],
    "disabled_map": {
      "fg_color": ["#B2B9BC", "#222527"],
      "selected_color": ["#70777B", "#45494D"],
      "text_color": ["#94A3B8", "#64748B"],
      "selected_text_color": ["#1F2937", "#FFFFFF"]
    }
  }
}
```

A couple of design decisions worth knowing if you're editing this block:

- **Unselected segments always match `fg_color`, by design** — there's no independent `unselected_color` key. Segments are meant to blend into the widget's own background color rather than appear individually distinct or transparent.
- **Hover is fully suppressed while disabled**, at the native widget level — confirmed by direct testing. There's no `disabled_map` entry for hover colors because a disabled segment never fires a hover event in the first place; any color set there would never be visible.
- **The selected segment keeps a distinct, more prominent text color while disabled** (`disabled_map.selected_text_color`), so you can still tell which option was chosen even though the whole control is grayed out. Every other segment's disabled text color comes from the plain `disabled_map.text_color`.

Every key above is required, at the top level or in `disabled_map` as shown. Construction raises `KeyError` naming the missing one.

The colour lookups previously carried hardcoded fallbacks. Those were unreachable given the theme block as shipped — every key they guarded was present — but that was a property of the *theme file*, not the code. Deleting a key would have silently activated a fallback, producing a plausible-looking wrong colour instead of the loud failure every other widget now gives. They're gone.

`selected_hover_color` has also been removed from the block. It was present in the theme but read by no code path at all — dead data, the same situation `pointer_color` was in for the dial family. A selected segment's hover behaviour comes from CustomTkinter's own default, since this widget never set it.

Every color in this widget comes from `sCTkThemes.json` — there are no hardcoded hex values left in the widget's own source.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonPrimary, sCTkSegmentedButton

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x300")
    root.title("SegmentedButton Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    widget = sCTkSegmentedButton(
        base,
        values=["Alpha", "Beta", "Gamma"],
        command=lambda choice: print(f"Selected: {choice}"),
    )
    widget.pack(expand=True, fill="none", padx=10, pady=10)
    widget.set("Beta")

    def toggle_widget_state():
        target = "disabled" if widget.get_state() == "normal" else "normal"
        widget.configure(state=target)
        btn_toggle.configure(text="Enable" if target == "disabled" else "Disable")

    btn_toggle = sCTkButtonPrimary(base, text="Disable", command=toggle_widget_state)
    btn_toggle.pack(side="bottom", pady=15)

    root.mainloop()
```

Pygubu Designer

Selectable by clicking. CustomTkinter's own designer plugin gives up here — its code carries the note "I can't select a segmented button in preview". The segments are `CTkButton`s the widget creates for itself, so they are not in the builder's widget map and a click on one resolves to nothing. This library's preview class hides them from the Designer's binding pass and forwards their clicks to the widget's internal canvas, which is where the Designer's own handler actually sits.

Clearing values restores ["CTkSegmentedButton"] — CustomTkinter's own placeholder, a hardcoded literal in `CTkSegmentedButton.__init__`. Note it does not carry this library's `s` prefix; the string was written by hand rather than derived from the class, so a subclass shows it unchanged. A cleared field therefore matches a freshly placed widget.

font is required in the theme block. Without it, clearing the font field in the inspector had nothing to revert to and reported the current value as its own default — so clearing appeared to do nothing.

Known Limitations

- `state()` treats any value other than `"disabled"` (case-insensitive) as `"normal"` — including typos like `"disbaled"`. No exception is raised and no warning is logged.
- `unselected_color`, `unselected_hover_color`, `border_width`, `border_color`, and `selected_color_padding` cannot be set via constructor kwargs (see [Constructor](#)); the color-related ones only take effect through the theme file or the widget's own repaint logic.
- Calling `configure("some_property_name")` with a single property name does not return a Pygubu-style query tuple the way `sCTkComboBox`/`sCTkCheckBox` do; it's forwarded to the native widget's `configure()`, which — per the wider Pygubu-query investigation set aside earlier in this project — does not support single-argument property queries at all for most properties.

[Return to Table of Contents](#)

sCTkSlider

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkSlider is a themeable subclass of `customtkinter.CTkSlider`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state. Unlike every other widget in this library, its state isn't tracked in a separate instance attribute — it reads and writes CustomTkinter's own native `state` property directly, treating it as the single source of truth.



Constructor

```
sCTkSlider(master=None, command=None, variable=None, **kw)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
command	callable	None	Called with the current value as the slider is dragged.
variable	tkinter.Variable	None	Optional variable bound to the current value.
**kw	—	—	Any native CTkSlider argument (e.g. <code>from_</code> , <code>to</code> , <code>number_of_steps</code>), or an override for one of the theme keys listed under Theming .

```

volume_slider = sCTkSlider(
    master=control_panel,
    command=on_volume_changed,
)
volume_slider.pack(fill="x", padx=40, pady=10)

```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's enabled/disabled state. Queries read directly from the native widget's own state property rather than a parallel attribute. Setting forwards to <code>configure(state=mode)</code> , which reaches the native widget's own state handling — confirmed by direct testing to correctly block interaction.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) /</code> <code>config(**kwargs)</code>	varies	Standard widget configuration, plus: <code>command/variable</code> are routed individually; <code>state</code> is not specially intercepted — it flows straight through to the native widget's own <code>configure()</code> , which is what makes this widget's disable mechanism correct. Calling <code>configure("propname")</code> with a single property name returns a Tkinter-style query tuple for <code>state</code> , <code>fg_color</code> , <code>progress_color</code> , <code>button_color</code> , and <code>button_hover_color</code> .

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block, including `width`, `height`, `button_length`, and `border_width`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` / `configure(state=...)` change** — `fg_color`, `progress_color`, `button_color`, and `button_hover_color` are recomputed from the theme's normal values or its `disabled_map`.

```
{
  "sCTkSlider": {
    "width": 200,
    "height": 24,
    "button_length": 12,
    "border_width": 9,
    "fg_color": ["#E5E7EB", "#4B5563"],
    "progress_color": ["#1A4375", "#2471A3"],
    "button_color": ["#2471A3", "#2471A3"],
    "button_hover_color": ["#112A4B", "#1F618D"],
    "disabled_map": {
      "fg_color": ["#CBD5E1", "#374151"],
      "progress_color": ["#CBD5E1", "#4B5563"],
      "button_color": ["#94A3B8", "#4B5563"]
    }
  }
}
```

`button_color` is the same value for both light and dark mode here — a deliberate accent color that doesn't shift with appearance mode. `disabled_map` has no `button_hover_color` entry; while disabled, the widget explicitly forces `button_hover_color` to match `button_color` instead, since hover can't trigger once natively disabled anyway.

Colors are stored and passed through as raw (light, dark) tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

A colour set at runtime survives a state change. `configure(progress_color="yellow")` holds through disable and re-enable, rather than being reverted by the repaint above. Clearing the property returns it to the theme's value, not to whatever was set before it. See [Theming](#) for the general rule.

This widget's theme block went missing from `sCTkThemes.json` at some point, and nothing failed — the slider simply rendered in CustomTkinter's own colours and never dimmed when disabled, because its `disabled_map` was empty. The values above are the original ones. If a slider looks untouched by your theme, check the block is present before looking anywhere else.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkSlider, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("Slider Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    volume_slider = sCTkSlider(base, command=lambda v: print(f"Value: {v:.2f}"))
    volume_slider.pack(fill="x", pady=10)

    def toggle_disabled():
        target = "disabled" if volume_slider.get_state() == "normal" else "normal"
        volume_slider.state(target)
        disable_toggle.configure(text="Enable Slider" if target == "disabled" else "Disable Slider")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Slider", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- **Fixed:** `configure("fg_color")` used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for four specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

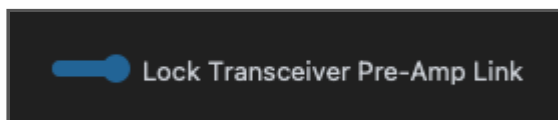
sCTkSwitch

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkSwitch is a themeable subclass of `customtkinter.CTkSwitch`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state that dims every color property, not just the label text. A previously separate widget, `sCTkSwitchAlt`, existed specifically to work around limitations that have since been resolved directly in this widget and has been retired.



Disabling combines two mechanisms: CustomTkinter's native `state="disabled"`, and a bindtag-based click interceptor that prepends a dedicated binding returning `"break"` on click. This is more robust than a simple event-unbind, since it intercepts clicks regardless of which internal level the native click handler is actually bound at.

Constructor

```
sCTkSwitch(master=None, onvalue=1, offvalue=0, command=None, **kw)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
onvalue	any	1	Value reported when the switch is on.
offvalue	any	0	Value reported when the switch is off.
command	callable	None	Called on toggle. May accept the new value as a single argument, or no arguments at all — both styles are supported.
**kw	—	—	state is pulled out explicitly. Everything else is either a native CTkSwitch argument or a theme-key override (see the "sCTkSwitch" block in Theming).

```

notify_switch = sCTkSwitch(
    master=control_panel,
    text="Enable notifications",
    command=on_notify_toggled,
)
notify_switch.pack(anchor="w", padx=40, pady=10)

```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's enabled/disabled state.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>cget("state") / cget("command")</code>	varies	Both intercepted specially, since they're tracked on the instance rather than delegated to the native widget.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration. Note the signature is <code>configure(require_redraw=None, **kwargs)</code> , matching real CTK's own convention, rather than <code>*args</code> — calling <code>configure("state")</code> positionally returns a Tkinter-style query tuple; a positional dict is merged into the update.

Exceptions from your `command` propagate normally. An earlier version silently swallowed every exception a command raised, hiding real bugs completely; this is now fixed, confirmed by direct testing. Tkinter's own default callback-exception handling reports propagated exceptions to the console without crashing the running application.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `font`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` / `configure()` change** — all five color properties (`fg_color`, `progress_color`, `button_color`, `button_hover_color`, `text_color`) are recomputed from the theme's normal values or its `disabled_map`. This full dimming is confirmed working by direct testing.

```

{
  "sCTkSwitch": {
    "font": ["Arial", 14, "normal"],
    "fg_color": ["#1A4375", "#1F6AA5"],
    "progress_color": ["#1A4375", "#1F6AA5"],
    "button_color": ["#CBD5E1", "#CBD5E1"],
    "button_hover_color": ["#E5E7EB", "#94A3B8"],
    "text_color": ["#1F2937", "#F9FAFB"],
    "disabled_map": {
      "text_color": ["#94A3B8", "#64748B"],
      "fg_color": ["#94A3B8", "#526071"],
      "progress_color": ["#64748B", "#526071"],
      "button_color": ["#CBD5E1", "#94A3B8"],
      "button_hover_color": ["#CBD5E1", "#94A3B8"]
    }
  }
}

```

`button_color` uses the same light-mode value for both normal and hover states by design — it was retuned from an earlier pure-white value, which had too little contrast against light backgrounds in general (not a code bug; CustomTkinter already resolves the widget's background to match its real parent correctly on its own). The disabled-state track colors (`fg_color` / `progress_color`) were similarly retuned for the same reason — the original values were close enough to typical light backgrounds that a disabled switch's track could become hard to see at all.

All five color properties are required to be present in both the top-level block and `disabled_map` — if any are missing, the widget raises immediately rather than substituting a hardcoded color.

Colors are stored and passed through as raw (`light`, `dark`) tuples rather than resolved to a single value ahead of time, so they correctly follow system/app appearance-mode changes automatically — confirmed by direct testing, including while disabled.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkSwitch, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("Switch Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    notify_switch = sCTkSwitch(base, text="Enable notifications", command=lambda v: print(f"Value: {v}"))
    notify_switch.pack(anchor="w", pady=10)

    def toggle_disabled():
        target = "disabled" if notify_switch.get_state() == "normal" else "normal"
        notify_switch.state(target)
        disable_toggle.configure(text="Enable Switch" if target == "disabled" else "Disable Switch")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Switch", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

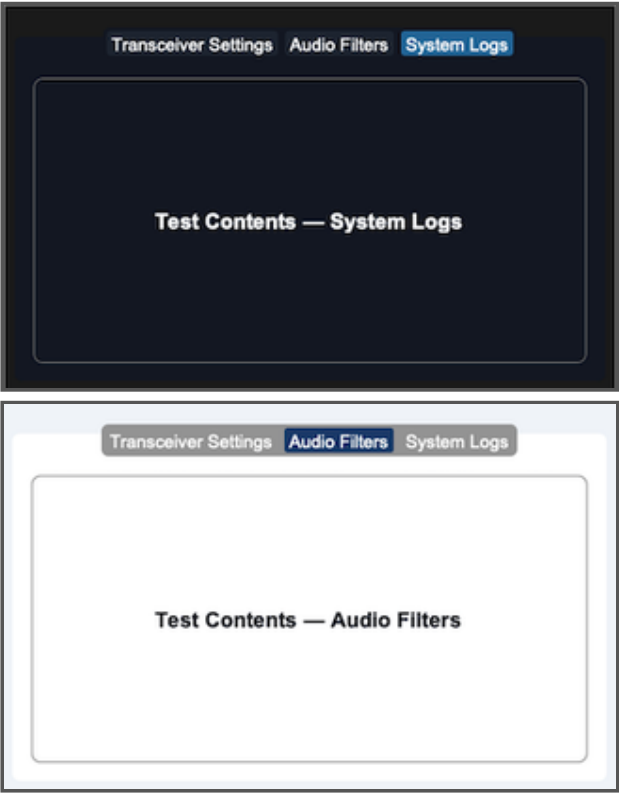
Known Limitations

- If your `command` accepts exactly one argument and raises a `TypeError` for a reason unrelated to argument count, the wrapper's fallback logic can't tell the difference from "this command doesn't accept an argument" — it will retry calling your command with no arguments, which then fails with a second, different `TypeError` (a missing-argument error) layered on top of your real bug. Python's exception chaining keeps both visible in the console, so the real bug isn't hidden, just noisier than ideal.
- Calling `configure("proprname")` for a property name other than `"state"` is forwarded to the native widget's `configure()`, which does not support single-argument property queries — a known limitation shared with the wider Pygubu query investigation set aside elsewhere in this project.

[Return to Table of Contents](#)

sCTkTabview

sCTkTabview is a themeable multi-page tab container — a subclass of customtkinter.CTkTabview with automatic light/dark theme resolution from sCTkThemes.json , a disabled state, and Pygubu Designer support.



Its one structural difference from the native widget: `add()` and `tab()` return an `sCTkFrame` , not a `ctk.CTkFrame` . See [Tab Pages](#).

Table of Contents

- [Constructor](#)
- [Tab Pages](#)
- [Pygubu Designer Tab Insertion](#)
- [Methods](#)
- [Theming \(sCTkThemes.json\)](#)
- [Example](#)
- [Known Limitations](#)

Constructor

```
tabview = sCTkTabview(master=None, **kw)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.

Parameter	Type	Default	Description
state	str	"normal"	"normal" or "disabled". Also settable via <code>configure(state=...)</code> or <code>state()</code> .
**kw	—	—	Any native <code>CTkTabview</code> argument, or an override for one of the theme keys listed under Theming .

Construction **raises `KeyError` immediately** if the theme block is incomplete — see [Theming](#).

Tab Pages

Native `CTkTabview.add()` constructs a plain `ctk.CTkFrame` for each tab and grids it inline, with no hook to substitute a different class. Rather than reimplement `add()` against CustomTkinter's internals — or mutate the created frame's `__class__` at runtime, a fragile pattern deliberately retired elsewhere in this project — this widget embeds an `sCTkFrame` **inside** the native tab frame and hands that back instead.

The native frame stays exactly where `CTkTabview` put it and keeps doing its own show/hide/grid work untouched; it just becomes an invisible outer shell. The wrapper is transparent with no border of its own, so the tab looks identical — the difference is purely structural: everything placed in a tab now has an `sCTk` widget as its parent.

```
# add() returns the page directly -- no separate tab() call needed.
page = widget.add("Transceiver Settings")

inner_panel = sCTkFrame(page, border_width=1)
inner_panel.pack(expand=True, fill="both", padx=10, pady=10)
```

`tab(name)` returns the same object on every call, and creates the wrapper on first use — so a tab created by any other path (`insert()`, or `CTkTabview`'s own machinery) still comes back correctly wrapped. If you specifically need the native outer shell, `ctk.CTkTabview.tab(widget, name)` still reaches it.

Pygubu Designer Tab Insertion

This library registers its own **`sCTkTabview.Tab`** under the `sCustomTkinter` palette section. Use that rather than CustomTkinter's `CTkTabview.Tab`.

1. Place an `sCTkTabview` into your layout.
2. In the widget palette, under **`sCustomTkinter`**, select **`sCTkTabview.Tab`**.
3. Drop it onto the `sCTkTabview` in the tree. It won't be offered anywhere else — a tab has nowhere else to live.
4. Repeat for each page. Set each tab's `label` in the properties panel.

A tab is not a widget you construct: it is created by its tabview, through `add()`. So the builder object has no widget class of its own, and the generated code reads:

```
tab1 = tabview1.add("Settings")
```

Children you drop onto a tab are parented to the `sCTkFrame` page that `add()` returns.

Duplicate names are renamed, not rejected. Native `CTkTabview.add()` raises `ValueError` on a name already in use, and inside the Designer that exception surfaces only on the console where nobody sees it — the tab silently fails to appear and the tree and preview disagree. A numeric suffix is appended instead, so a second tab labelled `mark` becomes `mark_2`, visibly, in both the tab strip and the property field. The widget itself still raises, so application code creating a duplicate tab fails loudly; the leniency is a design-time affordance only.

CustomTkinter's `CTkTabview.Tab` is still accepted as a child, so existing `.ui` files keep loading. Calling `widget.tab(name)` on one still returns a wrapped `sCTkFrame`, since wrapping happens lazily on first access.

Methods

Method	Returns	Description
<code>add(name)</code>	<code>sCTkFrame</code>	Creates a tab and returns its content page. Return type differs from native <code>CTkTabview.add()</code> .
<code>tab(name)</code>	<code>sCTkFrame</code>	Returns a tab's content page, creating the wrapper on first use. Stable across calls.
<code>delete(name)</code>	—	Deletes a tab, tearing down its page wrapper first so no stale entry is left behind.
<code>rename(old_name, new_name)</code>	—	Renames a tab. Overrides the native method to re-key the internal page registry as well — without that the wrapper would stay filed under the old name, so <code>tab()</code> and <code>delete()</code> would miss it and a second wrapper would be built inside the same native tab frame.
<code>state()</code> / <code>state(mode)</code>	<code>str</code>	Getter with no argument; setter with "normal" or "disabled". Dims text, flattens the tab bar, and locks tab selection.
<code>get_state()</code>	<code>str</code>	Equivalent to <code>state()</code> with no argument.
<code>configure(**kwargs)</code> / <code>config(**kwargs)</code>	<code>None</code>	Standard configuration. Accepts state alongside any native option.
<code>configure("state")</code>	<code>tuple</code>	Pygubu-style single-argument query, returning (name, name, name, default, current).
<code>cget(name)</code>	Any	Extended to know about state; everything else passes through to the native widget.

All four state paths — `state()`, `get_state()`, `cget("state")`, and `configure(state=...)` — operate on the same underlying value and agree with each other.

On `bind()`: native `CTkTabview.bind()` raises `NotImplementedError`. This widget overrides it to route through `tkinter.Frame.bind` instead, so Pygubu Designer click handling doesn't crash the workspace.

Theming (sCTkThemes.json)

```
{
  "sCTkTabview": {
    "font": ["Arial", 15, "normal"],
    "segmented_button_height": 36,
    "fg_color": ["#FFFFFF", "#111827"],
    "text_color": ["#FFFFFF", "#FFFFFF"],
    "segmented_button_fg_color": ["#9E9E9E", "#111827"],
    "segmented_button_selected_color": ["#1A4375", "#2471A3"],
    "segmented_button_selected_hover_color": ["#112A4B", "#1F618D"],
    "segmented_button_unselected_color": ["#9E9E9E", "#1F2937"],
    "segmented_button_unselected_hover_color": ["#7D7D7D", "#374151"],
    "disabled_map": {
      "segmented_button_fg_color": ["#FFFFFF", "#111827"],
      "segmented_button_selected_color": ["#CBD5E1", "#374151"],
      "segmented_button_unselected_color": ["#CBD5E1", "#374151"],
      "text_color": ["#94A3B8", "#64748B"]
    }
  }
}
```

Every key above is required. Construction raises `KeyError` naming exactly what's missing, rather than substituting a guessed color. This is the fail-loud principle used across the project — an earlier version fell back to hardcoded literals for all ten colors and the font, and because those guesses looked plausible, a broken or partial theme block was invisible.

The split between the two blocks:

Keys	Required in
text_color, segmented_button_fg_color, segmented_button_selected_color, segmented_button_unselected_color	top level and disabled_map
segmented_button_selected_hover_color, segmented_button_unselected_hover_color, font, segmented_button_height	top level only

The two hover colors deliberately have no `disabled_map` entry. A disabled tab bar must not light up under the cursor, so when disabled, hover collapses to the corresponding non-hover disabled color. There is no meaningful "dimmed hover" distinct from "dimmed", so requiring a separate key would only invite them to drift apart. `font` and `segmented_button_height` are top level only because neither changes with state.

`font` and `segmented_button_height` are both intercepted before native construction and forwarded to the internal segmented button. This is not optional: `CTkTabview` names every parameter explicitly with no `**kwargs` catch-all, so any key it doesn't recognize raises `ValueError` from its constructor. They're applied once rather than on every repaint, since neither varies by state. See [Known Limitations](#) regarding what `segmented_button_height` actually achieves.

Disabled tab text now honours `disabled_map.text_color`. It did not before: disabling puts the internal segmented button into CustomTkinter's own disabled state, which puts its inner buttons there too — and a disabled `CTkButton` draws from `text_color_disabled` and ignores `text_color` entirely. The theme's value was set and then discarded at draw time in favour of CustomTkinter's default grey. Both options are now set on each button.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Validation is scoped to direct construction. A subclass reaches this constructor with `final_kw` built from *its own* theme block — `ThemeableWidget`'s run-once guard means it is never rebuilt — so validating these keys against a subclass's block would raise on every construction. Subclasses own their own theme contract.

Example

```
#!/usr/bin/python3
import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTkLabelPrimary, sCTk, sCTkTabview

if __name__ == "__main__":
    root = sCTk()
    root.geometry("640x480")
    root.title("sCTkTabview Container Validation Bench")
    root.configure(fg_color=("#F1F5F9", "#1C1C1C"))

    base = sCTkFrame(root, border_width=2)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    widget = sCTkTabview(base)
    widget.pack(expand=True, fill="both", padx=10, pady=10)

    for page_name in ["Transceiver Settings", "Audio Filters", "System Logs"]:
        # add() returns the sCTkFrame content page directly.
        page_viewport = widget.add(page_name)

        inner_frame = sCTkFrame(page_viewport, border_width=1, corner_radius=8)
        inner_frame.pack(expand=True, fill="both", padx=10, pady=10)

        test_label = sCTkLabelPrimary(inner_frame, text=f"Test Contents - {page_name}")
        test_label.pack(expand=True, fill="none", padx=20, pady=20)

    def toggle_tab_lock():
        target = "disabled" if widget.state() == "normal" else "normal"
        widget.configure(state=target)
        btn_lock.configure(
            text="Unlock Tabview Navigation" if target == "disabled" else "Lock Tabview (Set 'disabled')"
        )
        print(f"state()={widget.state()}  cget={widget.cget('state')}")

    def toggle_temp_page():
        if "Scratch Pad" in widget._sctk_pages:
            widget.delete("Scratch Pad")
            btn_temp.configure(text="Add Runtime Page")
        else:
            page = widget.add("Scratch Pad")
            sCTkLabelPrimary(page, text="Created at runtime").pack(expand=True, padx=20, pady=20)
            btn_temp.configure(text="Delete Runtime Page")

    def toggle_skin_preference():
        ctk.set_appearance_mode("Light" if ctk.get_appearance_mode() == "Dark" else "Dark")

    control_tray = sCTkFrame(root, fg_color="transparent")
    control_tray.pack(side="bottom", fill="x", padx=20, pady=(0, 15))

    btn_lock = sCTkButtonPrimary(control_tray, text="Lock Tabview (Set 'disabled')", command=toggle_tab_lock)
    btn_lock.pack(side="left", expand=True, padx=4)

    btn_temp = sCTkButtonPrimary(control_tray, text="Add Runtime Page", command=toggle_temp_page)
    btn_temp.pack(side="left", expand=True, padx=4)

    btn_skin = sCTkButtonPrimary(control_tray, text="Toggle UI Light/Dark Appearance", command=toggle_skin_preference)
    btn_skin.pack(side="right", expand=True, padx=4)

    root.mainloop()
```

Known Limitations

- **segmented_button_height is currently a no-op, retained for possible future use.** The value is applied to the internal segmented button and `cget("height")` reports it back accurately, but the visible tab strip does **not** grow to match. `CTkTabview` grids the segmented button into a row whose `minsize` comes from its own private spacing constants, and deliberately overlaps the button with the page frame below to produce the connected-tab look. A taller button is clipped by that row rather than expanding it. Confirmed by direct testing: a height of 128 reported back correctly and produced no visible change.

The key is deliberately kept, and kept **required**, rather than removed. It costs nothing, it keeps the theme contract stable, and it's already wired end-to-end — so if a future CustomTkinter release exposes the strip height, or the internals approach below is revisited, only the application step changes. Do not treat it as broken and delete it; changing the number is expected to do nothing today.

Making the strip actually taller would mean writing `CTkTabview`'s private `_top_spacing` / `_top_button_overhang` attributes and re-running its `_configure_grid()` — a dependency on CustomTkinter internals that could break on any upstream release. Deliberately not done.

Note this is a `CTkTabview` layout constraint, **not** a limitation of the segmented button: a standalone `sCTkSegmentedButton` honors `height` normally.

- **Tabs are selected from the widget tree, not the design canvas.** Clicking a tab page in Pygubu Designer does not select that tab. `CTkTabview` stacks every page in one grid cell with only the active one mapped, so a click cannot be attributed to the page you aimed at — an attempt at this produced a highlight on one tab while the tree showed another, which is worse than not working. CustomTkinter's own tabs have the same limitation; their designer plugin contains a commented-out attempt at the same fix. Select the tab in the tree to edit its `label`.
- **A tabview does not grow with its contents.** It is a fixed size: drop widgets into a tab and the tabview stays as it was, clipping anything that does not fit. Set `width` and `height` on the tabview itself. This is native `CTkTabview` behaviour — the widget takes explicit dimensions and does not propagate its children's requested size — and its theme block sets neither, so the size you get without asking is CustomTkinter's own default.
- **Disabling does not cascade to children.** It dims the tab bar and locks tab selection, but widgets placed inside a page are unaffected — disabling them is the caller's responsibility.
- **`add()` and `tab()` return a different type than the native widget.** Code doing an `isinstance` check against `ctk.CTkFrame`, or reaching for `CTkFrame`-specific internals on a tab page, would notice. `ctk.CTkTabview.tab(widget, name)` still reaches the native shell.
- **The internal segmented button is a native `CTkSegmentedButton`, not `sCTkSegmentedButton`.** It is created inside `CTkTabview.__init__` and re-themed afterwards by pushing colors onto it. Replacing it with the themed variant would let it theme itself and remove most of that code, but the swap hasn't been made.
- **Each tab page carries one extra frame layer** — the native shell plus the `sCTkFrame` wrapper inside it. Transparent and borderless, so invisible, but present in the widget tree.
- **`text_color` is applied by reaching into the segmented button's private `_buttons_dict`,** since CustomTkinter exposes no public way to set per-button text color on a segmented button. This depends on a CustomTkinter internal and could break on a future release.
- **Colors are resolved to a single value** via `_resolve_color()` rather than passed through as raw `(light, dark)` tuples, so appearance-mode changes rely on this widget's own `_set_appearance_mode()` hook re-running the theme pass rather than on CustomTkinter's native tracking.

[Return to Table of Contents](#)

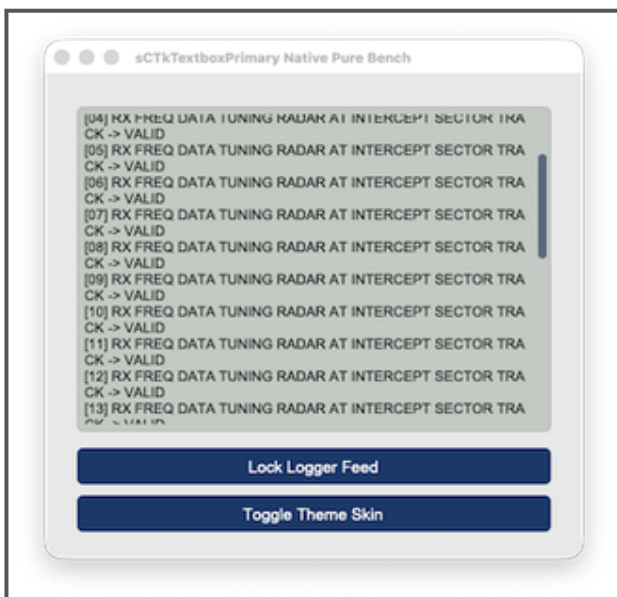
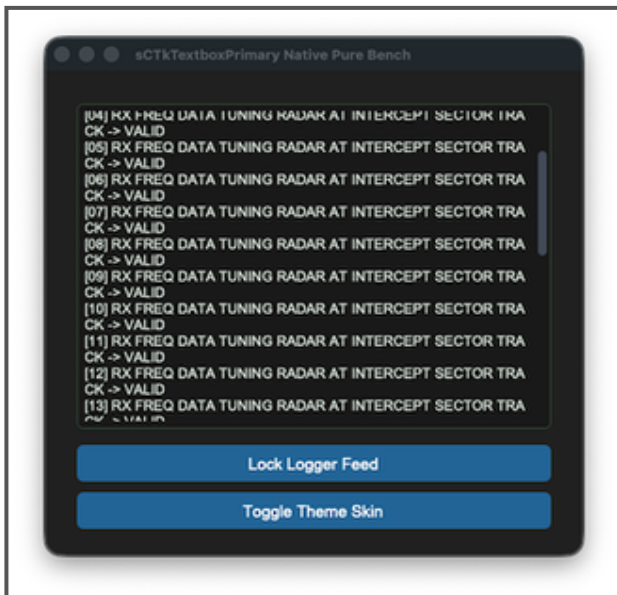
sCTkTextboxPrimary

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkTextboxPrimary is a themeable subclass of `customtkinter.CTkTextbox` — a multi-line text area, the higher-emphasis of the library's two textbox tiers (see also `sCTkTextboxSecondary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state, using CustomTkinter's native `state="disabled"`.



If `fg_color` resolves to `"transparent"` at construction, the widget copies its parent's actual `fg_color` instead, since `CTkTextbox` doesn't render true transparency the way canvas-based widgets can.

Constructor

```
sCTkTextboxPrimary(master=None, **kw)
```

Parameter	Type	Default	Description
<code>master</code>	<code>widget</code>	<code>None</code>	Parent container.
<code>**kw</code>	—	—	state is pulled out and applied after construction. Everything else is any native <code>CTkTextbox</code> argument, or an override for one of the theme keys listed under Theming .

```
log_area = sCTkTextboxPrimary(master=control_panel)
log_area.pack(fill="both", expand=True, padx=40, pady=10)
```

Methods

Method	Returns	Description
<code>state(state_string=None)</code>	<code>str</code>	Gets or sets the widget's enabled/disabled state. Only <code>"disabled"</code> (case-insensitive) disables it; <code>"normal"</code> , <code>"enabled"</code> , or <code>"active"</code> all enable it. Uses CTk's native <code>state="disabled"</code> , consistent with the other widgets in this library confirmed to correctly block interaction this way.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> updates the tracked state and triggers a repaint. Calling <code>configure("propname")</code> with a single property name returns a Tkinter-style query tuple for <code>state</code> , <code>fg_color</code> , <code>text_color</code> , <code>border_color</code> , <code>scrollbar_button_color</code> , and <code>scrollbar_button_hover_color</code> .

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block, including `font`, `border_width`, and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `border_color`, `text_color`, `scrollbar_button_color`, and `scrollbar_button_hover_color` are recomputed from the theme's normal values or its `disabled_map`. This includes manually re-theming the widget's internal scrollbar, which isn't automatically covered by a single `configure()` call.

```
{
  "sCTkTextboxPrimary": {
    "font": ["Arial", 13, "normal"],
    "border_width": 1,
    "corner_radius": 6,
    "border_color": ["#b5beb6", "#3d5242"],
    "fg_color": ["#cbcfcb", "#1a1a1a"],
    "text_color": ["#1c1d1c", "#e3ece4"],
    "scrollbar_button_color": ["#64748B", "#4B5563"],
    "scrollbar_button_hover_color": ["#1A4375", "#2471A3"],
    "disabled_map": {
      "fg_color": ["#E5E7EB", "#111827"],
      "border_color": ["#CBD5E1", "#1F2937"],
      "text_color": ["#94A3B8", "#64748B"],
      "scrollbar_button_color": ["#E5E7EB", "#1F2937"],
      "scrollbar_button_hover_color": ["#E5E7EB", "#1F2937"]
    }
  }
}
```

`scrollbar_button_color` and `scrollbar_button_hover_color` are required to be present in whichever map is active — if either is missing, the widget raises immediately rather than substituting a hardcoded color.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw (light, dark) tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkTextboxPrimary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x350")
    root.title("TextboxPrimary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    log_area = sCTkTextboxPrimary(base)
    log_area.pack(fill="both", expand=True, pady=10)
    log_area.insert("1.0", "Log output appears here...")

    def toggle_disabled():
        target = "disabled" if log_area.get_state() == "normal" else "normal"
        log_area.state(target)
        disable_toggle.configure(text="Enable Log" if target == "disabled" else "Disable Log")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Log", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- **Fixed:** single-argument queries used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for six specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

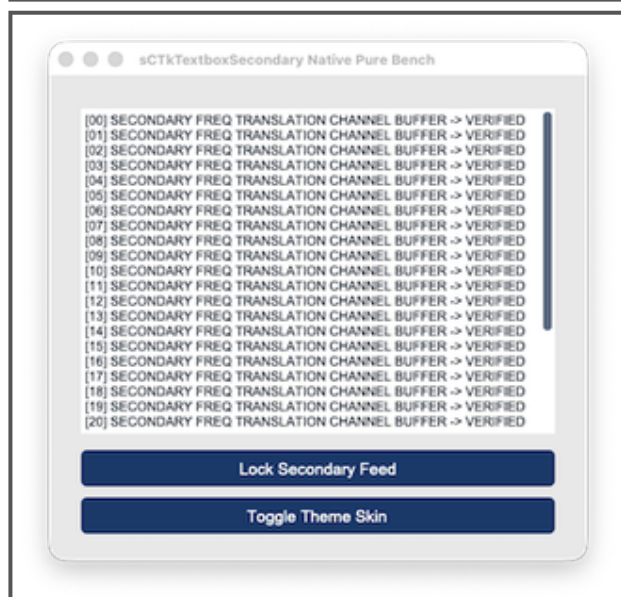
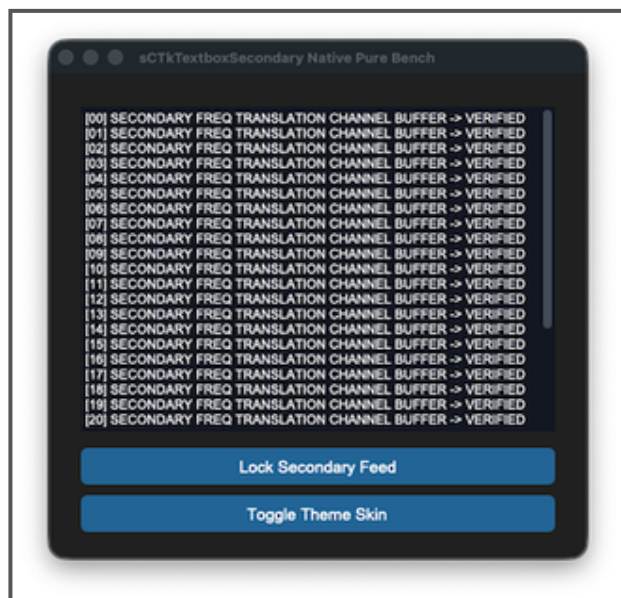
sCTkTextboxSecondary

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkTextboxSecondary is a themeable subclass of `customtkinter.CTkTextbox` — a multi-line text area, the lower-emphasis of the library's two textbox tiers (see also `sCTkTextboxPrimary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state, using CustomTkinter's native `state="disabled"`.



If `fg_color` resolves to `"transparent"` at construction, the widget copies its parent's actual `fg_color` instead — same rationale as `sCTkTextboxPrimary`'s identical fallback.

Constructor

```
sCTkTextboxSecondary(master=None, **kw)
```

Parameter	Type	Default	Description
<code>master</code>	widget	None	Parent container.
<code>**kw</code>	—	—	state is pulled out and applied after construction. Everything else is any native <code>CTkTextbox</code> argument, or an override for one of the theme keys listed under Theming .

```
notes_area = sCTkTextboxSecondary(master=control_panel)
notes_area.pack(fill="both", expand=True, padx=40, pady=10)
```

Methods

Method	Returns	Description
<code>state(state_string=None)</code>	str	Gets or sets the widget's enabled/disabled state. Only <code>"disabled"</code> (case-insensitive) disables it; <code>"normal"</code> , <code>"enabled"</code> , or <code>"active"</code> all enable it.
<code>get_state()</code>	str	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, plus: passing <code>state=...</code> updates the tracked state and triggers a repaint. Calling <code>configure("propname")</code> with a single property name returns a Tkinter-style query tuple for <code>state</code> , <code>fg_color</code> , <code>text_color</code> , <code>border_color</code> , <code>scrollbar_button_color</code> , and <code>scrollbar_button_hover_color</code> .

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block, including `font`, is merged with any matching keyword arguments and applied when the widget is built. Note `border_width` and `corner_radius` are both `0` for this style — no visible border, unlike `sCTkTextboxPrimary`.
- **Re-applied on every `state()` change** — `fg_color`, `text_color`, `scrollbar_button_color`, and `scrollbar_button_hover_color` are recomputed from the theme's normal values or its `disabled_map`, including manually re-theming the internal scrollbar.

```
{
  "sCTkTextboxSecondary": {
    "font": ["Arial", 12, "normal"],
    "border_width": 0,
    "corner_radius": 0,
    "fg_color": ["#FFFFFF", "#111827"],
    "text_color": ["#1F2937", "#F9FAFB"],
    "scrollbar_button_color": ["#64748B", "#4B5563"],
    "scrollbar_button_hover_color": ["#1A4375", "#2471A3"],
    "disabled_map": {
      "fg_color": ["#E5E7EB", "#111827"],
      "text_color": ["#94A3B8", "#64748B"],
      "scrollbar_button_color": ["#E5E7EB", "#1F2937"],
      "scrollbar_button_hover_color": ["#E5E7EB", "#1F2937"]
    }
  }
}
```

`scrollbar_button_color` and `scrollbar_button_hover_color` are required to be present in whichever map is active — if either is missing, the widget raises immediately rather than substituting a hardcoded color.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkTextboxSecondary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x350")
    root.title("TextboxSecondary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    notes_area = sCTkTextboxSecondary(base)
    notes_area.pack(fill="both", expand=True, pady=10)
    notes_area.insert("1.0", "Notes go here...")

    def toggle_disabled():
        target = "disabled" if notes_area.get_state() == "normal" else "normal"
        notes_area.state(target)
        disable_toggle.configure(text="Enable Notes" if target == "disabled" else "Disable Notes")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Notes", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- **Fixed:** single-argument queries used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.

- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for six specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

Menus

Not a lot of choices here, but they should suffice.

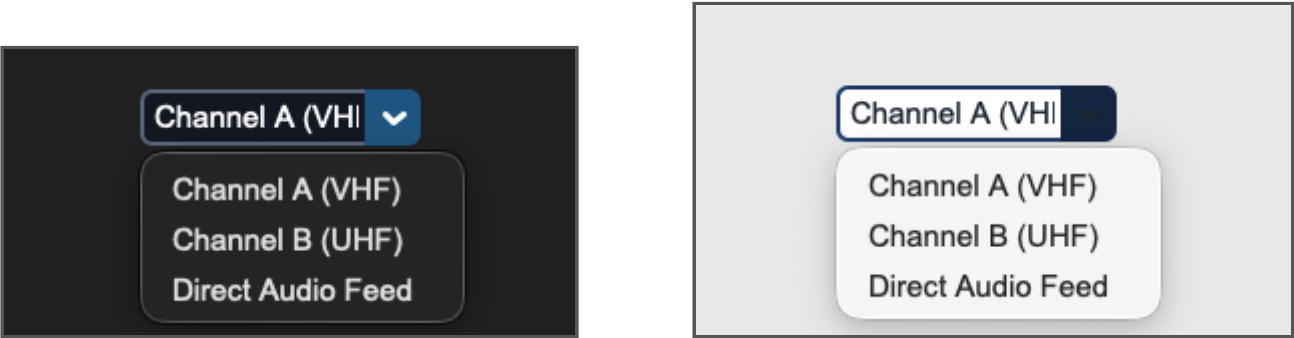
sCTkComboBox

Table of Contents

- [Overview](#)
- [Constructor](#)
- [Methods](#)
- [Theming \(sCTkThemes.json\)](#)
- [Example](#)
- [Known Limitations](#)

Overview

sCTkComboBox is a themeable subclass of customtkinter.CTkComboBox. It adds automatic light/dark theme resolution from sCTkThemes.json, a distinct enabled/disabled visual state (separate from, but synchronized with, the widget's native interactive lock), and Pygubu Designer property introspection support.



Constructor

```
sCTkComboBox(master=None, values=None, command=None, variable=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
values	list[str]	[""]	Dropdown options. If the first entry is a non-empty string, it is selected automatically on creation.
command	callable	None	Called with the selected value when the user picks an item.
variable	tkinter.StringVar	None	Optional variable bound to the current selection.

Parameter	Type	Default	Description
**kwargs	—	—	Any native CTKComboBox argument (e.g. width, height, font, corner_radius), or an override for one of the theme keys listed under Theming . Anything not supplied falls back to the sCTkComboBox block of sCTkThemes.json.

```
frequency_dropdown = sCTkComboBox(
    master=control_panel,
    values=["Channel A (VHF)", "Channel B (UHF)", "Direct Audio Feed"],
    command=on_frequency_channel_changed,
)
frequency_dropdown.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
<code>get()</code>	str	Currently selected text (native CTKComboBox behavior).
<code>set(value)</code>	None	Sets the displayed text (native CTKComboBox behavior).
<code>state(mode=None)</code>	str None	Gets or sets the widget's enabled/disabled visual state. Accepts "normal", "enabled", or "active" (all equivalent) and "disabled". Any other value is silently ignored. Called with no argument, returns the current state as a lowercase string.
<code>get_state()</code>	str	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard widget configuration, with two additions: passing <code>state=...</code> routes to <code>state()</code> rather than the native Tkinter state option; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for state, fg_color, border_color, text_color, and hover_color. Queries for any other single property name fall through to the native CTKComboBox.configure.

Theming (sCTkThemes.json)

Theme values are used in two different ways, worth keeping separate:

- **Applied once, at construction** — every key in the widget's theme block (including `corner_radius`) is merged with any matching keyword arguments you pass in and applied when the widget is built.

- **Re-applied on every state() change** — only `fg_color`, `border_color`, `text_color`, `button_color`, `button_hover_color`, `dropdown_fg_color`, `dropdown_text_color`, `dropdown_hover_color`, `border_width`, and `font` are swapped between the theme's normal values and its `disabled_map` values when you call `state("disabled") / state("normal")`. Toggling state also sets the native Tkinter `state` flag, so a disabled combo box is both visually muted and non-interactive.

```
{
  "sCTkComboBox": {
    "fg_color": ["#FFFFFF", "#1E1E1E"],
    "border_color": ["#94A3B8", "#4B5563"],
    "text_color": ["#111827", "#F9FAFB"],
    "button_color": ["#1A4375", "#1F6AA5"],
    "button_hover_color": ["#112A4B", "#194A7A"],
    "dropdown_fg_color": ["#FFFFFF", "#1F2937"],
    "dropdown_text_color": ["#374151", "#F3F4F6"],
    "dropdown_hover_color": ["#F3F4F6", "#374151"],
    "border_width": 2,
    "corner_radius": 6,
    "disabled_map": {
      "fg_color": ["#F3F4F6", "#111111"],
      "border_color": ["#CBD5E1", "#333333"],
      "text_color": ["#94A3B8", "#4B5563"],
      "button_color": ["#E5E7EB", "#222222"]
    }
  }
}
```

Note: `disabled_map` above has no entries for `dropdown_fg_color`, `dropdown_text_color`, `dropdown_hover_color`, or `button_hover_color`. Since only keys present in `disabled_map` are swapped, those four properties keep their *enabled*-state color when the widget is disabled. Add entries for them here if you want the dropdown portion to visually mute along with the rest of the widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonPrimary, sCTkComboBox

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x300")
    root.title("ComboBox Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    widget = sCTkComboBox(
        base,
        values=["Channel A (VHF)", "Channel B (UHF)", "Direct Audio Feed"],
        command=lambda choice: print(f"Selected: {choice}"),
    )
    widget.pack(expand=True, fill="none", padx=10, pady=10)

    def toggle_widget_state():
        target = "disabled" if widget.get_state() == "normal" else "normal"
        widget.configure(state=target)
        btn_toggle.configure(text="Enable" if target == "disabled" else "Disable")

    btn_toggle = sCTkButtonPrimary(base, text="Disable", command=toggle_widget_state)
    btn_toggle.pack(side="bottom", pady=15)

    root.mainloop()
```

Known Limitations

- `state()` silently ignores any value other than `normal`, `enabled`, `active`, or `disabled` — no exception is raised and no warning is logged.
- Passing a positional dict to `configure()` (e.g. `configure({"fg_color": "red"})`) is not merged into the update; only keyword arguments are applied. Use `configure(**your_dict)` instead.

[Return to Table of Contents](#)

sCTkOptionMenuPrimary

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

`sCTkOptionMenuPrimary` is a themeable subclass of `customtkinter.CTkOptionMenu` — a dropdown option-selection button. It adds automatic light/dark theme resolution from `sCTkThemes.json`, a distinct enabled/disabled visual state, and border support. See also [sCTkOptionMenuSecondary](#), the quiet variant, which has the same architecture and differs only in its theme.



Constructor

```
sCTkOptionMenuPrimary(master=None, values=None, command=None, variable=None, **kw)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
values	list[str]	native default	The dropdown options.
command	callable	None	Called with the selected value when the user picks an item.
variable	tkinter.StringVar	None	Optional variable bound to the current selection.
**kw	—	—	Any native CTkOptionMenu argument, <code>border_width</code> / <code>border_color</code> , or an override for one of the theme keys listed under Theming .

```
mode_menu = sCTkOptionMenuPrimary(
    master=control_panel,
    values=["AM", "FM", "SSB"],
    command=on_mode_changed,
)
mode_menu.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's enabled/disabled state. Only "disabled" (case-insensitive) disables it; "normal", "enabled", or "active" all enable it. Uses CTK's native <code>state="disabled"</code> , consistent with the other widgets in this library confirmed to correctly block interaction this way, though not independently re-tested for this specific widget.
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) /</code> <code>config(**kwargs)</code>	varies	Standard widget configuration, plus: values/ command/variable are routed individually; <code>state=...</code> routes through <code>state()</code> ; calling <code>configure("propname")</code> with a single property name returns a Tkinter-style (name, name, name, default, current) tuple for <code>state</code> , <code>fg_color</code> , <code>button_color</code> , <code>button_hover_color</code> , and <code>text_color</code> . Queries for any other property name fall through to the native <code>CTkOptionMenu.configure</code> .
<code>update_list(new_values,</code> <code>default_index=0)</code>	<code>None</code>	Replaces the dropdown's options and resets the visible selection. If <code>new_values</code> is empty, the widget is set to a single blank option. If <code>default_index</code> is out of range, falls back to index 0 rather than raising.

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block, including `font`, `dropdown_font`, and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `button_color`, `button_hover_color`, `text_color`, `dropdown_fg_color`, `dropdown_text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```

{
  "sCTkOptionMenuPrimary": {
    "font": ["Arial", 15, "normal"],
    "dropdown_font": ["Arial", 15, "normal"],
    "fg_color": ["#1A4375", "#2471A3"],
    "button_color": ["#112A4B", "#1F618D"],
    "button_hover_color": ["#0D1F38", "#1A5276"],
    "text_color": ["#FFFFFF", "#FFFFFF"],
    "corner_radius": 6,
    "border_width": 0,
    "border_color": ["#112A4B", "#1F618D"],
    "dropdown_fg_color": ["#FFFFFF", "#1F2937"],
    "dropdown_text_color": ["#1F2937", "#F9FAFB"],
    "dropdown_hover_color": ["#E5E7EB", "#374151"],
    "disabled_map": {
      "fg_color": ["#CBD5E1", "#374151"],
      "button_color": ["#CBD5E1", "#374151"],
      "text_color": ["#94A3B8", "#64748B"],
      "border_color": ["#CBD5E1", "#374151"]
    }
  }
}

```

border_width is 0, so this variant has no border — that is the difference between the two option menus, along with `button_color` giving the arrow its own colour. The keys are present rather than omitted so the choice is visible and reversible: raise the width and Primary gets an outline, exactly as `sCTkOptionMenuSecondary` has one.

Native `CTkOptionMenu` has no border option at all. It comes from `sCTkOptionMenuBorderMixin`, which both variants share — see that module for what it depends on.

`disabled_map` doesn't cover `button_hover_color`, `dropdown_fg_color`, `dropdown_text_color`, or `dropdown_hover_color` — consistent with every other themed widget in this library: once natively disabled, hover and dropdown-open interactions can't fire in the first place, so there's nothing for a disabled-state color on those properties to ever visibly apply to.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkOptionMenuPrimary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("OptionMenuPrimary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    mode_menu = sCTkOptionMenuPrimary(
        base, values=["AM", "FM", "SSB"], command=lambda choice: print(f"Selected: {choice}")
    )
    mode_menu.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if mode_menu.get_state() == "normal" else "normal"
        mode_menu.state(target)
        disable_toggle.configure(text="Enable Menu" if target == "disabled" else "Disable Menu")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Menu", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes "disabled" and "normal"/"enabled"/"active"; any other value matches neither branch, though colors are still harmlessly re-applied.
- **The border depends on a CustomTkinter internal.** `sCTkOptionMenuBorderMixin` replaces the widget's private `_draw()` and calls the private draw engine, because native `CTkOptionMenu` passes a hardcoded `0` where the border width belongs. If an upstream release changes that method, the border disappears — a visual regression, not a crash.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.

[Return to Table of Contents](#)

sCTkOptionMenuSecondary

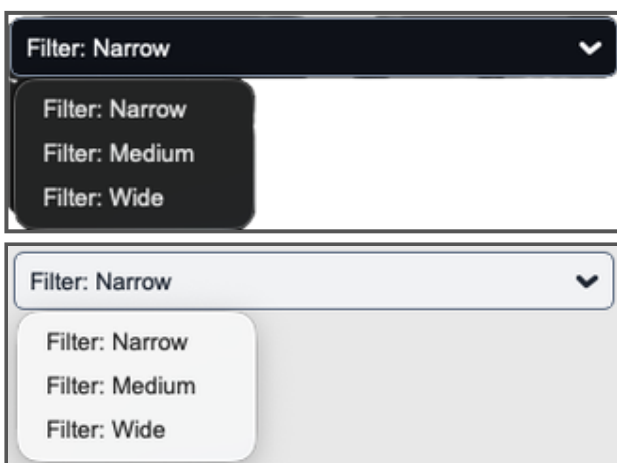
Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

`sCTkOptionMenuSecondary` is the quiet variant of the dropdown option-selection menu — a themeable subclass of `customtkinter.CTkOptionMenu`, with a border. See also `sCTkOptionMenuPrimary`, the emphasised variant.

The two are now **architecturally identical**. They differ only in their theme blocks: this one has a border and an arrow that blends into the control, Primary has no border and an arrow with its own colour. Either look is reachable from either widget by changing the theme.



This widget used to be a composite — a `CTkFrame` wrapping a plain `CTkOptionMenu`, because native `CTkOptionMenu` has no border option and this variant needs one. That structure cost more than it bought: `values`, `command` and `variable` lived on the inner menu, so `cget("values")` returned `None` here while returning a list on Primary; every option CustomTkinter added had to be forwarded by hand; and the frame's rounded rectangle did not line up with the menu's, leaving visibly broken corners.

`sCTkOptionMenuBorderMixin` now supplies the border, so this is a plain subclass and `get()`, `set()`, `values`, `command` and `variable` are all native again. `self._menu` no longer exists.

Constructor

```
sCTkOptionMenuSecondary(master=None, width=160, height=28, **kw)
```

Parameter	Type	Default	Description
<code>master</code>	widget	<code>None</code>	Parent container.

Parameter	Type	Default	Description
width	int	160	Widget width, used unless overridden by a kwarg or the theme.
height	int	28	Widget height, used unless overridden by a kwarg or the theme.
**kw	—	—	values (list[str]), command (callable) and variable (tkinter.StringVar) are applied once the native widget exists. Everything else is a native CTKOptionMenu argument, border_width/border_color, or an override for one of the theme keys under Theming .

```
band_menu = sCTkOptionMenuSecondary(
    master=control_panel,
    values=["80m", "40m", "20m", "10m"],
    command=on_band_changed,
)
band_menu.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
state(mode=None)	str	Gets or sets the widget's enabled/disabled state. Only "disabled" (case-insensitive) disables it; "normal", "enabled" or "active" all enable it.
get_state()	str	Equivalent to calling state() with no argument.
get() / set(value)	—	Native CTKOptionMenu behaviour.
cget(name)	varies	Native, extended to border_width and border_color.
configure(**kwargs) / config(**kwargs)	varies	Standard configuration, plus: border_width/ border_color; values/command/variable routed individually; state=... through state(). configure("propname") returns a Tkinter-style query tuple for state, the border properties, and fg_color/button_color/ button_hover_color/text_color; anything else gets a proper tuple from the shared query helper.
update_list(new_values, default_index=0)	None	Replaces the options and resets the visible selection. Empty list falls back to a blank option; out-of-range default_index falls back to 0.

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block.
- **Re-applied on every state() change** — `fg_color`, `text_color` and `border_color` are recomputed from the theme's normal values or its `disabled_map`. The font and dropdown keys are static and set once.

```
{
  "sCTkOptionMenuSecondary": {
    "border_width": 1.25,
    "corner_radius": 6,
    "border_color": ["#64748B", "#94A3B8"],
    "fg_color": ["#F3F4F6", "#0B0F19"],
    "font": ["Arial", 13, "normal"],
    "dropdown_font": ["Arial", 13, "normal"],
    "text_color": ["#1F2937", "#F9FAFB"],
    "button_hover_color": ["#94A3B8", "#374151"],
    "dropdown_fg_color": ["#FFFFFF", "#1F2937"],
    "dropdown_text_color": ["#1F2937", "#F9FAFB"],
    "dropdown_hover_color": ["#E5E7EB", "#374151"],
    "disabled_map": {
      "text_color": ["#94A3B8", "#64748B"],
      "border_color": ["#CBD5E1", "#374151"],
      "fg_color": ["#E5E7EB", "#0B0F19"]
    }
  }
}
```

button_color is absent deliberately. It is set from `fg_color` in code, so the arrow blends into the control rather than standing out — that is what makes this the quiet variant. Adding the key would be misleading: the value would be read and then overwritten. Primary gives the arrow its own colour.

The border comes from sCTkOptionMenuBorderMixin, which both variants share. Native `CTkOptionMenu` has no border option; the mixin supplies one. Primary carries the same keys with `border_width` at 0.

`fg_color` and `text_color` are required to be present in whichever map is active — if either is missing, the widget raises immediately rather than substituting a hardcoded color, per this project's design of failing hard on incomplete theme data (see `sCTkLabelPrimary / Secondary / Tertiary` for the precedent). An earlier version of this widget used hardcoded hex fallbacks for both, and separately had a real bug where the theme's actual `button_hover_color` was computed correctly and then immediately overwritten with `fg_color` — both are fixed as of this project's audit.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw (`light`, `dark`) tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkOptionMenuSecondary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("OptionMenuSecondary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    band_menu = sCTkOptionMenuSecondary(
        base, values=["80m", "40m", "20m", "10m"], command=lambda choice: print(f"Selected: {choice}")
    )
    band_menu.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if band_menu.get_state() == "normal" else "normal"
        band_menu.state(target)
        disable_toggle.configure(text="Enable Menu" if target == "disabled" else "Disable Menu")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Menu", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes "disabled" and "normal"/"enabled"/"active"; any other value matches neither branch, though colors are still harmlessly re-applied.
- **Fixed:** single-argument queries used to return `str(value)` of a (light, dark) tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.
- **The border depends on a CustomTkinter internal.** `sCTkOptionMenuBorderMixin` replaces the widget's private `_draw()` and calls the private draw engine, because native `CTkOptionMenu` passes a hardcoded 0 where the border width belongs. If an upstream release changes that method, the border disappears — a visual regression, not a crash.
- **`self._menu` is gone.** Code written against the previous composite structure, which reached the inner dropdown directly, needs updating: this widget *is* the dropdown now.

[Return to Table of Contents](#)

Additional Widgets Provided by sCustomTkinter

These are all the extra widgets that were added to the stock set provided with CustomTkinter.

sCTK DialBase

Abstract base class for the rotary dial family. It owns the canvas rendering, the mouse and scroll interaction model, the theme contract, and the state machine shared by `sCTK DialContinuous`, `sCTK DialSelector`, and `sCTK DialRange`.



This page is the reference for how dials are drawn and themed. The three variant pages describe only what differs.

Note the spelling: the class is `sCTK DialBase` with a capital K. It is never instantiated directly and has no theme block of its own — each concrete subclass resolves its own block by class name.

Table of Contents

- Knob rendering
- Theme contract
- Reading theme colours
- Latching
- Shared API
- Redraw model
- Known limitations

Knob rendering

The knob is drawn as a stack of concentric ovals stepping from `dial_shadow_color` at the rim to `dial_highlight_color` off-centre, each ring shifted toward a light source in the upper left. Tk's canvas has no gradient primitive and **no alpha channel**, so this is the only way to get a domed surface — every colour is a solid fill computed by interpolation, never a translucent overlay.

Two arcs finish it: `dial_rim_light_color` across the upper-left edge and `dial_rim_shadow_color` across the lower-right.

The rim light does most of the work on dark knobs. A black knob's shading range is clamped at the bottom — you cannot go darker than black at the edge — so it has roughly half the dynamic range of a light one. Remove the bright rim arc and a dark knob collapses back to a flat disc. This is why the highlight and shadow colours are explicit theme keys rather than derived from `dial_color` by a fixed lighten/darken: a percentage that produces a visible rim on black blows out on aluminium, and vice versa.

Tuning constants live on the base class, so they can be overridden per subclass or per instance:

Constant	Default	Meaning
KNOB_SHADE_STEPS	18	Ring count. Below ~12 the steps read as contour bands; above ~24 costs more than it shows.
KNOB_SHADE_SHRINK	0.55	How far the stack shrinks from rim to centre, as a fraction of radius.
KNOB_LIGHT_OFFSET	0.55	How far each ring drifts toward the light.
DIMPLE_RADIUS_FRAC	0.36	Finger dimple radius, as a fraction of knob radius.
DIMPLE_RIM_CLEARANCE_FRA C	0.06	Gap between dimple edge and rim, same units.
POINTER_WIDTH	3.0	Pointer line width in pixels.
POINTER_RIM_INSET	3	How far short of the rim the pointer stops.

The dimple and clearance are **fractions, not pixels**. An earlier version used a fixed 14px inset with a fixed 14.5px radius, so the dimple was lost on a large dial and swallowed a small one.

Recesses shade opposite to domes. The knob body is a dome, lit on the upper left. The dimple is a hole, so its shading inverts — shadowed on the upper-left interior wall, lit on the lower right. Drawn with the body's light direction it reads as a raised bump instead of something you can put a finger in.

Theme contract

Every concrete dial requires these at the **top level** of its theme block:

```
fg_color, text_color, shadow_color, dial_color, dial_highlight_color, dial_shadow_color,
dial_rim_light_color, dial_rim_shadow_color
```

and these inside **disabled_map**:

```
text_color, dial_color
```

Plus one variant-specific key each — see the individual pages.

Construction raises `KeyError` naming the missing key and where it belongs. This replaced a pattern of `.get(key)` or `("hex", "hex")` throughout the draw routine, which silently substituted a plausible guess and made an incomplete theme block look merely slightly-off rather than broken.

`fg_color` is deliberately **not** required in `disabled_map`: the background does not dim when disabled, the knob face and text carry the signal. This also fixes a latent bug — the old code read `fg_color` from `disabled_map` for *both* the dial face and the background, so once that key existed the knob would have rendered the same colour as the surface behind it and vanished. The two only looked different because the map was empty and their hardcoded fallbacks happened to differ.

The flat `disabled_text_color` / `disabled_dial_color` / `disabled_dimple_glow` keys used by earlier theme files are **retired**. They now live in `disabled_map` under their normal names, matching every other widget in the library.

Reading theme colours

Custom drawing colours must be read from the raw theme registry, not from `final_kw`. This is a trap that produces plausible-looking wrong colours rather than an error, and it went unnoticed in this widget family for its entire existence.

`ThemeableWidget` maintains a `CUSTOM_VECTOR_KEYS` set — `dial_color`, `shadow_color`, `text_color`, `pointer_color`, `pointer_glow_color`, `knob_diameter` and others — which it strips out of `final_kw` for vector widgets, so they never reach the native `CTkFrame` constructor and raise `ValueError`. That stripping is correct and necessary.

What was wrong was reading those colours back out of `final_kw` afterwards. They were never in there. Every fallback in the old draw code was therefore *always* taken, and the configured values for `dial_color`, `shadow_color`, `text_color` and `pointer_glow_color` were decorative — the dials rendered in hardcoded colours regardless of what the theme said. Applying fail-loud validation is what surfaced it.

The base class now builds `_local_defaults` from the raw registry block, with `final_kw` layered on top so non-vector keys and constructor overrides keep their precedence:

```
raw_block = _tw.GLOBAL_THEME_REGISTRY.get(self.__class__.__name__) or {}
raw_colors = {k: v for k, v in raw_block.items() if not isinstance(v, dict)}
self._local_defaults = ThemeableWidget._convert_lists_to_tuples(raw_colors)
self._local_defaults.update(self.final_kw)
```

The registry is reached as a **module attribute**, not a direct name import, because

`load_initial_framework_themes()` rebinds that global on load — `from ... import GLOBAL_THEME_REGISTRY` captures the empty dict that exists at import time.

Latching: arming a dial before it responds

Opt-in, and off by default. A dial built without `latching` behaves as it always has: always live, `pressed_map` ignored and not required in its theme block.

With `latching=True` the dial starts switched off. Clicks, drags and the wheel are ignored — the bindings are simply not installed — and it draws the resting colours from the top level of its theme block. Arming it swaps to `pressed_map` and restores the input.

```
def arm(dial):
    dial.set_pressed(True)

def disarm(dial):
    dial.set_pressed(False)

mode_switch = sCTKDialSelector(
    panel,
    labels=["AM", "FM", "LSB", "USB", "CW"],
    latching=True,
    double_click_command=arm,
    shift_double_click_command=disarm,
)
```

This exists for controls where an accidental brush of the wheel does real damage — changing band or sideband, slamming the volume. It is not the right default for a plain volume knob: an inert rotary control with nothing on screen to explain itself is a discoverability problem, which is why opting in is deliberate.

The two callbacks

Both are plain callbacks. **The widget wires neither of them to the latch** — toggling on a double-click is one use and a natural one, but the decision belongs to the application. Each is called with the dial itself, so one handler can serve several dials without a closure over any name.

Property	Gesture	Bound when
<code>double_click_command</code>	<code><Double-Button-1></code>	Enabled, and not an armed latching dial
<code>shift_double_click_command</code>	<code><Shift-Double-Button-1></code>	Enabled

Why arming and disarming use different gestures

Every mouse button steps a dial, so a plain double-click competes with ordinary clicking to turn the knob. Tk gives `<Double-Button-1>` precedence over the second `<Button-1>`, so while both are bound an accidental double-click costs a step — two clicks, one step.

That only matters in one direction. While a latching dial is **off**, nothing is bound, so arming by double-click cannot collide with anything. It is disarming that is unsafe, which is why it has its own gesture.

The plain double-click is therefore **not bound at all on an armed latching dial**, removing the collision rather than compensating for it. One consequence: `double_click_command` does not fire while a latching dial is armed. A non-latching dial keeps it bound throughout, since there it is a general-purpose callback with no arming role.

Shift is already the drag modifier, but a shift-double-click only fires `<Shift-ButtonPress-1>` twice, which records a drag origin and nothing else — so the gesture is free.

A dial you never click, driven by the wheel or from code, can ignore all of this and put `toggle_pressed()` on the plain double-click.

Disabling clears the latch

`state("disabled")` switches a latching dial **off** as well as locking it, so re-enabling leaves it inert until armed again. That is what makes a panel lock useful: unlocking gives you a screen of dials that cannot be nudged by accident, rather than one that comes back live because it was live before the lock.

API

Member	Description
<code>latching</code>	Constructor argument and property. True opts in. Available in the Designer.
<code>pressed</code>	Initial armed state. Constructor argument and property, not offered in the Designer — see below.
<code>is_pressed()</code>	Whether the dial is currently armed. False on a non-latching dial.
<code>set_pressed(value)</code>	Arms or disarms. A no-op returning False on a non-latching dial, so code that arms every dial on a panel needs no special cases.
<code>toggle_pressed()</code>	Flips the state.

`pressed` is deliberately absent from the Designer. The inspector reflects the **live** widget, so double-clicking a dial on the design canvas set it True and pygubu wrote `pressed=True` into the generated code as a non-default — the application then started armed, the opposite of what latching is for.

Theme

`pressed_map` holds the operational colours; the top-level block holds the resting, dimmed ones. Each key falls back to the top level, so a `pressed_map` naming only the few colours that should brighten is valid.

A **non-latching dial draws from `pressed_map` too**, because it is permanently on. That keeps opting out from changing how a dial looks — and a theme block with no `pressed_map` at all falls through to the top level, which in the older theme shape holds the bright values. Both shapes therefore behave correctly.

`pressed_map` is required only when `latching=True`. `text_color` and `dial_color` must be present, matching `disabled_map`.

Shared API

Member	Type	Description
<code>state(mode=None)</code>	method	Getter with no argument; setter with "normal" or "disabled". Unbinds clicks, wheel and trackpad input, and repaints from <code>disabled_map</code> . On a latching dial it also clears the latch — see Latching .
<code>get_state()</code>	method	Equivalent to <code>state()</code> with no argument.
<code>configure(state=...)</code>	method	Same effect as <code>state()</code> . Both routes are supported.
<code>configure(name)</code>	method	Pygubu-style single-argument query.
<code>config</code>	alias	Bound to <code>configure</code> on every class in the family .
<code>knob_diameter</code>	int	Diameter of the KNOB, in pixels. Was <code>diameter</code> , which set the canvas width and height together — so it named the widget, not the circle, and a diameter of 300 gave a 244px knob. <code>width</code> and <code>height</code> now name the canvas independently, and dials sharing a <code>knob_diameter</code> have identical knobs whatever their labels.
<code>divisions</code>	int	Tick count drawn around the outer ring.

`config = configure` is declared separately on each class, and must be. Tkinter binds `.config` to `.configure` as its own class attribute — it does not track whichever `configure()` a subclass defines. Without a per-class line, `.config(...)` skips every override and lands on the native widget, bypassing `divisions/command/knob_diameter` handling and the theme repaint entirely. This was missing from all four dial classes; the same bug was confirmed on `sCTKSegmentedButton` earlier in this project's audit. An inherited alias would not help — it would point at the *parent's* `configure()`.

Redraw model

Two entry points, deliberately separate:

- `_draw_dial_base()` rebuilds everything. Call on geometry, theme or state change.

- `_redraw_indicator()` redraws only the dimple or pointer line, leaving the knob body, ticks and labels alone. Call on a **value** change.

The body is now roughly twenty shaded ovals plus ticks and labels, none of which changes as the dial turns. Rebuilding all of it per detent would make the shading cost real; the split makes it free while tuning. Same pattern as `sCTkSMeter._execute_needle_draw()`, which redraws its needle against a static face.

`_redraw_indicator()` falls back to a full pass if the body isn't on the canvas — first paint, or after a resize wiped it — so a partial update can't leave the dial blank.

Known limitations

- **Constructor overrides don't work for vector-guarded colours.** `sCTkDialContinuous(master, dial_color="#FF0000")` is silently ignored, because `ThemeableWidget` strips those names from its `kwargs` loop as well as from the theme block. Changing this means touching `themeable_widget.py`.
- **The five shading keys aren't in `CUSTOM_VECTOR_KEYS`**, so they do land in `final_kw`. Harmless here — the dial's own `FRAME_VALID_KEYS` whitelist filters them before the native constructor — but a widget that forwarded `final_kw` wholesale would raise.
- **Knurling teeth and the canvas background fallback remain hardcoded.** The teeth are a shadow effect rather than a palette choice; the background fallback is the "a raw canvas needs a renderable colour" case accepted elsewhere in this library.
- **Ticks and labels are not affected by the body shading** — they sit outside the knob radius and draw flat in `text_color`.
- **Scroll handling is duplicated across the three subclasses.** `_process_mac_touchpad_scroll` and `_process_scroll_wheel` are near-identical in each, differing only in the line that applies the step. This is not a candidate for `ScrollBindingMixin`: a dial steps discretely with a cooldown and has no `yview_scroll` target. It belongs in this base class with one overridable step method.

[Return to Table of Contents](#)

sCTkDialContinuous

Table of Contents

- Overview
- Constructor
- Callbacks
- Latching
- Colours you can set per instance
- Centralized Stylesheet Setup
- Other Notes
- Example

An infinite flywheel tuning encoder module tracking signed velocity delta step increments across an endless 360-degree rotation path (ideal for high-fidelity radio VFO controls, audio mixers, and multi-channel squelch encoders).



Overview

Property / Feature	Type / Signature	Description
Instantiation	<i>Constructor</i>	<code>sCTkDialContinuous(master)</code> (<i>Infinite Tuning Wheel Encoder</i>)
File Mapping	<i>Inheritance Tree</i>	Inherits vector math mechanics and 3D knob rendering directly out of <code>sCTkDial.py</code> .
<code>_scroll_cooldown_seconds</code>	<code>float</code>	Throttle limiting touchpad refresh rates to stabilize fast tuning rolls.
<code>set_position_index(delta)</code>	Method (<code>int</code>)	Manually advances the 3D dimple coordinates via an integer step.
<code>left_click_callback</code>	Callable / None	Replaces the built-in single step taken when the left half of the canvas is clicked. Use it to move by more than one position per click.
<code>right_click_callback</code>	Callable / None	Replaces the built-in single step taken when the right half of the canvas is clicked.
<code>state(mode)</code>	<code>str</code>	"normal" or "disabled". <code>configure(state=...)</code> does the same thing. Disabling removes the click, wheel and trackpad bindings and repaints from <code>disabled_map</code> . On a latching dial it also clears the latch.
<code>latching</code>	<code>bool</code>	Opt-in. The dial starts switched off and ignores input until armed — see Latching . Default <code>False</code> .

Property / Feature	Type / Signature	Description
double_click_command	Callable / None	Called on a double-click, with the dial itself. Not wired to anything by the widget; typically used to arm a latching dial.
shift_double_click_command	Callable / None	Called on a shift-double-click, with the dial itself. Typically used to disarm.

Constructor

Initialize an infinite flywheel encoder instance. Keyword properties layer safely over centralized configuration defaults and are automatically sanitized by the `ThemeableWidget` mixin layer before the native constructor fires.

```
# Instantiate the themed infinite VFO wheel element
tuning_dial = sCTkDialContinuous(
    master=frame_continuous,
    divisions=24,
    knob_diameter=130,
    command=on_vfo_dial_rotated,
    left_click_callback=my_custom_left_click,
    right_click_callback=my_custom_right_click
)
```

Callbacks

Dispatches a raw signed directional integer step change directly to runtime listeners upon rotation changes.

Command

```
# Fires automatically on valid mouse scrolling, touchpad rolling, or click-drag actions
def on_vfo_dial_rotated(clicks_delta: int):
    # Clockwise rotation yields positive steps (+1); Counter-clockwise yields negative steps (-1)
    global current_frequency_hz
    current_frequency_hz += clicks_delta * 100
```

Colours you can set per instance

Three of this widget's colours are properties as well as theme keys — set them in the constructor, through `configure()`, or in the Designer's inspector, and leave blank for the theme's value.

Property	Applies to
text_color	The labels and the tick marks — both are drawn with this one key.
dial_color	The knob face.
pointer_glow_color	The glow around the finger dimple.

The remaining five — `dial_highlight_color`, `dial_shadow_color`, `dial_rim_light_color`, `dial_rim_shadow_color` and `shadow_color` — stay theme-only on purpose. They produce the shaded dome together, and changing one in isolation tends to read as broken rather than different.

There is no separate tick colour. If you want ticks and labels to differ, that needs a new theme key and a change to the draw code.

Theming (sCTkThemes.json)

```
{
  "sCTkDialContinuous": {
    "fg_color": ["#F1F5F9", "#0A0A0A"],
    "text_color": ["#7B93B0", "#915404"],
    "shadow_color": ["#DCE3EC", "#06070A"],
    "dial_color": ["#C3C5C7", "#1C1E26"],
    "dial_highlight_color": ["#EAEFE2", "#292A2E"],
    "dial_shadow_color": ["#9FA4A8", "#070708"],
    "dial_rim_light_color": ["#F9FAFC", "#53565A"],
    "dial_rim_shadow_color": ["#8F9396", "#040404"],
    "pointer_glow_color": ["#DCE3EC", "#242A37"],

    "pressed_map": {
      "text_color": ["#1A4375", "#FF9100"],
      "shadow_color": ["#CBD5E1", "#02040A"],
      "dial_color": ["#9E9E9E", "#2A2F3D"],
      "dial_highlight_color": ["#E4E8EC", "#42454B"],
      "dial_shadow_color": ["#5C6165", "#050507"],
      "dial_rim_light_color": ["#FFFFFF", "#8E949C"],
      "dial_rim_shadow_color": ["#3E4245", "#000000"],
      "pointer_glow_color": ["#CBD5E1", "#3A455C"]
    },

    "disabled_map": {
      "text_color": ["#94A3B8", "#4B5563"],
      "dial_color": ["#E2E8F0", "#1A1D24"],
      "pointer_glow_color": ["#CBD5E1", "#334155"]
    }
  }
}
```

Every key above is required — construction raises `KeyError` naming any that are missing. `pressed_map` is the exception: it is read only when the dial is built with `latching=True`.

A dial can be made to require arming before it responds. Opt in with `latching=True` and wire the double-click callbacks; the dial then starts switched off, ignores input, and draws the resting colours until armed. The `pressed_map` above holds the operational set. See [Latching](#) on the base class page for the whole mechanism, including why arming and disarming use different gestures.

A colour set at runtime survives a state change. Disable the widget and enable it again and your value is still there; clearing the property returns it to the theme's, not to whatever was set before. See [Theming](#) for the general rule. See [the base class page](#) for the shared contract.

`pointer_glow_color` is **specific to this variant**: it colours the ring around the finger dimple, and only this dial draws one. It is required in both the top level and `disabled_map`. Selector and Range require `pointer_color` instead.

The dark-mode values above give a black anodised knob. For a brushed-aluminium look, raise `dial_shadow_color` and `dial_highlight_color` toward the light end and brighten the rim.

Sizing

knob_diameter names the knob; width and height name the canvas. The property was previously called `diameter` and set both canvas dimensions to its own value, so the name described neither. This variant draws no labels, so a canvas only slightly larger than the knob is enough — given `knob_diameter` alone, it defaults to 40px more on each side.

Known Limitations

- **Knob rendering:** the body is a shaded dome and the indicator is a recessed finger dimple, sized at 36% of the knob radius with 6% rim clearance — a VFO operator puts a finger in it to spin the dial quickly. Both scale with the knob. See [the base class page](#).
 - **`.config()` now works.** This class previously had no `config = configure` alias, so `.config(...)` bypassed every override and landed on the native widget. If existing code called it expecting no effect, it will now have one.
 - **Theme colours are live for the first time.** Colours were previously read from `final_kw`, which never contained them, so every dial rendered in hardcoded fallbacks regardless of the theme file. See [reading theme colours](#).
 - **No end stops.** The dimple travels continuously around the knob. There is no `arc_angle`, no first or last position, and scrolling never reaches a boundary — this variant reports a signed step delta rather than a position, so there is nothing to clamp.
 - **Click callbacks replace the single step**, rather than adding to it. `set_position_index(2)` inside one moves two positions per click instead of the built-in one.
 - **`_finalize_themeable_lifecycle()` fires at the end of the constructor**, which is what tells a Pygubu-style consumer the widget is ready. Every widget in the library does this; a missing call means an `on_first_object_cb` callback silently never runs.
-

Example

Below is a complete, self-contained test execution script demonstrating how to properly embed an `sCTkDialContinuous` alongside custom click jump hooks and an interactive VFO digital frequency display counter readout.

```

#!/usr/bin/python3
# =====
# TESTING HARNESS IMPORTS & SETUP for Dial Continuous
# =====

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkLabelSecondary, sCTkDialContinuous

# Global state trackers for the interactive bench loop
current_frequency_hz = 14032000

def refresh_frequency_display():
    """Formats integers into a clean MHz telemetry layout readout string."""
    freq_str = f"{current_frequency_hz:08d}"
    formatted_freq = f"{freq_str[-8:-6]}.{freq_str[-6:-3]}.{freq_str[-3:]}"
    if formatted_freq.startswith("."):
        formatted_freq = formatted_freq[1:]

    if lbl_vfo_display.winfo_exists():
        lbl_vfo_display.configure(text=f"VFO Freq: {formatted_freq} MHz")

def on_vfo_dial_rotated(clicks_delta):
    """Event-driven callback tracking signed velocity delta step changes."""
    global current_frequency_hz
    current_frequency_hz += clicks_delta * 100
    current_frequency_hz = max(0, current_frequency_hz)
    refresh_frequency_display()

def my_custom_left_click():
    """Moves two positions left per click, instead of the built-in one."""
    if tuning_dial.cget("state") == "disabled":
        return
    tuning_dial.set_position_index(-2) # Jump 2 steps left natively

def my_custom_right_click():
    """Moves two positions right per click, instead of the built-in one."""
    if tuning_dial.cget("state") == "disabled":
        return
    tuning_dial.set_position_index(2) # Jump 2 steps right natively

def toggle_operational_state():
    """Toggles interaction channels and visual states back and forth."""
    current_mode = tuning_dial.cget("state")
    target = "disabled" if current_mode == "normal" else "normal"

    tuning_dial.configure(state=target)
    lbl_vfo_display.configure(state=target)
    btn_toggle.configure(text="Lock Dial (Set 'disabled')" if target == "normal" else "Unlock Dial (Set 'normal')")
    print(f"Logged Verification Hook -> tuning_dial.get_state() = {tuning_dial.get_state()}")

if __name__ == "__main__":
    root = sCTk()
    root.title("sCTkDialContinuous Test Deck")
    root.geometry("380x360")

    base = sCTkFrame(root, corner_radius=8)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    lbl_title = sCTkLabelSecondary(base, text="3. Continuous VFO WHEEL", font=("Arial", 12, "bold"))
    lbl_title.pack(pady=(12, 2))

```

```

tuning_dial = sCTkDialContinuous(
    base,
    divisions=24,
    knob_diameter=130,
    command=on_vfo_dial_rotated,
    left_click_callback=my_custom_left_click,
    right_click_callback=my_custom_right_click
)
tuning_dial.pack(pady=10)

lbl_vfo_display = sCTkLabelSecondary(base, text="VFO Freq: 14.032.000 MHz", font=("Arial", 11, "bold"))
lbl_vfo_display.pack(pady=10)

btn_toggle = sCTkButtonPrimary(base, text="Lock Dial (Set 'disabled')", command=toggle_operational_state)
btn_toggle.pack(side="bottom", pady=15)

print("--- BOOT INITIALIZATION PASSTHROUGH ---")
print(f"Initial Dial State = {tuning_dial.get_state().upper()}")
print("=====\n")

root.mainloop()

```

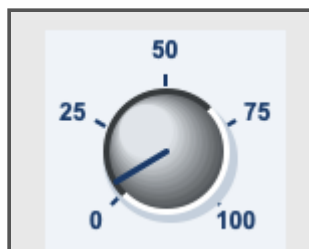
[Return to Table of Contents](#)

sCTkDialRange

Table of Contents

- Overview
- Constructor
- Sizing and Label Placement
- Callbacks
- Latching
- Colours you can set per instance
- Centralized Stylesheet Setup
- Other Notes
- Example

A concrete rotary encoder range variant designed for hard-bounded linear controls (e.g., AF/RF volume gain level sliders, squelch limits, or power thresholds). It enforces absolute mechanical dead stops at outer thresholds, preventing directional wraparound loops.



Overview

Property / Feature	Type / Signature	Description
Instantiation	<i>Constructor</i>	<code>sCTkDialRange(master)</code> (<i>Bounded Linear Range Dial</i>)
File Mapping	<i>Inheritance Tree</i>	Streamlined and compiled programmatically inside <code>sCTkDial.py</code> and <code>ThemeableWidget.py</code> .
<code>from_ / min_value</code>	<code>int</code>	Lower boundary threshold (default 0) enforcing absolute counter-clockwise dead stops.
<code>to / max_value</code>	<code>int</code>	Upper boundary threshold (default 100) enforcing absolute clockwise dead stops.
<code>divisions</code>	<code>int</code>	Quantized subdivision tick line count painted geometrically across the arc limit sweep.
<code>_scroll_cooldown_seconds</code>	<code>float</code>	Throttle limiting touchpad refresh rates to stabilize fast range adjustments.
<code>get() / set(val)</code>	Methods $\rightarrow$ <code>int</code>	Unified index query mechanisms to get or force selected integer values.
<code>left_click_callback</code>	Callable / None	Replaces the built-in single step taken when the left half of the canvas is clicked. Use it to move by more than one position per click.
<code>right_click_callback</code>	Callable / None	Replaces the built-in single step taken when the right half of the canvas is clicked.

Property / Feature	Type / Signature	Description
state(mode)	str	"normal" or "disabled". configure(state=...) does the same thing. Disabling removes the click, wheel and trackpad bindings and repaints from disabled_map. On a latching dial it also clears the latch.
latching	bool	Opt-in. The dial starts switched off and ignores input until armed — see Latching . Default False.
double_click_command	Callable / None	Called on a double-click, with the dial itself. Not wired to anything by the widget; typically used to arm a latching dial.
shift_double_click_command	Callable / None	Called on a shift-double-click, with the dial itself. Typically used to disarm.

Constructor

Initialize a custom bounded linear range potentiometer instance. Custom parameters passed from Pygubu builder allocations (like string translator tracks or data_pool environments) are automatically intercepted, processed, and purged early by the ThemeableWidget mixin layer before the native constructor fires. Bounding geometry sizes and limits scale out of central stylesheet registries.

```
# Instantiate an AF Volume gain potentiometer control dial
volume_potentiometer = sCTkDialRange(
    master=control_panel,
    from_=0,
    to=30,
    divisions=6,
    arc_angle=270,
    command=on_volume_level_changed,
    left_click_callback=my_custom_left_click,
    right_click_callback=my_custom_right_click
)
```

Sizing and Label Placement

knob_diameter is the knob. **width** and **height** are the canvas.

That distinction is new. The property was called `diameter` and set both canvas dimensions to its own value, so `diameter=300` produced a 300px *widget* with a 244px knob inside it — the name described neither. It is now honest: the knob is drawn at exactly `knob_diameter`, and the canvas is whatever `width` and `height` say.

```
dial = sCTkDialRange(parent, knob_diameter=120)           # canvas defaults to 200x200
dial = sCTkDialRange(parent, knob_diameter=120, width=400) # wide canvas, same knob
```

Given only `knob_diameter`, the canvas defaults to 40px larger on each side — enough for the default labels at the default font.

Three dials sharing a `knob_diameter` have identical knobs, however long their labels are. That is the point of the separation. An earlier version measured the labels and shrank the knob to fit them inside a fixed widget; that kept layouts predictable but made a row of controls look inconsistent, which is the case that matters most.

Labels that do not fit are clipped. Widen the canvas, shorten the label, or break it:

```
labels=["Very\nLong", "12", "RTTY"]
```

Labels anchor **away** from the dial — one on the left grows leftward, one at the top grows upward — so a long label extends outward rather than across the knob face. The gap between knob and text scales with `label_font`, so larger text still clears the edge.

Property	Type	Description
<code>knob_diameter</code>	<code>int</code>	Diameter of the knob itself, in pixels. Default 120.
<code>width / height</code>	<code>int</code>	Canvas size. Defaults to <code>knob_diameter + 80</code> when not given.
<code>label_font</code>	<code>tuple</code>	Font for the labels. Blank uses the theme's <code>label_font</code> .

Callbacks

Dispatches the current absolute active integer value directly to runtime tracking listeners upon position changes.

Command

```
# Fires automatically on valid mouse scrolling, touchpad rolling, or click-drag actions
def on_volume_level_changed(active_value: int):
    # active_value is hard constrained between your from_ and to boundary integers
    print(f"Active Selected Option Value position tracker = {active_value}")
```

Colours you can set per instance

Three of this widget's colours are properties as well as theme keys — set them in the constructor, through `configure()`, or in the Designer's inspector, and leave blank for the theme's value.

Property	Applies to
<code>text_color</code>	The labels and the tick marks — both are drawn with this one key.
<code>dial_color</code>	The knob face.
<code>pointer_color</code>	The pointer line on the knob.

The remaining five — `dial_highlight_color`, `dial_shadow_color`, `dial_rim_light_color`, `dial_rim_shadow_color` and `shadow_color` — stay theme-only on purpose. They produce the shaded dome together, and changing one in isolation tends to read as broken rather than different.

There is no separate tick colour. If you want ticks and labels to differ, that needs a new theme key and a change to the draw code.

Theming (sCTkThemes.json)

```
{
  "sCTkDialRange": {
    "fg_color": ["#F1F5F9", "#0A0A0A"],
    "text_color": ["#7B93B0", "#915404"],
    "shadow_color": ["#DCE3EC", "#06070A"],
    "label_font": ["Arial", "9"],
    "dial_color": ["#C3C5C7", "#1C1E26"],
    "dial_highlight_color": ["#EAEF2", "#292A2E"],
    "dial_shadow_color": ["#9FA4A8", "#070708"],
    "dial_rim_light_color": ["#F9FAFC", "#53565A"],
    "dial_rim_shadow_color": ["#8F9396", "#040404"],
    "pointer_color": ["#7B93B0", "#915404"],

    "pressed_map": {
      "text_color": ["#1A4375", "#FF9100"],
      "shadow_color": ["#CBD5E1", "#02040A"],
      "dial_color": ["#9E9E9E", "#2A2F3D"],
      "dial_highlight_color": ["#E4E8EC", "#42454B"],
      "dial_shadow_color": ["#5C6165", "#050507"],
      "dial_rim_light_color": ["#FFFFFF", "#8E949C"],
      "dial_rim_shadow_color": ["#3E4245", "#000000"],
      "pointer_color": ["#1A4375", "#FF9100"]
    },

    "disabled_map": {
      "text_color": ["#94A3B8", "#4B5563"],
      "dial_color": ["#E2E8F0", "#1A1D24"]
    }
  }
}
```

Every key above is required — construction raises `KeyError` naming any that are missing. `pressed_map` is the exception: it is read only when the dial is built with `latching=True`.

A dial can be made to require arming before it responds. Opt in with `latching=True` and wire the double-click callbacks; the dial then starts switched off, ignores input, and draws the resting colours until armed. The `pressed_map` above holds the operational set. See [Latching](#) on the base class page for the whole mechanism, including why arming and disarming use different gestures.

A colour set at runtime survives a state change. Disable the widget and enable it again and your value is still there; clearing the property returns it to the theme's, not to whatever was set before. See [Theming](#) for the general rule. See [the base class page](#) for the shared contract.

`pointer_color` is **specific to this variant and its Selector sibling**, and colours the pointer line. It was present in the theme file for a long time but read by no code path at all — the pointer drew in `text_color` instead. It is now live, so the pointer can differ from the tick labels. It has no `disabled_map` entry; a disabled pointer falls back to the disabled `text_color`.

Known Limitations

- **Knob rendering:** the body is a shaded dome, marked with a plain straight line from dead centre out to just short of the rim. An earlier version drew an arrowhead and a raised centre cap; both are gone, along with the cap's two hardcoded outline colours. See [the base class page](#).
- **`.config()` now works.** This class previously had no `config = configure` alias, so `.config(...)` bypassed every override and landed on the native widget. If existing code called it expecting no effect, it will now have one.
- **Theme colours are live for the first time.** Colours were previously read from `final_kw`, which never contained them, so every dial rendered in hardcoded fallbacks regardless of the theme file. See [reading theme colours](#).

- **Inherits `ctk.CTkFrame` and `ThemeableWidget` directly**, with no intermediate template class in between. That ordering matters: the native class comes first, so every `super()` call in the dial's own methods resolves to `CustomTkinter` rather than to the mixin.
 - **`_finalize_themeable_lifecycle()` fires at the end of the constructor**, which is what tells a Pygubu-style consumer the widget is ready. Every widget in the library does this; a missing call means an `on_first_object_cb` callback silently never runs.
 - **Hard end stops.** Scrolling past `from_` or `to` clamps rather than wrapping, unlike the `Selector`, which wraps around to the first position. A range dial that jumped from maximum to minimum on one extra click would be a hazard on a volume control.
-

Example

Below is a complete, self-contained test execution script demonstrating how to properly embed an `sCTkDialRange` alongside custom click jump hooks and an active volume gain control panel display tracker.

```

#!/usr/bin/python3
# =====
# TESTING HARNESS IMPORTS & SETUP for Dial Range
# =====

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkLabelSecondary, sCTkDialRange

if __name__ == "__main__":

    root = sCTk()
    root.geometry("450x350")
    root.title("Ranged Potentiometer Telemetry Bench")

    base = sCTkFrame(root, corner_radius=8)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    # 1. Live feedback display lane tracking
    lbl_volume = sCTkLabelSecondary(base, text="AF Volume: 15 %", font=("Arial", 11, "bold"))
    lbl_volume.pack(pady=15)

    def my_custom_left_click():
        """Drops three units per click, instead of the built-in one."""
        if volume_pot.get_state() == "disabled": return
        volume_pot.set(volume_pot.get() - 3)

    def my_custom_right_click():
        """Adds three units per click, instead of the built-in one."""
        if volume_pot.get_state() == "disabled": return
        volume_pot.set(volume_pot.get() + 3)

    # 2. Instantiate with explicit limits and tracking labels
    volume_pot = sCTkDialRange(
        base,
        from_=0,
        to=100,
        divisions=5,
        arc_angle=270,
        command=lambda val: lbl_volume.configure(text=f"AF Volume: {int((val / 100) * 100)} %"),
        left_click_callback=my_custom_left_click,
        right_click_callback=my_custom_right_click
    )
    volume_pot.pack(expand=True, fill="none", padx=10, pady=10)
    volume_pot.set(5) # Initialize baseline startup volume index

    # 3. Dynamic panel interactive state toggle test layout
    def toggle_pot_lock():
        current_mode = volume_pot.get_state()
        target = "disabled" if current_mode == "normal" else "normal"

        volume_pot.configure(state=target)
        btn_toggle.configure(text="UNLOCK VOLUME DECK" if target == "disabled" else "LOCK
POTENTIOMETER")
        print(f"Logged Verification Hook -> volume_pot.get_state() = {volume_pot.get_state()}")

    btn_toggle = sCTkButtonPrimary(base, text="LOCK POTENTIOMETER", command=toggle_pot_lock)
    btn_toggle.pack(side="bottom", pady=15)

    # Standard test assertions routine verification sequences
    print("---- BOOT INITIALIZATION PASSTHROUGH ----")
    volume_pot.state("disabled")
    print("state (Disabled Pass) =", volume_pot.get_state()) # Output: disabled

```

```
volume_pot.state("normal")
print("state (Normal Pass)   =", volume_pot.get_state()) # Output: normal
print("=====\n")

root.mainloop()
```

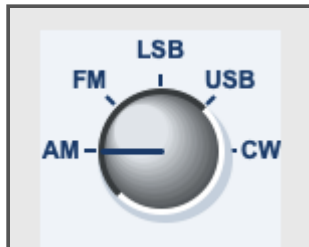
[Return to Table of Contents](#)

sCTkDialSelector

Table of Contents

- Overview
- Constructor
- Sizing and Label Placement
- Callbacks
- Latching
- Colours you can set per instance
- Centralized Stylesheet Setup
- Other Notes
- Example

A concrete rotary encoder switch variant designed for stepped selector controls (e.g., band configurations, operating modes, or filter sub-selections). It uses an explicit bounding arc configuration and outputs a clean integer mapping parameter tracking list item indices natively.



Overview

Property / Feature	Type / Signature	Description
Instantiation	<i>Constructor</i>	<code>sCTkDialSelector(master)</code> (<i>Stepped Arc Selector Dial</i>)
File Mapping	<i>Inheritance Tree</i>	Streamlined and compiled programmatically inside <code>sCTkDial.py</code> and <code>ThemeableWidget.py</code> .
labels	list [str]	Ordered array list mapping string tags directly above calculated step lines. Supports raw comma-separated strings inside layout inspectors.
arc_angle	float	Angular geometric limit (default 270) restricting the pointer range sweep layout.
_scroll_cooldown_seconds	float	Throttle limiting touchpad refresh rates to stabilize fast selector rolls.
get() / set(idx)	Methods -> int	Unified index query mechanisms to get or force selected positions.
left_click_callback	Callable / None	Replaces the built-in single step taken when the left half of the canvas is clicked. Use it to move by more than one position per click.
right_click_callback	Callable / None	Replaces the built-in single step taken when the right half of the canvas is clicked.

Property / Feature	Type / Signature	Description
state(mode)	str	"normal" or "disabled". configure(state=...) does the same thing. Disabling removes the click, wheel and trackpad bindings and repaints from disabled_map. On a latching dial it also clears the latch.
latching	bool	Opt-in. The dial starts switched off and ignores input until armed — see Latching . Default False.
double_click_command	Callable / None	Called on a double-click, with the dial itself. Not wired to anything by the widget; typically used to arm a latching dial.
shift_double_click_command	Callable / None	Called on a shift-double-click, with the dial itself. Typically used to disarm.

Constructor

Initialize a custom stepped rotary selector switch instance. Properties like `labels` support raw string array text list configurations natively for absolute Pygubu inspector panel compatibility. Custom attributes from Pygubu builder allocations (like string `translator` tracks) are automatically intercepted and sanitized by the `ThemeableWidget` mixin layer before the native constructor fires.

```
# Instantiate a 5-position operating mode rotary switch selector
mode_switch = sCTkDialSelector(
    master=control_panel,
    labels=["AM", "FM", "LSB", "USB", "CW-N"],
    arc_angle=180,
    command=on_operating_mode_changed,
    left_click_callback=my_custom_left_click,
    right_click_callback=my_custom_right_click
)
```

Sizing and Label Placement

`knob_diameter` is the knob. `width` and `height` are the canvas.

That distinction is new. The property was called `diameter` and set both canvas dimensions to its own value, so `diameter=300` produced a 300px *widget* with a 244px knob inside it — the name described neither. It is now honest: the knob is drawn at exactly `knob_diameter`, and the canvas is whatever `width` and `height` say.

```
dial = sCTkDialSelector(parent, knob_diameter=120)           # canvas defaults to 200x200
dial = sCTkDialSelector(parent, knob_diameter=120, width=400) # wide canvas, same knob
```

Given only `knob_diameter`, the canvas defaults to 40px larger on each side — enough for the default labels at the default font.

Three dials sharing a `knob_diameter` have identical knobs, however long their labels are. That is the point of the separation. An earlier version measured the labels and shrank the knob to fit them inside a fixed widget; that kept layouts predictable but made a row of controls look inconsistent, which is the case that matters most.

Labels that do not fit are clipped. Widen the canvas, shorten the label, or break it:

```
labels=["Very\nLong", "12", "RTTY"]
```

Labels anchor **away** from the dial — one on the left grows leftward, one at the top grows upward — so a long label extends outward rather than across the knob face. The gap between knob and text scales with `label_font`, so larger text still clears the edge.

Property	Type	Description
<code>knob_diameter</code>	<code>int</code>	Diameter of the knob itself, in pixels. Default 120.
<code>width / height</code>	<code>int</code>	Canvas size. Defaults to <code>knob_diameter + 80</code> when not given.
<code>label_font</code>	<code>tuple</code>	Font for the labels. Blank uses the theme's <code>label_font</code> .

Callbacks

Dispatches the current absolute active list item integer index directly to runtime configuration listeners.

Command

```
# Fires automatically on valid mouse scrolling, touchpad rolling, or click-drag actions
def on_operating_mode_changed(active_index: int):
    # active_index maps directly to items in your labels block list (0, 1, 2, etc.)
    print(f"Active Selected Option Index position tracker = {active_index}")
```

Colours you can set per instance

Three of this widget's colours are properties as well as theme keys — set them in the constructor, through `configure()`, or in the Designer's inspector, and leave blank for the theme's value.

Property	Applies to
<code>text_color</code>	The labels and the tick marks — both are drawn with this one key.
<code>dial_color</code>	The knob face.
<code>pointer_color</code>	The pointer line on the knob.

The remaining five — `dial_highlight_color`, `dial_shadow_color`, `dial_rim_light_color`, `dial_rim_shadow_color` and `shadow_color` — stay theme-only on purpose. They produce the shaded dome together, and changing one in isolation tends to read as broken rather than different.

There is no separate tick colour. If you want ticks and labels to differ, that needs a new theme key and a change to the draw code.

Theming (sCTkThemes.json)

```
{
  "sCTkDialSelector": {
    "fg_color": ["#F1F5F9", "#0A0A0A"],
    "text_color": ["#7B93B0", "#915404"],
    "shadow_color": ["#DCE3EC", "#06070A"],
    "label_font": ["Arial", "9"],
    "dial_color": ["#C3C5C7", "#1C1E26"],
    "dial_highlight_color": ["#EAEF2", "#292A2E"],
    "dial_shadow_color": ["#9FA4A8", "#070708"],
    "dial_rim_light_color": ["#F9FAFC", "#53565A"],
    "dial_rim_shadow_color": ["#8F9396", "#040404"],
    "pointer_color": ["#7B93B0", "#915404"],

    "pressed_map": {
      "text_color": ["#1A4375", "#FF9100"],
      "shadow_color": ["#CBD5E1", "#02040A"],
      "dial_color": ["#9E9E9E", "#2A2F3D"],
      "dial_highlight_color": ["#E4E8EC", "#42454B"],
      "dial_shadow_color": ["#5C6165", "#050507"],
      "dial_rim_light_color": ["#FFFFFF", "#8E949C"],
      "dial_rim_shadow_color": ["#3E4245", "#000000"],
      "pointer_color": ["#1A4375", "#FF9100"]
    },

    "disabled_map": {
      "text_color": ["#94A3B8", "#4B5563"],
      "dial_color": ["#E2E8F0", "#1A1D24"]
    }
  }
}
```

Every key above is required — construction raises `KeyError` naming any that are missing. `pressed_map` is the exception: it is read only when the dial is built with `latching=True`.

A dial can be made to require arming before it responds. Opt in with `latching=True` and wire the double-click callbacks; the dial then starts switched off, ignores input, and draws the resting colours until armed. The `pressed_map` above holds the operational set. See [Latching](#) on the base class page for the whole mechanism, including why arming and disarming use different gestures.

A colour set at runtime survives a state change. Disable the widget and enable it again and your value is still there; clearing the property returns it to the theme's, not to whatever was set before. See [Theming](#) for the general rule. See [the base class page](#) for the shared contract.

`pointer_color` is **specific to this variant and its Range sibling**, and colours the pointer line. It was present in the theme file for a long time but read by no code path at all — the pointer drew in `text_color` instead. It is now live, so the pointer can differ from the tick labels. It has no `disabled_map` entry; a disabled pointer falls back to the disabled `text_color`.

Known Limitations

- **Knob rendering:** the body is a shaded dome, marked with a plain straight line from dead centre out to just short of the rim. An earlier version drew an arrowhead and a raised centre cap; both are gone, along with the cap's two hardcoded outline colours. See [the base class page](#).
- **`.config()` now works.** This class previously had no `config = configure` alias, so `.config(...)` bypassed every override and landed on the native widget. If existing code called it expecting no effect, it will now have one.
- **Theme colours are live for the first time.** Colours were previously read from `final_kw`, which never contained them, so every dial rendered in hardcoded fallbacks regardless of the theme file. See [reading theme colours](#).

- **Inherits `ctk.CTkFrame` and `ThemeableWidget` directly**, with no intermediate template class in between. That ordering matters: the native class comes first, so every `super()` call in the dial's own methods resolves to `CustomTkinter` rather than to the mixin.
 - **`_finalize_themeable_lifecycle()` fires at the end of the constructor**, which is what tells a Pygubu-style consumer the widget is ready. Every widget in the library does this; a missing call means an `on_first_object_cb` callback silently never runs.
 - **Wraps around.** Scrolling past the last position returns to the first, unlike the Range dial, which clamps. A mode switch is a ring of choices with no natural end.
-

Example

Below is a complete, self-contained test execution script demonstrating how to properly embed an `sCTkDialSelector` alongside custom click jump hooks and an active mode switch control panel display tracker.

```

#!/usr/bin/python3

# =====
# TESTING HARNESS IMPORTS & SETUP for Dial Rotary Switch (sCTkDialSelector)
# =====

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkLabelSecondary, sCTkDialSelector

if __name__ == "__main__":

    root = sCTk()
    root.geometry("450x350")
    root.title("Rotary Switch Selector Bench")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    # 1. Attach a live telemetry readout label
    lbl_mode_tag = sCTkLabelSecondary(base, text="Selected Mode: AM", font=("Arial", 11, "bold"))
    lbl_mode_tag.pack(pady=15)

    def my_custom_left_click():
        """Moves two positions left per click, instead of the built-in one."""
        if mode_selector.get_state() == "disabled":
            return
        mode_selector.set(mode_selector.get() - 2)

    def my_custom_right_click():
        """Moves two positions right per click, instead of the built-in one."""
        if mode_selector.get_state() == "disabled":
            return
        mode_selector.set(mode_selector.get() + 2)

    # 2. Instantiate with unique radio deck selector labels and selection trackers
    mode_selector = sCTkDialSelector(
        base,
        labels=["AM", "FM", "LSB", "USB", "CW"],
        arc_angle=180, # Half-circle step selector arc
        command=lambda idx: lbl_mode_tag.configure(text=f"Selected Mode: {mode_selector._labels[idx]}"),
        left_click_callback=my_custom_left_click,
        right_click_callback=my_custom_right_click
    )
    mode_selector.pack(expand=True, fill="none", padx=10, pady=10)

    # 3. Standard application dashboard interaction lock toggle simulation
    def toggle_widget_lock():
        current_mode = mode_selector.get_state()
        target = "disabled" if current_mode == "normal" else "normal"

        mode_selector.configure(state=target)
        btn_lock.configure(
            text="UNLOCK CHANNELS" if target == "disabled" else "LOCK SWITCH (Set 'disabled')"
        )
        print(f"Logged Verification Hook -> mode_selector.get_state() = {mode_selector.get_state()}")

    btn_lock = ctk.CTkButton(base, text="LOCK SWITCH (Set 'disabled')", command=toggle_widget_lock)
    btn_lock.pack(side="bottom", pady=10)

    # Standard test assertions routine verification sequences
    print("---- BOOT INITIALIZATION PASSTHROUGH ----")
    mode_selector.state("disabled")

```

```
print("state (Disabled Pass) =", mode_selector.get_state()) # Output: disabled

mode_selector.state("normal")
print("state (Normal Pass)   =", mode_selector.get_state()) # Output: normal
print("=====\n")

root.mainloop()
```

[Return to Table of Contents](#)

sCTkDialog

`sCTkDialog` is a popup dialog with a consistent shape: a heading, a content area you fill, and a row of action buttons. It creates its own window, so a dialog is always a window — you don't build a `Toplevel` and put a dialog in it.

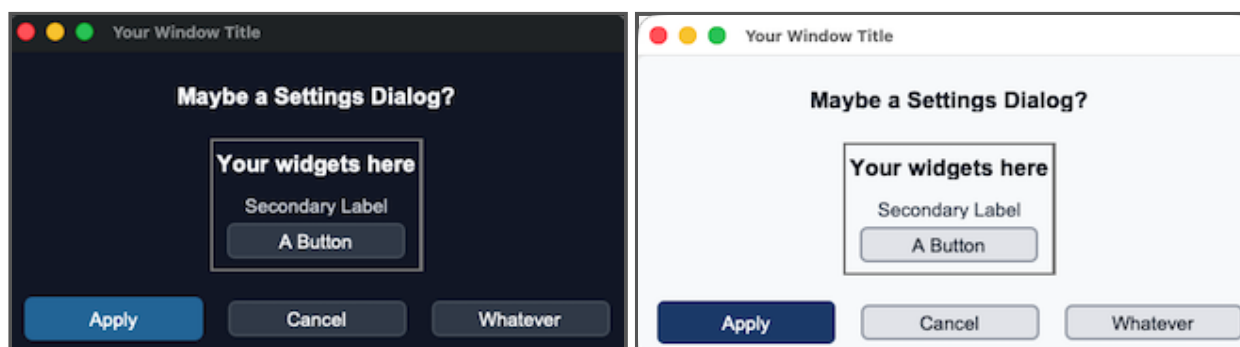


Table of Contents

- Quick start
- Constructor
- The content area
- Buttons
- Modality
- Sizing
- Placement
- Methods
- Theming (`sCTkThemes.json`)
- Pygubu Designer
- Example
- Known Limitations

Quick start

```
dialog = sCTkDialog(self, title="Station Settings", heading="Transceiver")

sCTkEntryPrimary(dialog.contentFrame).pack(padx=20, pady=10)

dialog.set_apply_button(button_command=self.save_settings)
dialog.run_and_wait()
```

Content goes into `contentFrame`. The window, the heading and the button row are already there.

Constructor

```
sCTkDialog(master=None, *, title=None, width=None, height=None,
            locate_over=None, offset_x=40, offset_y=40, modal=False,
            transient=True, heading="Heading Title", heading_anchor="center",
            heading_font=None, heading_color=None, buttons=3,
            apply_text="Apply", cancel_text="Cancel", reset_text="Reset",
            apply_command=None, cancel_command=None, reset_command=None,
            toplevel=None, **kw)
```

Parameter	Type	Default	Description
master	widget	None	Parent widget. Also the default for <code>locate_over</code> .
title	str	None	Window title bar text.
width / height	int	None	WINDOW size in pixels. Omit to size to content, subject to a 320x180 floor. These size the window, not the frame — the frame fills its window, so its own dimensions would have no effect. <code>configure()</code> and <code>cget()</code> treat them the same way.
locate_over	widget	None	The window to appear over. Defaults to master's own toplevel — see Placement .
offset_x / offset_y	int	40	Pixels right of and below <code>locate_over</code> 's top-left corner.
modal	bool	False	Block interaction with the rest of the application — see Modality .
transient	bool	True	Tie the window to its parent for the window manager.
heading	str	"Heading Title"	Text above the content area.
heading_anchor	str	"center"	"w", "e" or "center".
heading_font	tuple	None	Overrides the theme's <code>heading_font</code> for this dialog.
heading_color	color	None	Overrides the theme's <code>text_color</code> for the heading.
buttons	int	3	How many action buttons — see Buttons .
apply_text / cancel_text / reset_text	str	"Apply" / "Cancel" / "Reset"	Button labels.
apply_command / cancel_command / reset_command	callable	None	Click callbacks.
toplevel	window	None	An existing window to use instead of creating one. Rarely needed.

The content area

`contentFrame` is where your widgets go. It sits between the heading and the button row and expands to fill whatever space is left.

```
dialog = sCTkDialog(self, title="Filters")

grid = sCTkFrame(dialog.contentFrame)
grid.pack(expand=True, fill="both", padx=20, pady=10)
```

The dialog itself is an `sCTkFrame`, so packing widgets into the dialog rather than into `contentFrame` puts them beside the heading and buttons instead of between them. In Pygubu Designer this can't go wrong — widgets dropped onto a dialog land in `contentFrame` automatically.

Buttons

`buttons` selects how many the dialog shows:

Value	Buttons
3 (default)	Apply, Cancel, Reset
2	Apply, Cancel
1	Apply

Apply is always present. A dialog with no way to accept is a message box; use `sCTkMessageBox` for that.

Buttons that weren't requested are **never created**, and their attributes are `None` rather than undefined:

```
dialog = sCTkDialog(self, buttons=2)
dialog.reset_Button      # None, not AttributeError
dialog.has_button("reset") # False
```

Use `has_button()` rather than testing the attribute directly — it also handles a button removed later by `set_two_button()`.

Each button gets its callback from the constructor, or from the corresponding overridable method:

```
class SettingsDialog(sCTkDialog):
    def apply_CB(self):
        save()
        self.dialog_close()

    def cancel_CB(self):
        self.dialog_close()
```

A callback passed to the constructor takes precedence over the method. Both work; the methods suit a subclass, the constructor arguments suit a dialog built inline.

Changing the count destroys and rebuilds the row. `set_buttons()` can't add a button that was never created, so it rebuilds. Labels and commands survive, because they're recorded on the dialog rather than only on the widgets.

Modality

Two different things, easily confused:

`modal=True` blocks *interaction*. The rest of the application stops responding to the mouse and keyboard while the dialog is open, but your code carries on running.

`run_and_wait()` blocks *execution* as well. The calling code stops at that line until the dialog closes.

```
dialog = sCTkDialog(self, title="Confirm", buttons=2)
dialog.run_and_wait()          # returns when the dialog closes
```

Use `run_and_wait()` when you need the answer before continuing. Use `modal=True` alone when the dialog should be exclusive but your code has other work to do.

Neither returns a value. Store the result on the dialog, or on `self`, from your Apply callback.

Sizing

Omit width and height and the dialog sizes itself to its content. That is usually what you want — a dialog should be as big as what it holds.

```
sCTkDialog(self, title="Filters")          # sized to content
sCTkDialog(self, title="Filters", width=640) # fixed width, content height
sCTkDialog(self, title="Filters", width=640, height=400)
```

An explicit size is honoured exactly, including below the floor: asking for 200x100 gets 200x100, because you had a reason to ask. The floor applies only to a dimension being derived from content — `MIN_WIDTH 320`, `MIN_HEIGHT 180` on `sCTkDialogToplevel`, so a nearly-empty dialog still looks like a dialog rather than a sliver.

CONTENT_WIDTH sets the natural width. It's a class attribute, default 500, and it's what a content-sized dialog comes out as. Override it on a subclass for consistently wider or narrower dialogs rather than passing `width` at every call site:

```
class WideDialog(sCTkDialog):
    CONTENT_WIDTH = 720
```

Height comes entirely from the content. The content area asks for no more room than its children need, so a dialog holding two entry fields is short and one holding a long form is tall.

width and height mean the window, not the frame. `sCTkDialog` inherits `sCTkFrame`, which also has those names — but the dialog fills its own window, so the frame's dimensions have no effect. Constructor, `configure()` and `cget()` all treat them as the window's size, so the three agree.

Placement

The window positions itself relative to another window rather than in screen coordinates, because that's what dialogs want: appear over whatever opened me, nudged down and right so the parent is still visible.

`locate_over` is deliberately separate from `master`. A dialog is often parented to a frame or a controller while needing to position over the main application window:

```
sCTkDialog(self.control_panel, locate_over=self.main_window)
```

Omit it and the dialog uses `master`'s own `toplevel`, which is right most of the time.

Placement is clamped to the screen, so a large offset from a window near an edge won't push the dialog off it.

`place_over()` can be called again later to reposition.

`transient=True` (the default) ties the window to its parent for the window manager: it stays above that window, minimises with it, and usually keeps out of the taskbar. `False` gives an independent window, which is what a long-lived tool panel wants. Placement is unaffected either way.

Methods

Method	Returns	Description
<code>run_and_wait()</code>	None	Makes the dialog modal and blocks until it closes.
<code>dialog_close()</code>	None	Closes and destroys the window.
<code>on_delete_window()</code>	None	Bound to the window manager's close button. Override to intercept.
<code>set_title(title)</code>	None	Sets the window title bar text.
<code>set_heading(heading=None, anchor=None)</code>	None	Sets the heading text and alignment. None leaves either unchanged.
<code>set_heading_font(font)</code>	None	Overrides the heading font.
<code>set_heading_color(color)</code>	None	Overrides the heading colour. None restores the theme.
<code>set_buttons(count)</code>	None	Changes the button count, rebuilding the row.
<code>has_button(name)</code>	bool	Whether "apply", "cancel" or "reset" exists.
<code>set_button_text(name, text)</code>	bool	Sets one button's label, remembering it across a rebuild.
<code>set_apply_button(button_name=None, button_command=None)</code>	bool	Sets the Apply button's label and callback. False if it doesn't exist.
<code>set_cancel_button(...)</code>	bool	As above, for Cancel.
<code>set_reset_button(...)</code>	bool	As above, for Reset.
<code>set_two_button()</code>	None	Removes Reset. Prefer <code>buttons=2</code> at construction.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Accepts every constructor property above, plus any native <code>sCTkFrame</code> option.
<code>cget(name)</code>	varies	Extended to buttons, the three labels, the three commands, heading and heading_anchor.

The camelCase names from the earlier mixin-based API — `setTitle`, `setHeading`, `setApplyButton`, `runAndWait` and the rest — remain as aliases.

Theming (sCTkThemes.json)

```
{
  "sCTkDialog": {
    "fg_color": ["#F8FAFC", "#0F172A"],
    "text_color": ["#111827", "#F9F6FB"],
    "heading_font": ["Arial", 18, "bold"]
  }
}
```

All three keys are required. Construction raises `KeyError` naming the missing one.

fg_color is the background, not the foreground. That's CustomTkinter's naming, not ours, and it catches people out. `text_color` is the foreground — used for the heading, and available to your own content through `cget("text_color")`.

The values above match what a dialog would otherwise inherit — `fg_color` from `sCTkToplevel`, `text_color` and `heading_font` from `sCTkLabelPrimary` — so a dialog starts out looking like the rest of the application and you change it from there.

The heading has its own keys rather than following `sCTkLabelPrimary` because a dialog heading is a distinct role. Without them, restyling dialog headings would move every primary label in the application.

Per-dialog overrides are `heading_font` and `heading_color`, or `set_heading_font()` and `set_heading_color()` at runtime.

Pygubu Designer

Drop an `sCTkDialog` onto your layout and fill it. Widgets dropped onto it land in `contentFrame` automatically.

Every constructor property above is in the inspector. Three behaviours worth knowing:

Clearing a field restores the default. Blanking `apply_text` shows "Apply" again rather than an empty button, matching what the generated code does — the property is omitted and the constructor default applies.

Changing buttons redraws immediately, so the canvas matches the preview and the generated code.

width and height are not reflected on the design canvas. They size the *window*, and the canvas has no window — the dialog is drawn as a frame standing in for one. An empty dialog therefore sits at its natural size inside the space the canvas allotted it, and the surplus shows as pygubu's light-green preview background. The gap closes as soon as you add content, and both the preview and the generated code use the real size.

Clicking any part of the dialog selects the dialog. The heading, the buttons, the strips around them and the bare content area all select it in the widget tree. Clicking a widget you placed inside selects that widget instead — so a frame you drop into the content area to hold your own layout behaves normally.

Labels for buttons you don't have are not generated. With `buttons=2`, a `reset_text` value is kept in the design but left out of the generated file, where it would read as a label for a button that doesn't exist. Commands are *not* filtered this way — each one generates a callback stub in your file, and losing that stub because you briefly reduced the button count would be worse than the noise.

A dialog can be the main widget of a `.ui` file. Because it builds its own window, the generated `__main__` correctly creates no separate root.

Example

```
#!/usr/bin/python3
import customtkinter as ctk
from scustomtkinter import (sCTk, sCTkFrame, sCTkButtonPrimary,
                             sCTkLabelPrimary, sCTkEntryPrimary, sCTkDialog)

class SettingsDialog(sCTkDialog):
    """A dialog that reports what the user chose."""

    def __init__(self, master, **kw):
        super().__init__(master, title="Station Settings",
                          heading="Transceiver", buttons=3, **kw)
        self.result = None

        self.call_entry = sCTkEntryPrimary(
            self.contentFrame, placeholder_text="Callsign")
        self.call_entry.pack(padx=20, pady=(20, 10), fill="x")

        self.grid_entry = sCTkEntryPrimary(
            self.contentFrame, placeholder_text="Grid square")
        self.grid_entry.pack(padx=20, pady=(0, 20), fill="x")

    def apply_CB(self):
        self.result = (self.call_entry.get(), self.grid_entry.get())
        self.dialog_close()

    def cancel_CB(self):
        self.dialog_close()

    def reset_CB(self):
        self.call_entry.delete(0, "end")
        self.grid_entry.delete(0, "end")

if __name__ == "__main__":
    root = sCTk()
    root.geometry("420x220")
    root.title("sCTkDialog Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    status = sCTkLabelPrimary(base, text="No settings yet")
    status.pack(pady=10)

    def open_settings():
        dialog = SettingsDialog(base, modal=True)
        dialog.run_and_wait()
        if dialog.result:
            status.configure(text=f"{dialog.result[0]} / {dialog.result[1]}")
        else:
            status.configure(text="Cancelled")

    sCTkButtonPrimary(base, text="Settings...", command=open_settings).pack(pady=10)

    root.mainloop()
```

Known Limitations

- **A dialog is always a window.** There is no way to embed one in a frame. If you want the same heading/content/buttons arrangement inline, build it from `sCTkFrame` directly.

- **run_and_wait()** **returns nothing**. Store the result on the dialog from your Apply callback, as the example does. A return value would mean deciding what "cancelled" looks like for every caller.
- **set_two_button()** **is irreversible**. It destroys the Reset button. `set_buttons(3)` afterwards creates a fresh one, but any command set directly on the old widget rather than through `set_reset_button()` is lost.
- **The content area collapses when empty**. Its height comes from its children, so a dialog with nothing in it falls back to the window's `MIN_HEIGHT`. Intentional — an empty content area reserving 200px was what made every dialog taller than it needed to be.
- **The button row is fixed at three**. Apply, Cancel and Reset in that order, with those roles. A dialog needing different actions should rename them with the `*_text` properties rather than expecting more buttons.
- **Commands for absent buttons still appear in generated code**. With `buttons=2`, a `reset_command` is emitted and does nothing. Deliberate — see [Pygubu Designer](#).

[Return to Table of Contents](#)

sCTkFileExplorer

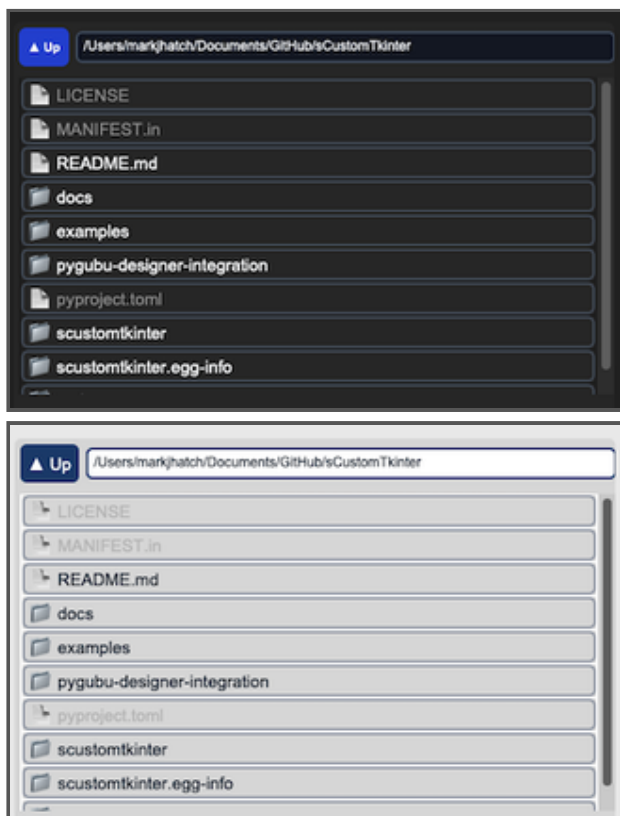
(Derived from FileExplorer class by Fastattack, 2024. This widget was made available to the community via the MIT License. Source Repository: [MoreCustomTkinterWidgets](#))

Table of Contents

- [Overview](#)
- [Constructor](#)
- [Methods](#)
- [Typing a path](#)
- [Theming \(sCTkThemes.json\)](#)
- [Example](#)
- [Known Limitations](#)

Overview

`sCTkFileExplorer` is a theme-compliant, scrollable file/folder browser — a back button, an editable current-path entry, and a scrollable list of clickable file/folder rows. It inherits `ctk.CTkFrame`, `ScrollBindingMixin`, and `ThemeableWidget`, and builds its own scrolling machinery internally (a raw `tkinter.Canvas` plus a `CTkScrollbar`) rather than composing `sCTkScrollableFrame`.



Scroll handling comes from `ScrollBindingMixin`, the library's single shared implementation. This widget supplies two hooks: `_scroll_target()` returns its own internal canvas — no `wininfo_parent()` lookup needed, unlike `sCTkScrollableFrame` — and `_scroll_layers()` assembles the widget, canvas, scrollbar and full row tree. It passes `explorer_frame` as the mixin's `content_widget`, since rows are added there rather than to the widget itself.

Bindings are automatic and self-maintaining: navigating to a new folder replaces every row widget, and the debounced `<Configure>` rebind picks them up with no explicit call. The mixin page covers the platform models, the activation mechanisms and the tuning constants.

Consolidating on the mixin fixed three problems specific to this widget. A global `bind_all("<MouseWheel>", ...)` once affected the entire application rather than just this widget. The scoped copy that replaced it walked only *one level* into the row frame, so a row's label or icon was never bound and the wheel did nothing over them. And it had no trackpad accumulator, scrolling on every raw event — trackpad scrolling here was markedly faster and coarser than everywhere else in the library, and now matches.

Constructor

```
sCTkFileExplorer(master=None, initialdir=None, type="file", filetypes=None, ...)
```

Parameter	Type	Description
master	widget	Parent container.
initialdir	str	Starting directory.
initialfile	str	File to select. A bare name resolves against <code>initialdir</code> ; an absolute path or one starting with <code>~</code> is used as given.
type	"file" / "directory"	Whether individual files are selectable, or only directories.
filetypes	list[str]	File extension filter (only meaningful when <code>type="file"</code>).
command	callable	Called with the clicked path (a string) on a single click.
double_click_command	callable	Called with (widget, path) on a double click — note this differs from <code>command</code> , which receives the path alone.
width / height	int	Overall widget dimensions.
selection_color	color	Highlight colour of the selected row. Omit to use the theme's <code>selection_color</code> .
**kwargs	—	Any native CTKFrame argument, or a theme-key override (see Theming).

```
explorer = sCTkFileExplorer(control_panel, initialdir="/Users/you/Documents", type="directory",
width=350, height=380)
explorer.pack(fill="both", expand=True)
```

filetypes requires type="file". Passing a filter while the mode is `"directory"` — which is the default — raises `ValueError`, and the same combination set in the Designer draws a "UI Mismatch" notice in place of the listing.

An earlier attempt had the widget switch to file mode silently instead, on the reasoning that asking for a filter is unambiguous. That was wrong for the Designer: a builder object cannot write the companion value back to the live tree node, so the inspector went on showing `directory` — and the `.ui` file saved that combination, making the divergence permanent rather than cosmetic. Refusing is better than correcting something the surrounding tooling cannot be told about.

`set_mode("directory")` clears any filter, since choosing directories makes one meaningless.

command receives a path, not the widget. An earlier version passed `self` (the widget instance) instead of the clicked path — confirmed and fixed, since the only real-world caller (`sCTkPathChooser`) expected a path string and would have received garbage.

Methods

Method	Returns	Description
<code>_finalize_split_bindings()</code>	None	Wires the back button, path entry, and canvas resize handling, then loads the initial directory. Auto-scheduled via <code>self.after(10, ...)</code> inside <code>__init__</code> — you don't need to call it yourself. It no longer governs scroll activation; <code>ScrollBindingMixin</code> handles that independently via <code>after_idle()</code> , which fires when Tk is actually idle rather than after a guessed delay.
<code>state(mode=None) / get_state()</code>	str	Gets or sets "normal"/"disabled", dimming the back button, path entry, scrollbar, and all rows. Disabling also stops scrolling entirely — wheel, trackpad, and scrollbar dragging — matching <code>sCTkScrollableFrame</code> .
<code>configure(**kwargs)</code>	None	Standard configuration, accepting <code>state</code> , <code>type</code> , <code>initialdir</code> , <code>initialfile</code> , <code>filetypes</code> , and <code>double_click_command</code> alongside native options.
<code>configure(name)</code>	tuple	Pygubu-style single-argument query for any of the six properties above. Previously broken twice: the implementation read <code>pname = args</code> rather than <code>args[0]</code> , so every comparison tested a tuple against a string; and the fall-through forwarded the property <i>name</i> to native <code>CTkFrame.configure()</code> , which takes it as <code>require_redraw</code> and returns None. Pygubu could read none of them.

There's currently no public method for programmatic navigation from outside the widget — `path_to_show` (a `StringVar`) has no automatic refresh trace of its own (unlike `selected_path`), so navigating externally means setting it *and* explicitly calling the private `_fill_explorer()` afterward, matching the pattern used internally by the back button. This is a real API gap, not a documented feature.

Typing a path

The path entry is editable, and a path typed into it is applied on **Return**, on keypad Enter, or when focus leaves the field.

A value is accepted only if it exists. Focus-out fires whenever you click elsewhere, so a half-typed path must not replace a good one. `~` is expanded. A directory navigates there; a file navigates to its folder, and is also selected when `type` is "file".

Three fixes were needed here. `~` was never expanded, so typing `~/Documents` silently did nothing. Only directories were accepted regardless of this widget's own `type`, so a file explorer would not let you type a file path. And there was no focus-out binding, so tabbing away left the text unapplied.

`sCTkPathChooser` behaves identically. The two are the same control, one with a button.

The entry scrolls to show the end of a long path, so the filename stays visible when you select a file. It previously showed the start and the filename ran off the right.

Driven by a trace on `selected_path` rather than by the existing one, which is gated by an internal flag switched off whenever the widget sets the path itself — exactly the case where the view needs moving. The move is also re-applied once Tk is idle, because the immediate call runs before the entry has recomputed the width of its new text and lands part-way through the filename.

Unlike `sCTkPathChooser` there is no `justify` property; the filename is the useful end.

Theming (sCTkThemes.json)

```
{
  "sCTkFileExplorer": {
    "btn_font": ["Arial", 11, "bold"],
    "entry_font": ["Arial", 12, "normal"],
    "btn_fg": ["#3B82F6", "#1D4ED8"],
    "btn_hover": ["#2563EB", "#1E40AF"],
    "btn_text_color": ["#FFFFFF", "#F9F9FB"],
    "btn_border_color": ["#1E3A8A", "#1E3A8A"],
    "entry_fg": ["#FFFFFF", "#111827"],
    "entry_text_color": ["#1F2937", "#F9F9FB"],
    "entry_border_color": ["#CBD5E1", "#475569"],
    "row_active_text": ["#1F2937", "#F9F9FB"],
    "row_dimmed_text": ["#94A3B8", "#64748B"],
    "button_color": ["#64748B", "#4B5563"],
    "fg_color": ["gray86", "gray17"],
    "selection_color": ["#3B82F6", "#1D4ED8"],
    "disabled_map": {
      "btn_fg": ["#CBD5E1", "#334155"],
      "btn_border_color": ["#CBD5E1", "#334155"],
      "btn_text_color": ["#94A3B8", "#64748B"],
      "entry_fg": ["#F3F4F6", "#1F2937"],
      "entry_border_color": ["#CBD5E1", "#475569"],
      "entry_text_color": ["#94A3B8", "#64748B"],
      "row_dimmed_text": ["#5A6672", "#3A4552"],
      "button_color": ["#CBD5E1", "#334155"]
    }
  }
}
```

button_color is required at the top level and in **disabled_map**. This is a harder requirement than it looks: `_process_live_theme_repaint()` is bound to `<Visibility>`, so it fires essentially every time the widget is displayed, not only when explicitly disabled. A theme block missing `button_color` therefore raises `KeyError` on first display, not merely on disable. The values shown above (`["#64748B", "#4B5563"]` normal, `["#CBD5E1", "#334155"]` disabled) are the suggested pair.

`button_color` controls the internal scrollbar's color, distinct from `btn_fg` (the back button).

selection_color is the highlighted row, and is deliberately separate from `btn_fg`. That key colours the navigation buttons as well, so exposing it directly would have meant changing the selection also recoloured Back and the path controls. It has no `disabled_map` entry: a disabled explorer draws no highlight at all, rather than a dimmed one, so there is nothing for a disabled value to apply to.

fg_color is now in the block. It was absent, so the background came from native `CTkFrame`'s own default and there was nothing for the Designer to revert to when the field was cleared — the query reported the *current* colour as the default, and clearing set it to what it already was. The value above matches CustomTkinter's default, so the appearance is unchanged.

Setting `fg_color` also repaints the internal canvas. It previously reached only the outer frame, because the canvas colour was recomputed on appearance-mode change and at construction but not when the property changed.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

`row_active_text` / `row_dimmed_text` control file/folder row text color — `row_active_text` for a normal, selectable row; `row_dimmed_text` for either a row excluded by the current filter, or every row when the whole widget is disabled. Both required at the top level; `row_dimmed_text` is also hard-required in `disabled_map` for the whole-widget-disabled case.

Only `button_color` and `row_dimmed_text` genuinely hard-fail if missing from `disabled_map`. The other six disabled-state keys (`btn_fg`, `btn_border_color`, `btn_text_color`, `entry_fg`, `entry_border_color`, `entry_text_color`) gracefully fall back to their top-level/normal value if `disabled_map` doesn't override them — not a crash risk, just means that specific property simply won't visually change when disabled unless you give it a distinct value.

The internal raw `Canvas`'s background color isn't part of this theme block at all — it's derived from this widget's own `fg_color`, falling back to a fixed neutral pair (`#1C1C1C` dark / `#F3F4F6` light) if `fg_color` is `"transparent"`, since a raw `Canvas` can't render CTK's transparent pseudo-value. This is a different kind of fallback than the ones above — not a theme gap, but a genuine "needs an actual renderable color" situation, the same accepted pattern used in `sCTkFrameLabeledPrimary`'s scrollbar hiding.

Example

```
from scustomtkinter import sCTk, sCTkFrame, sCTkFileExplorer, sCTkLabelPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("420x480")
    root.title("FileExplorer Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    status = sCTkLabelPrimary(base, text="Selected: (none yet)")
    status.pack(anchor="w", pady=(0, 8))

    explorer = sCTkFileExplorer(
        base, type="directory",
        command=lambda path: status.configure(text=f"Selected: {path}"),
    )
    explorer.pack(expand=True, fill="both")

    root.mainloop()
```

Known Limitations

- No public method for programmatic navigation — see [Methods](#) above.

- **The inspector does not follow the widget's own rule.** `filetypes` and `type` constrain each other, but a builder object cannot write the companion value back to the Designer's copy of the properties, so the two can be left in a combination the widget refuses. The widget complains rather than correcting silently, which is why this is visible rather than hidden.
- **`command` and `double_click_command` have different signatures** — the first receives the path, the second receives the widget and the path. A wart rather than a design: `command` was changed at some point from passing the widget to passing the path, and the double-click callback was not changed with it. `sCTkPathChooser` reads `args[-1]` to work around it.
- Missing a required theme key raises `KeyError` at first use, naming exactly which key and whether it's needed at the top level or in `disabled_map`.
- **The debounced rebind also runs on genuine resizes.** `<Configure>` on the row frame doesn't distinguish "rows were added" from "the window was dragged", so resizing rebinds too. One coalesced pass rather than one per event, but on a very large directory it isn't free.
- **The internal Canvas is a raw `tkinter.Canvas`**, not a themed widget, so its background is derived rather than themed — see the note at the end of [Theming](#).
- **The scrollbar stays visible when disabled, just inert.** It can't be dragged, but it isn't hidden — CustomTkinter's scrollbar has no native disabled state to lock. Same limitation as `sCTkScrollableFrame`.

Fixed: dragging the scrollbar when the files didn't fill the frame used to push the rows down to the bottom, leaving empty space above them. The scroll region was set straight from `bbox("all")`, which is *shorter* than the visible canvas when content is short, and Tk will still scroll within an undersized region. The region is now grown to at least the canvas height in that case, so `yview` has nowhere to go and scrolling correctly does nothing.

[Return to Table of Contents](#)

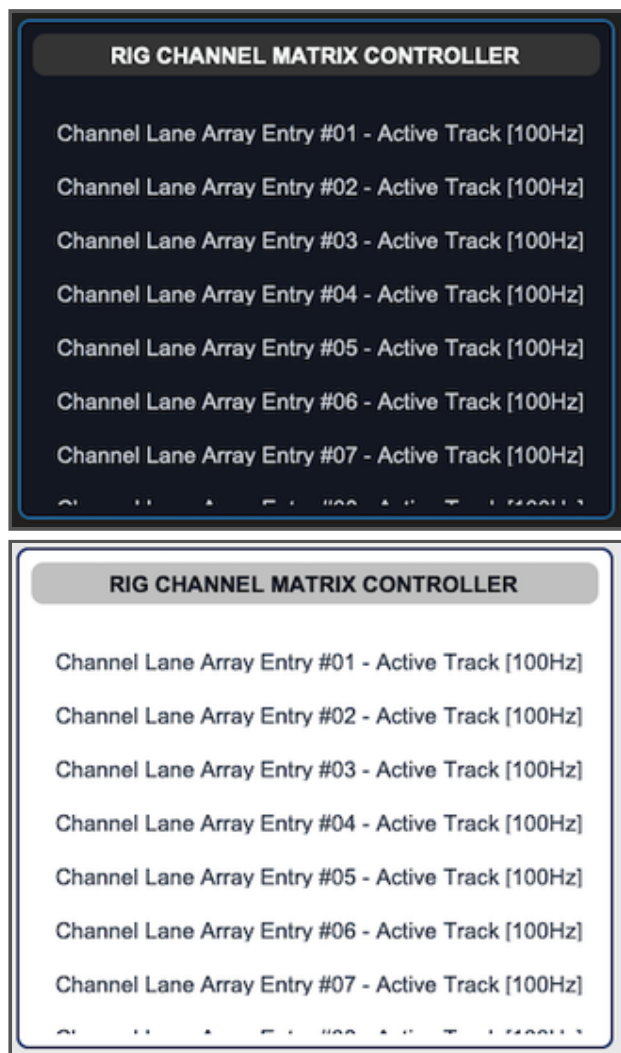
sCTkFrameLabeledPrimary

Table of Contents

- Overview
- Constructor
- Methods
- In Pygubu Designer
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

`sCTkFrameLabeledPrimary` is a themeable, high-emphasis labeled container panel — the more prominent of the library's two labeled frame tiers (see also `sCTkFrameLabeledSecondary`). It's built on `customtkinter.CTkScrollableFrame`, but deliberately used purely for its native title-label feature — the model here is `ttk.LabelFrame`, which never scrolls. Scrolling is intentionally suppressed; this is a labeled, bordered panel, not a scroll viewport.



Constructor

```
sCTkFrameLabeledPrimary(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
**kwargs	—	—	Any native CTKScrollableFrame argument (most usefully label_text, the panel's title), or an override for one of the theme keys listed under Theming .

```
channel_panel = sCTkFrameLabeledPrimary(
    master=control_root,
    label_text="Channel Settings",
)
channel_panel.pack(expand=True, fill="both", padx=25, pady=25)
```

Methods

Method	Returns	Description
state(mode=None)	str	Gets or sets the widget's visual "disabled" state. This is purely cosmetic — plain frame-family widgets have no native interactivity to lock. Disabling this panel does not automatically disable widgets placed inside it; that's the caller's responsibility, the same pattern used in this project's own test harness (loop over the panel's children and call .configure(state=...) on each one).
get_state()	str	Equivalent to calling state() with no argument.
configure(**kwargs) / config(**kwargs)	varies	Standard widget configuration, plus: passing state=... routes to state(). Calling configure("propname") with a single property name returns a Tkinter-style query tuple for state, fg_color, border_color, and label_text_color.
wininfo_children(include_private=False)	list	By default, filters out children whose exact class name is "CTkLabel", "Label", "CTkFrame", or "Frame" — internal furniture CTKScrollableFrame creates for its own title row and canvas wrapper. Pass include_private=True for the raw, unfiltered list. Known limitation: this is a class-name check, not an identity check — a plain, un-themed customtkinter.CTkLabel/CTkFrame added directly as a child would be filtered out too, since its class name matches. Themed sCTk-prefixed widgets are unaffected.

Method	Returns	Description
<code>get_children()</code>	list	Equivalent to <code>winfo_children(include_private=False)</code> .
<code>get_all_children()</code>	list	Equivalent to <code>winfo_children(include_private=True)</code> .
<code>get_container()</code>	self	Returns the widget itself. Provided for API symmetry with composite widgets (like <code>sCTkOptionMenuSecondary</code>) that wrap a separate inner container.

In Pygubu Designer

state is now a property in the inspector. The widget has always implemented it, but the builder object never declared it — so a panel could only be disabled from code. It offers `normal` and `disabled`, defaulting to `normal`.

This panel cannot be selected by clicking it on the canvas. Select it in the widget tree instead. A widget dropped *inside* it selects normally, so this affects only the container itself.

That is a dead end rather than an open bug. `CTkScrollableFrame` — which this widget is built on — makes the widget the *inner* frame inside a canvas owned by a separate outer frame, so the surface you click is the widget's parent and it has no real children of its own. Forwarding the click, binding the canvas, and a transparent overlay were each tried and each is closed off by something specific; `dev/docs/Developing.md` records which.

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block, including `label_font` and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every state() change** — `fg_color`, `border_color`, `label_text_color`, `border_width`, and `label_font` are recomputed from the theme's normal values or its `disabled_map`.

```
{
  "sCTkFrameLabeledPrimary": {
    "border_width": 2,
    "border_color": ["#1A4375", "#2471A3"],
    "fg_color": ["#FFFFFF", "#111827"],
    "corner_radius": 8,
    "label_font": ["Arial", 15, "bold"],
    "label_text_color": ["#111827", "#F9FAFB"],
    "disabled_map": {
      "border_color": ["#CBD5E1", "#374151"],
      "label_text_color": ["#94A3B8", "#64748B"]
    }
  }
}
```

On the internal scrollbar: since this widget is built on `CTkScrollableFrame`, a scrollbar exists internally even though scrolling isn't the intent. It's suppressed by matching its colors to the frame's background and collapsing its width to `0`. This is a workaround, not a true disable — confirmed by direct investigation, CustomTkinter's native scrollbar has no disabled state to lock in the first place, even on an unwrapped `CTkScrollableFrame`. Matching colors and zeroing width is the closest achievable approximation.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Safe to use as a base class for your own composite widgets. If you build a composite widget by inheriting `sCTkFrameLabeledPrimary` directly, construction is protected on two fronts: a run-once guard in `ThemeableWidget.__init__` stops your composite's own `final_kw` from being silently overwritten if your widget explicitly calls `ThemeableWidget.__init__` before `super().__init__()`; and this widget's own constructor only forwards the specific keys native `CTkScrollableFrame` actually accepts (confirmed directly against CustomTkinter's source, which has no fallback `**kwargs` at all — every parameter is explicitly named, so this matters more here than for most widgets). This only matters for that composition pattern — constructing a plain `sCTkFrameLabeledPrimary` directly is unaffected either way.

Example

```
from scustomtkinter import sCTkButtonPrimary, sCTkLabelSecondary, sCTk, sCTkFrameLabeledPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x450")
    root.title("FrameLabeledPrimary Example")

    channel_panel = sCTkFrameLabeledPrimary(root, label_text="Channel Settings")
    channel_panel.pack(expand=True, fill="both", padx=25, pady=25)

    for i in range(1, 6):
        item = sCTkLabelSecondary(channel_panel, text=f"Setting {i}")
        item.pack(pady=4, fill="x", padx=10)

    def toggle_panel_state():
        target = "disabled" if channel_panel.get_state() == "normal" else "normal"
        channel_panel.configure(state=target)

        # Disabling the panel is purely cosmetic -- cascade to children explicitly.
        for child in channel_panel.get_children():
            if hasattr(child, "configure"):
                child.configure(state=target)

        toggle_btn.configure(text="Enable Panel" if target == "disabled" else "Disable Panel")

    toggle_btn = sCTkButtonPrimary(root, text="Disable Panel", command=toggle_panel_state)
    toggle_btn.pack(pady=15)

    root.mainloop()
```

Known Limitations

- Disabling this widget is purely cosmetic — it does not lock interactivity, and does not cascade to child widgets automatically.
- The internal scrollbar cannot be truly disabled (a CustomTkinter limitation, confirmed by direct investigation, not something this wrapper can work around) — only visually hidden via color-matching and zero width.
- `winfo_children()`'s default filtering is a class-name check, not an identity check — see the Methods table above for the specific edge case this can miss.
- **Fixed:** `configure("fg_color")` used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair.
- **Not selectable by clicking on the design canvas** — see [In Pygubu Designer](#).

[Return to Table of Contents](#)

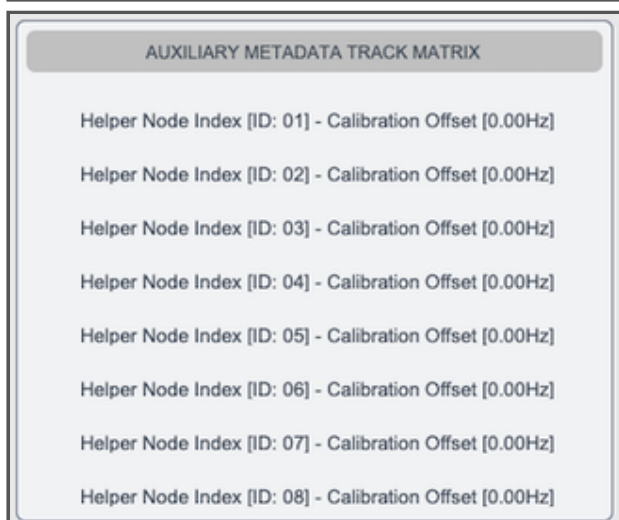
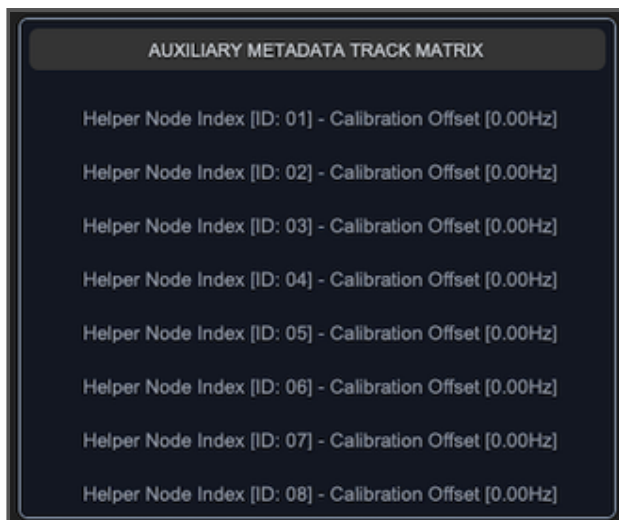
sCTkFrameLabeledSecondary

Table of Contents

- Overview
 - Constructor
 - Methods
 - In Pygubu Designer
 - Theming (sCTkThemes.json)
 - Example
 - Known Limitations
-

Overview

`sCTkFrameLabeledSecondary` is a themeable, lower-emphasis labeled container panel — see also `sCTkFrameLabeledPrimary`. It's built on `customtkinter.CTkScrollableFrame`, but deliberately used purely for its native title-label feature — the model here is `ttk.LabelFrame`, which never scrolls. Scrolling is intentionally suppressed; this is a labeled, bordered panel, not a scroll viewport.



Constructor

```
sCTkFrameLabeledSecondary(master=None, **kwargs)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
**kwargs	—	—	Any native CtkScrollableFrame argument (most usefully label_text, the panel's title), or an override for one of the theme keys listed under Theming .

```
notes_panel = sCTkFrameLabeledSecondary(
    master=control_root,
    label_text="Notes",
)
notes_panel.pack(expand=True, fill="both", padx=25, pady=25)
```

Methods

Method	Returns	Description
state(mode=None)	str	Gets or sets the widget's visual "disabled" state. Purely cosmetic — does not lock interactivity, and does not cascade to child widgets automatically.
get_state()	str	Equivalent to calling state() with no argument.
configure(**kwargs) / config(**kwargs)	varies	Standard widget configuration, plus: passing state=... routes to state(). Calling configure("propname") with a single property name returns a Tkinter-style query tuple for state, fg_color, border_color, and label_text_color.
wininfo_children(include_private=False)	list	By default, filters out children whose exact class name is "CTkLabel", "Label", "CTkFrame", or "Frame" — internal furniture CtkScrollableFrame creates for its own title row and canvas wrapper. Pass include_private=True for the raw, unfiltered list. Same class-name-based known limitation as sCTkFrameLabeledPrimary — see that widget's docs for the specific edge case.
get_children()	list	Equivalent to wininfo_children(include_private=False).
get_all_children()	list	Equivalent to wininfo_children(include_private=True).
get_container()	self	Returns the widget itself. Provided for API symmetry with composite widgets that wrap a separate inner container.

In Pygubu Designer

state is now a property in the inspector. The widget has always implemented it, but the builder object never declared it — so a panel could only be disabled from code. It offers `normal` and `disabled`, defaulting to `normal`.

This panel cannot be selected by clicking it on the canvas. Select it in the widget tree instead. A widget dropped *inside* it selects normally, so this affects only the container itself.

That is a dead end rather than an open bug. `CTkScrollableFrame` — which this widget is built on — makes the widget the *inner* frame inside a canvas owned by a separate outer frame, so the surface you click is the widget's parent and it has no real children of its own. Forwarding the click, binding the canvas, and a transparent overlay were each tried and each is closed off by something specific; `dev/docs/Developing.md` records which.

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every state() change** — `fg_color`, `border_color`, `label_text_color`, `border_width`, and `label_font` are recomputed from the theme's normal values or its `disabled_map`.

```
{
  "sCTkFrameLabeledSecondary": {
    "border_width": 1,
    "border_color": ["#64748B", "#94A3B8"],
    "fg_color": ["#F3F4F6", "#111827"],
    "corner_radius": 6,
    "label_font": ["Arial", 12, "normal"],
    "label_text_color": ["#4B5563", "#D1D5DB"],
    "disabled_map": {
      "border_color": ["#CBD5E1", "#374151"],
      "label_text_color": ["#94A3B8", "#64748B"]
    }
  }
}
```

On the internal scrollbar: same situation as `sCTkFrameLabeledPrimary` — a scrollbar exists internally since this is built on `CTkScrollableFrame`, even though scrolling isn't the intent. It's suppressed by matching its colors to the frame's background and collapsing its width to `0`, since CustomTkinter's native scrollbar has no disabled state to lock in the first place.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Safe to use as a base class for your own composite widgets. Same protection as `sCTkFrameLabeledPrimary` — see that widget's docs for the full reasoning (the run-once guard in `ThemeableWidget.__init__`, plus this widget's own constructor filtering keys down to only what native `CTkScrollableFrame` actually accepts before its own constructor call).

Example

```
from scustomtkinter import sCTkButtonPrimary, sCTkLabelSecondary, sCTk, sCTkFrameLabeledSecondary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x450")
    root.title("FrameLabeledSecondary Example")

    notes_panel = sCTkFrameLabeledSecondary(root, label_text="Notes")
    notes_panel.pack(expand=True, fill="both", padx=25, pady=25)

    for i in range(1, 6):
        item = sCTkLabelSecondary(notes_panel, text=f"Note {i}")
        item.pack(pady=4, fill="x", padx=10)

    def toggle_panel_state():
        target = "disabled" if notes_panel.get_state() == "normal" else "normal"
        notes_panel.configure(state=target)

        # Disabling the panel is purely cosmetic -- cascade to children explicitly.
        for child in notes_panel.get_children():
            if hasattr(child, "configure"):
                child.configure(state=target)

        toggle_btn.configure(text="Enable Panel" if target == "disabled" else "Disable Panel")

    toggle_btn = sCTkButtonPrimary(root, text="Disable Panel", command=toggle_panel_state)
    toggle_btn.pack(pady=15)

    root.mainloop()
```

Known Limitations

- Disabling this widget is purely cosmetic — it does not lock interactivity, and does not cascade to child widgets automatically.
- The internal scrollbar cannot be truly disabled (a CustomTkinter limitation, confirmed by direct investigation) — only visually hidden via color-matching and zero width.
- `wininfo_children()` 's default filtering is a class-name check, not an identity check — see `sCTkFrameLabeledPrimary` 's docs for the specific edge case this can miss.
- **Fixed:** `configure("fg_color")` used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair.
- **Not selectable by clicking on the design canvas** — see [In Pygubu Designer](#).

[Return to Table of Contents](#)

sCTkMessageBox

(Derived from Separator class by Fastattack, 2024. This widget was made available to the community via the MIT License. Source Repository: [MoreCustomTkinterWidgets](#))

Table of Contents

- [Constructor](#)
- [Methods](#)
- [Simple Syntax Quick-Reference Guide](#)
- [Centralized Stylesheet Setup](#)
- [Layout and Text Wrapping Rules](#)
- [Configuration](#)
- [Example](#)

The `sCTkMessageBox` is an advanced, themeable dialog window system designed to provide critical messages to the user. It replaces standard OS message alerts with modular, center-positioned dialogue boxes featuring dynamic text-wrapping, automated parent window tracking calculations, custom asset handling, and support for dual high-contrast action selection layouts that return boolean runtime parameters.



Constructor

```
sCTkMessageBox(title, message, typ, master=None, buttons="ok", ok_text="Ok", yes_text="Yes", no_text="No", width=400)
```

Parameter Name	Data Type	Default Value	Description
title	str	<i>Required</i>	Text displayed inside the top operating window header bar title deck.
message	str	<i>Required</i>	Body text string message container paragraph to display inside the prompt panel.

Parameter Name	Data Type	Default Value	Description
typ	str	<i>Required</i>	Alert asset track type classification identifier. Accepts "info", "warning", or "error".
master	any	None	Reference pointer tracking your root window or parent sCTkFrame to calculate centering bounds.
buttons	str	"ok"	Layout selection control mapping. Accepts "ok" (single center prompt) or "yes_no" (twin balanced selections).
ok_text	str	"Ok"	Custom display string label mapped to the single button layout option track.
yes_text	str	"Yes"	Display string assigned to the primary confirmation button choice track.
no_text	str	"No"	Display string assigned to the secondary dismissal button choice track.
width	int	400	Manual window width boundary tracking restriction limit measured in pixels.

Methods

To launch modal dialog blocks quickly inside callback triggers without handling complete class instantiations manually, utilize these pre-wired shortcuts via the `messagebox` namespace proxy:

Standard Alert Prompts (Returns True upon closure)

```
sCTkMessageBox.showinfo(title, message, ok_text="Ok", width=400, master=root)
sCTkMessageBox.showwarning(title, message, ok_text="Ok", width=400, master=root)
sCTkMessageBox.showerror(title, message, ok_text="Ok", width=400, master=root)
```

Confirmation Prompt Shortcuts (Returns primitive Python True or False boolean states)

```
sCTkMessageBox.askyesno(title, message, yes_text="Yes", no_text="No", width=400, master=root)
sCTkMessageBox.askwarningyesno(title, message, yes_text="Yes", no_text="No", width=400, master=root)
sCTkMessageBox.askerroryesno(title, message, yes_text="Yes", no_text="No", width=400, master=root)
```

Simple Syntax Quick-Reference Guide

Below are clean, minimal use-cases showcasing how to call each convenience shortcut using the standardized `messagebox` proxy engine.

1. sCTkMessageBox.showinfo

Used for general application notifications, status confirmations, and completions.

```
from scustomtkinter import sCTkMessageBox

# Displays a standard informative dialog popup
sCTkMessageBox.showinfo("System Init", "Satellite link successfully established.", master=root)
```

2. sCTkMessageBox.showwarning

Used to display alert parameters, non-fatal operational boundary breaches, or layout cautions.

```
from scustomtkinter import sCTkMessageBox

# Displays a warning alert box with a custom approval button text
sCTkMessageBox.showwarning("Battery Low", "Backup power source dropped below 15%.", ok_text="Acknowledge", master=root)
```

3. sCTkMessageBox.showerror

Used to halt operations when a severe terminal failure or unhandled exception block is triggered.

```
from scustomtkinter import sCTkMessageBox

# Displays a fatal critical error box
sCTkMessageBox.showerror("TX Failure", "Transmitter hardware thermal overload detected.", master=root)
```

4. sCTkMessageBox.askyesno

Launches a standard query dialogue window, returning a boolean flag based on the user's action.

```
from scustomtkinter import sCTkMessageBox

# Captures true/false verification states
if sCTkMessageBox.askyesno("Log Session", "Do you wish to save the active telemetry log files?",
master=root):
    print("User clicked YES: Executing write loop..")
else:
    print("User clicked NO: Dropping record data...")
```

5. sCTkMessageBox.askwarningyesno

Launches a critical query box carrying high-visibility alert graphics for destructive actions.

```
from scustomtkinter import sCTkMessageBox

# Captures permission states for hazardous overrides
override_allowed = messagebox.askwarningyesno(
    "Frequency Sync",
    "VF0 phase lock is currently unstable. Force manual override?",
    yes_text="Force Override",
    no_text="Abort Scan",
    master=root
)
```

6. sCTkMessageBox.askerroryesno

Launches an error-status confirmation panel, typical for prompt actions following a hard code drop.

```

from scustomtkinter import sCTkMessageBox

# Captures choice states to run system self-healing scripts
if sCTkMessageBox.askerroryesno("Cascade Failure", "Buffer buffer overflow hit. Attempt a cold
reset?", master=root):
    # Execute recovery sequence...
    pass

```

Theming (sCTkThemes.json)

```

{
  "sCTkMessageBox": {
    "fg_color": ["#F1F5F9", "#1C1C1C"],
    "font": ["Arial", 14],
    "text_color": ["#1A1A1A", "#E5E5E5"]
  }
}

```

Every key above is required. Construction raises `KeyError` naming the missing one, rather than substituting a plausible default that would make an incomplete block look merely slightly-off.

`font` and `text_color` style the message label. `fg_color` is the dialog window background, and is **new** — this widget previously forwarded its raw constructor keywords to native `CTkToplevel` rather than the resolved theme keywords, so the theme block never reached the window at all and the dialog rendered in CustomTkinter's own default background. `ThemeableWidget`'s resolution work was discarded for everything except the two label keys read back manually.

Keyword filtering. Theme keywords are now filtered against a whitelist before the native constructor sees them, because `CTkToplevel` names only `fg_color` explicitly and passes everything else through to `tkinter.Toplevel`, which raises `TclError` on any option it doesn't recognise. This closes a latent crash as well: a caller passing `font=` to this widget would previously have had it forwarded straight through.

There is no `disabled_map` and no `state()`. This is a modal dialog — it grabs input on construction and destroys itself on dismissal, so there is no interval in which a disabled appearance would mean anything.

Layout and Text Wrapping Rules

To completely bypass CustomTkinter's internal multi-line font calculation limitations, this widget uses Python's native `textwrap` module to inject hard newline coordinates before passing layout parameters to your primary text components.

Observe these implementation traits:

- **Horizontal Capsule Brackets:** When `buttons="yes_no"` is active, Column 0 and Column 1 utilize an interlocking `uniform="dialog_buttons"` constraint map. This completely locks both buttons to an identical layout grid pixel width, regardless of text length mismatches.
- **Vertical Safety Gutter:** Text layout nodes use `padx=(10, 35)` paired alongside a calculated character width subtraction map. This forces word bounds to drop downwards well before interacting with the physical window frame margin boundary.
- **Autonomous Resizing:** The `_center_window` geometry calculations lock your custom manual `width` pixel profile constraint, but query the active required widget layout height parameters dynamically via `winfo_reqheight()`. This allows window frames to expand or shrink vertically based on your text content volume requirements automatically.

Configuration

`configure()` and `config()` behave as they do elsewhere in the library: keyword arguments are applied normally, a single positional dict is merged into them, and any other single positional value is forwarded to the native widget.

Three separate defects were fixed here, all silent:

- **`super().configure(args)` passed the whole tuple** as one positional argument instead of unwrapping it, so every single-argument call forwarded a malformed value.
 - **`if args and isinstance(args, dict)`** — `args` is always a tuple, so that branch could never fire and the dict form of `configure()` was dead code.
 - **No `config = configure` alias existed**, so `.config(...)` bypassed the override entirely and landed on the native widget. Tkinter binds `.config` as a separate class attribute; it does not track a subclass's override.
-

Example

Below is a complete, self-contained test execution script demonstrating how to properly map shortcut handlers, custom text boundaries, and dynamic boolean feedback out of an interactive transceiver dashboard setup.

```

#!/usr/bin/python3
# =====
# TESTING HARNESS IMPORTS & SETUP for MessageBox
# =====

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkMessageBox

if __name__ == "__main__":
    root = sCTk()
    root.geometry("300x520")
    root.title("Message Example")

    long_msg = "Warning: The VF0 phase lock loop has lost lock synchronization with the master
synthesizer. Override?"

    # Clean functional callbacks using the messagebox namespace!
    def trigger_info_ask():
        print(f"Feedback: {sCTkMessageBox.askyesno('Info Query', 'Log parameter data?',
yes_text='Log', no_text='Skip', master=root)}")

    def trigger_warning_ask():
        print(f"Feedback: {sCTkMessageBox.askwarningyesno('Band Switch', long_msg,
yes_text='Override', no_text='Drop', width=450, master=root)}")

    def trigger_error_ask():
        print(f"Feedback: {sCTkMessageBox.askerroryesno('Fatal Error', 'Attempt buffer cold reset?',
yes_text='Reset', no_text='Quit', master=root)}")

    # Native drop-in style execution pass!
    sCTkButtonPrimary(root, text="Test Info (OK)", width=200, command=lambda:
sCTkMessageBox.showinfo("Message Example", "Short statement alert.", ok_text="Acknowledge", master=root)).pack(pady=8)
    sCTkButtonPrimary(root, text="Test Info (Yes/No)", width=200,
command=trigger_info_ask).pack(pady=(8, 25))
    sCTkButtonPrimary(root, text="Test Warning (OK)", width=200, command=lambda:
sCTkMessageBox.showwarning("Warning", "Listen carefully", ok_text="Proceed",
master=root)).pack(pady=8)
    sCTkButtonPrimary(root, text="Test Warning (Yes/No)", width=200, command=trigger_warning_ask).pack
(pady=(8, 25))
    sCTkButtonPrimary(root, text="Test Error (OK)", width=200, command=lambda:
sCTkMessageBox.showerror("Error", "Dead meat", ok_text="Close", master=root)).pack(pady=8)
    sCTkButtonPrimary(root, text="Test Error (Yes/No)", width=200, command=trigger_error_ask).pack(pady=8)

    root.mainloop()

```

[Return to Table of Contents](#)

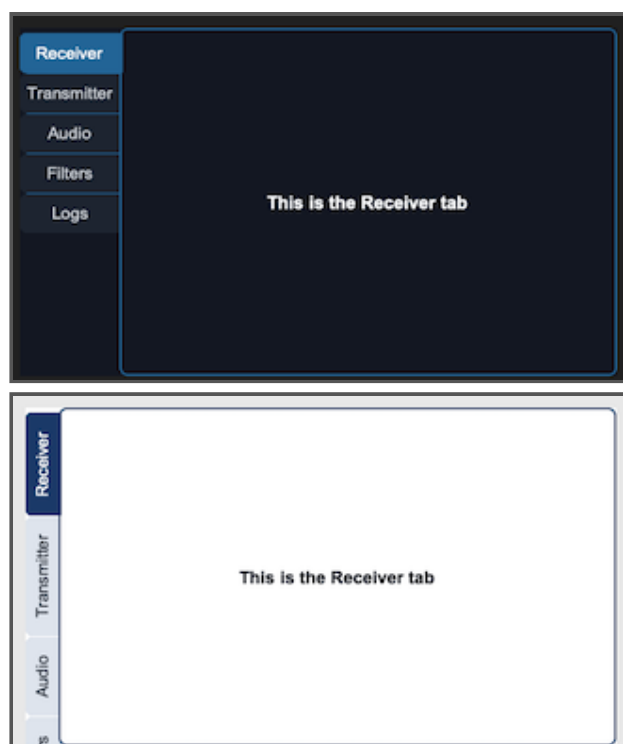
sCTkNotebook

Table of Contents

- Overview
 - Constructor
 - Methods
 - Tab Pages
 - Appearance
 - Pygubu Designer
 - Theming (sCTkThemes.json)
 - Example
 - Known Limitations
-

Overview

sCTkNotebook is a multi-page container with its tabs down the left or right edge, rather than across the top. The same idea as `sCTkTabview`, turned ninety degrees.



Use it when a panel has more pages than a horizontal strip can show, or when the layout has vertical space to spare and horizontal space to protect.

It is not a subclass of `sCTkTabview`. Native `CTkTabview` builds its strip from a `CTkSegmentedButton`, which lays its buttons out in a single grid row and has no vertical mode. The geometry, the selection handling and the colours all come from that widget, so reusing it would mean overriding nearly all of it. This class owns its strip outright.

Everything is drawn on one canvas. Tab labels run along the strip rather than across it, so the text is rotated — and a Tk button cannot rotate its text. That alone would only need a canvas for the strip, but the page outline has to break where the selected tab meets it, and a border drawn by a frame cannot have a gap cut in it. Outline and tabs are therefore drawn together, with the content pages placed on top.

Constructor

```
sCTkNotebook(master=None, side="left", tab_width=34, tab_style="rounded",
              text_orientation="auto", show_page_border=True,
              show_tab_separators=False, state="normal", **kw)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
side	str	"left"	Which edge the strip sits on, "left" or "right".
tab_width	int	34	Width of the strip in pixels. See Appearance for why this is explicit rather than measured.
tab_style	str	"rounded"	"rounded" or "angled".
text_orientation	str	"auto"	"auto", "up", "down" or "horizontal".
show_page_border	bool	True	Draws the outline around the page, broken where the selected tab meets it.
show_tab_separators	bool	False	A line between adjacent tabs.
state	str	"normal"	"normal" or "disabled".
**kw	—	—	Native CTkFrame arguments, or theme-key overrides.

```
notebook = sCTkNotebook(panel, side="left", tab_width=34)
notebook.pack(expand=True, fill="both")

page = notebook.add("Receiver")
sCTkLabelPrimary(page, text="Receiver settings").pack(padx=20, pady=20)
```

Methods

Method	Returns	Description
add(name)	sCTkFrame	Creates a tab and returns its content page directly — no separate <code>tab()</code> call needed. Raises <code>ValueError</code> on a name already in use.
tab(name)	sCTkFrame	The page for an existing tab. Raises <code>KeyError</code> if there is none.
delete(name)	—	Removes a tab and destroys its page. If it was selected, the next remaining tab is selected instead.
rename(old, new)	—	Renames a tab, keeping its position and its page. Renaming by deleting and re-adding would move the tab to the end and destroy its children.
set(name)	—	Selects a tab and raises its page.

Method	Returns	Description
<code>get()</code>	<code>str</code>	The selected tab's name, or <code>None</code> when there are no tabs.
<code>tabs()</code>	<code>list[str]</code>	Tab names, in order.
<code>state(mode=None)</code>	<code>str</code>	Gets or sets "normal"/"disabled". Disabling dims the strip and the outline and stops tab selection.
<code>get_state()</code>	<code>str</code>	Equivalent to <code>state()</code> with no argument.
<code>required_length()</code>	<code>int</code>	The height the widget wants so every tab is reachable without scrolling. See Known Limitations .
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard configuration, plus every constructor property above. <code>configure("propname")</code> returns a Pygubu-style query tuple.

Tab Pages

`add()` returns an `sCTkFrame`, and that is the page. Children go into it directly:

```
page = notebook.add("Audio")
sCTkFrame(page, border_width=1).pack(expand=True, fill="both", padx=10, pady=10)
```

Every page occupies the same grid cell, with the selected one raised. So a page keeps its children and its geometry between visits — switching tabs costs nothing and nothing is rebuilt.

All pages report `wininfo_ismapped()` as `true`, including the ones you cannot see. `tkraise()` changes stacking, not mapping, so that call is not a way to find out which tab is showing. Use `get()`.

Appearance

Which way the labels read

<code>text_orientation</code>	Result
"auto"	Reads outside-in — up a left strip, down a right one, the way a book's spine is set — and flips when side changes.
"up"	90° regardless of side.
"down"	270° regardless of side.
"horizontal"	Flat text.

"up" and "down" matter if a panel has notebooks on both edges and you want their labels to match rather than mirror.

"horizontal" needs a wider strip. With rotated text, a tab's length along the strip comes from its label's width and `tab_width` only has to fit the text's height. Flat text inverts that: the tab is barely taller than one line, and the label's width becomes `tab_width`'s problem. The widget cannot solve this for you without overriding the width you asked for, so it does not — set `tab_width` to something that fits your longest label, around 100 for ordinary words.

Orientation is a property of the notebook, not of a tab. Every tab in a strip reads the same way, which is deliberate: mixed orientations would need per-tab sizing along two axes and would look like a mistake.

Tab shape

`tab_style="rounded"` curves the two outer corners; `"angled"` cuts them off straight, the shape of a real notebook divider. The **inner** edge is square either way, so the selected tab reads as continuous with its page.

The chamfer is measured separately across the strip and along the tab, because the two edges are not equivalent — the strip's width is fixed, while a tab's length depends on its label and on whether the text is rotated:

```
sCTkNotebook.TAB_CHAMFER_X = 6      # in from the corner, across the strip
sCTkNotebook.TAB_CHAMFER_Y = 6      # in from the corner, along the tab
```

Equal values give a 45-degree cut. A small x with a larger y gives a shallower lean.

Why `tab_width` is explicit

The strip takes its width from the page, so a strip that measured itself would resize the content area whenever a tab was renamed. A layout that shifts under you is worse than one you set.

Pygubu Designer

Drop an `sCTkNotebook`, then drop `sCTkNotebook.Tab` onto it — the tab is offered only there, since it has nowhere else to live. Set each tab's `label` in the properties panel.

A tab is not a widget you construct: it is created *by* its notebook, through `add()`. The generated code reads:

```
tab1 = notebook1.add("Receiver")
```

Duplicate names are renamed, not rejected. `add()` raises `ValueError` on a name already in use, which is right for application code — but inside the Designer that exception surfaces only on the console where nobody sees it, and the tab silently fails to appear while the tree and the preview disagree. A numeric suffix is appended instead, visibly, in both the strip and the property field.

A tab cannot be selected by clicking it on the canvas. Select it in the widget tree, which also switches the canvas to that tab. This is the same limitation as `sCTkTabview` and for the same reason: the strip is canvas-drawn, so a click there has no widget for the Designer to resolve. A widget dropped *inside* a tab is selectable by clicking as normal.

Theming (sCTkThemes.json)

```
{
  "sCTkNotebook": {
    "fg_color": ["#FFFFFF", "#111827"],
    "font": ["Arial", 13, "normal"],
    "corner_radius": 6,
    "text_color": ["#1F2937", "#D1D5DB"],
    "selected_text_color": ["#FFFFFF", "#FFFFFF"],
    "tab_fg_color": ["#E2E8F0", "#1F2937"],
    "tab_selected_color": ["#1A4375", "#2471A3"],
    "tab_hover_color": ["#CBD5E1", "#374151"],
    "border_color": ["#1A4375", "#2471A3"],

    "border_width": 2,
    "page_corner": 8,
    "page_inset": 8,
    "tab_corner": 6,
    "tab_chamfer_x": 6,
    "tab_chamfer_y": 6,
    "tab_gap": 2,
    "tab_pad": 18,
    "separator_inset": 5,
    "strip_margin": 6,

    "disabled_map": {
      "text_color": ["#94A3B8", "#64748B"],
      "tab_fg_color": ["#F1F5F9", "#1A1D24"],
      "tab_selected_color": ["#CBD5E1", "#374151"],
      "selected_text_color": ["#94A3B8", "#64748B"],
      "border_color": ["#CBD5E1", "#374151"]
    }
  }
}
```

The colours are required, at the top level and in `disabled_map` — `fg_color`, `font`, `text_color`, `selected_text_color`, `tab_fg_color`, `tab_selected_color`, `tab_hover_color` and `border_color`. Construction raises `KeyError` naming the missing key and where it belongs.

The ten geometry values are optional. Absent from a block, the class attribute of the same name in upper case applies, so a theme need only mention what it wants to change — a squarer palette might set `tab_corner` and nothing else. They remain settable per instance for one-off tuning:

```
notebook.TAB_CHAMFER_Y = 10
```

`fg_color` is deliberately **not** in `disabled_map`: the page area does not dim, matching the dials and `sCTkScrollableFrame`. The content carries its own state, and a greyed page would hide it.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Example

```
#!/usr/bin/python3
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkLabelPrimary, sCTkNotebook

TABS = ["Receiver", "Transmitter", "Audio", "Filters", "System Logs"]

if __name__ == "__main__":
    root = sCTk()
    root.title("sCTkNotebook")
    root.geometry("640x420")

    base = sCTkFrame(root, fg_color="transparent", border_width=0)
    base.pack(expand=True, fill="both", padx=16, pady=16)

    notebook = sCTkNotebook(base, side="left", tab_width=34)
    notebook.pack(expand=True, fill="both")

    for name in TABS:
        page = notebook.add(name)
        sCTkLabelPrimary(page, text=f"This is the {name} tab").pack(
            expand=True, padx=20, pady=20)

    notebook.set("Audio")
    print("tabs:", notebook.tabs(), " selected:", notebook.get())

    root.mainloop()
```

Known Limitations

- **Tabs are selected from the widget tree, not the design canvas** — see [Pygubu Designer](#).
- **The strip scrolls when the tabs do not fit**, under the wheel or a trackpad, and only when there is something to scroll — otherwise the wheel belongs to whatever is on the page. There is no scrollbar and no indication that more tabs exist below. `required_length()` reports the height that would show them all, but the widget cannot insist: a parent packing it with `fill="both"` decides its size regardless. Use the figure to set a minimum on the containing window, or let it scroll.
- **Disabling does not cascade to children**. It dims the strip and the outline and locks tab selection; widgets on a page are unaffected, which is the caller's responsibility. Same as `sCTkTabview`.
- **The page area is a raw canvas underneath**. `"transparent"` cannot be rendered on one, so a transparent `fg_color` falls back to whatever is actually behind the widget — the same accommodation `sCTkFileExplorer` makes for its canvas.
- **`text_orientation` and `tab_style` are properties of the notebook, not of a tab**. Every tab in one strip shares them.

[Return to Table of Contents](#)

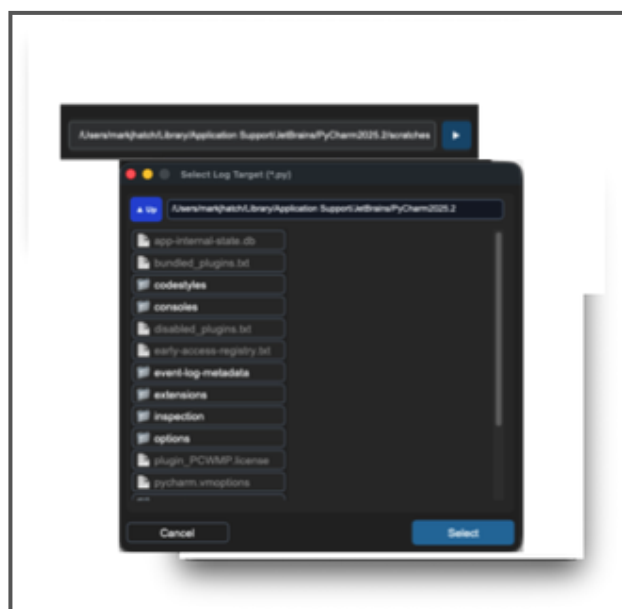
sCTkPathChooser

Table of Contents

- Overview
- Constructor
- Methods
- Typing a path
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkPathChooser is a theme-compliant single-line path entry paired with a "Browse..." button that opens an sCTkFileExplorer in a modal popup. It inherits `ctk.CTkFrame` directly, composing an internal `sCTkEntryPrimary` and `sCTkButtonPrimary`.



Every property this widget forwards to its internal entry (`justify`, `width`, `height`) has been confirmed valid against CustomTkinter's own real `CTkEntry` source, the same verification done for `sCTkSpinbox`. There's no risk of this widget sending an unrecognized property to its own entry.

Constructor

```
sCTkPathChooser(master=None, initialdir=None, initialfile=None, type="file",
                 filetypes=None, title=None, defaulttextextension=None,
                 justify="left", entry_height=None, browser_width=None,
                 browser_height=None, btn_text="Browse...", **kwargs)
```

Parameter	Type	Description
<code>master</code>	<code>widget</code>	Parent container.
<code>initialdir</code>	<code>str</code>	Starting directory for the browser popup, and where a bare <code>initialfile</code> is resolved.
<code>initialfile</code>	<code>str</code>	File to seed the entry with. A bare name resolves against <code>initialdir</code> ; an absolute path or one starting with <code>~</code> is used as given. It does not have to exist — see Typing a path .
<code>type</code>	<code>"file" / "directory"</code>	Whether individual files are selectable, or only directories.
<code>filetypes</code>	<code>list[str]</code>	File extension filter.
<code>justify</code>	<code>str</code>	Which end of a long path stays visible: <code>"right"</code> to see the filename, <code>"left"</code> to see the root. See Typing a path .
<code>btn_text</code>	<code>str</code>	The browse button's label.
<code>**kwargs</code>	—	Any native <code>CTkFrame</code> argument, or a theme-key override (see Theming).

```
save_path = sCTkPathChooser(control_panel, type="directory", initialdir="/Users/you/Documents")
save_path.pack(fill="x", padx=20, pady=10)
```

Methods

Method	Returns	Description
<code>get()</code>	<code>str</code>	Current path text.
<code>set(path)</code>	<code>None</code>	Sets the displayed path, normalizing and expanding it.
<code>state(mode=None) / get_state()</code>	<code>str</code>	Gets or sets <code>"normal"/"disabled"</code> , dimming both the entry and the browse button.

Method	Returns	Description
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard configuration, accepting state, type, title, justify, btn_text, entry_height, btn_width, btn_height, and — newly — <code>initialdir</code> , <code>initialfile</code> and <code>filetypes</code> as first-class properties. Those three had no <code>configure()</code> branch at all and were constructor-only, so setting any of them in the Designer, which applies every property through <code>configure()</code> , did nothing.
<code>configure(name)</code>	tuple	Pygubu-style single-argument query for any of the eight properties above. Previously broken: the implementation read <code>pname = args</code> rather than <code>args[0]</code> , so every comparison tested a tuple against a string and failed — all eight queries fell through to the native widget and Pygubu could read none of them. The dict form of <code>configure()</code> was dead for the same reason (<code>isinstance(args, dict)</code> on a tuple is never true).

Clicking "Browse..." opens an `sCTkFileExplorer` in a modal popup; selecting a path there calls `self.set(...)` on this widget automatically.

Typing a path

The entry is editable, and a path typed into it is applied on **Return**, on keypad Enter, or when focus leaves the field.

A value is accepted only if it exists. Focus-out fires whenever you click elsewhere, so a half-typed path must not replace a good one. `~` is expanded. A directory becomes the new `initialdir`, so browsing afterwards opens there; anything else is treated as a file selection.

The entry previously had **no bindings at all**, so a typed path was never applied — it stayed as loose text, nothing navigated, and pressing Browse then replaced it with the real value, which made the typing appear to vanish.

`sCTkFileExplorer` behaves identically. The two are the same control, one with a button.

justify decides which end of a long path you see. A Tk entry only honours the option when the text is *shorter* than the field; a longer path scrolls, and what shows is governed by the view position. `"right"` therefore moves the view to the end so the filename is visible.

That needed two fixes. The move was applied to the `CTkEntry` rather than the real `tk.Entry` it wraps, which does not reliably forward it. And it ran before the entry had recomputed the width of its new text, so a long path stopped part-way through the filename — it is now re-applied once Tk is idle.

Changing `justify` no longer calls `set()`, so it does not fire `command`. Changing alignment is not a path selection.

A nonexistent initialfile is still displayed. Naming a file that has yet to be created is legitimate — a "save as" field being the obvious case — and the browser opens on `initialdir` regardless. The visible consequence is that the entry shows a file while the browser behaves as though only the directory were set.

Theming (sCTkThemes.json)

```
{
  "sCTkPathChooser": {
    "entry_font": ["Arial", 13],
    "entry_fg": ["#F9F9FA", "#343638"],
    "entry_border_color": ["#979DA2", "#565B5E"],
    "entry_text_color": ["#000000", "#FFFFFF"],
    "btn_font": ["Arial", 13, "bold"],
    "btn_fg": ["#3B8ED0", "#1F6AA5"],
    "btn_hover": ["#2C74B3", "#144E75"],
    "btn_text_color": ["#DCE4EE", "#F9F9FA"],
    "btn_border_color": ["#3B8ED0", "#1F6AA5"],
    "disabled_map": {
      "entry_fg": ["#EAEAEA", "#2B2B2C"],
      "entry_border_color": ["#D3D3D3", "#3A3A3C"],
      "entry_text_color": ["#A0A0A0", "#7C7C7C"],
      "btn_fg": ["#D3D3D3", "#2D2F31"],
      "btn_border_color": ["#D3D3D3", "#2D2F31"],
      "btn_text_color": ["#A0A0A0", "#5A5C5E"]
    }
  }
}
```

Every key the code references is present in both the top-level block and `disabled_map` — confirmed by direct cross-check against the actual source, nothing missing.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Every top-level key is now required and validated at construction, matching `sCTkFileExplorer`/`sCTkTableview`/`sCTkSpinbox`/`sCTkSelector` — missing any raises immediately, naming the exact key. `disabled_map` entries deliberately keep their original, more lenient behavior: gracefully falling back to the top-level/normal value if not overridden, rather than hard-failing, since that's intentional and already correct.

Example

```
from scustomtkinter import sCTk, sCTkFrame, sCTkPathChooser, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x200")
    root.title("PathChooser Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    chooser = sCTkPathChooser(base, type="directory")
    chooser.pack(fill="x", pady=10)

    def toggle_disabled():
        target = "disabled" if chooser.get_state() == "normal" else "normal"
        chooser.state(target)
        toggle_btn.configure(text="Enable" if target == "disabled" else "Disable")

    toggle_btn = sCTkButtonPrimary(base, text="Disable", command=toggle_disabled)
    toggle_btn.pack(pady=10)

    root.mainloop()
```

Known Limitations

- **No readonly support** — unlike `sCTkSpinbox`, this widget only has `"normal" / "disabled"`, even though the same design opportunity applies (the entry could be readonly-locked while `"Browse..."` stays clickable, since it's the intended alternative way to set the value). Identified as a genuine future enhancement, not implemented yet.

[Return to Table of Contents](#)

sCTkScrollArea

`sCTkScrollArea` is a scrollable viewport container built on a raw `tkinter.Canvas`, offered as an alternative to `ctk.CTkScrollableFrame` for cases where you want to supply your own external scrollbar and control child event binding explicitly. It inherits `ctk.CTkFrame` and `ScrollBindingMixin`.

Its companion is `sCTkScrollbar`. Scroll handling comes from `ScrollBindingMixin`, which is the reference for how scrolling works across this library.

Table of Contents

- [Constructor](#)
- [Attributes](#)
- [Methods](#)
- [Wiring it up](#)
- [Theming](#)
- [Example](#)
- [Known Limitations](#)

Constructor

```
scroll_area = sCTkScrollArea(master=None, **kwargs)
```

Parameter	Type	Description
<code>master</code>	<code>widget</code>	Parent container.
<code>**kwargs</code>	—	Passed to <code>ctk.CTkFrame</code> . Note <code>fg_color="transparent"</code> and <code>border_width=0</code> are set internally and can't be overridden.

Attributes

Attribute	What it is
<code>scroll_content</code>	The frame your content goes into. A raw <code>tkinter.Frame</code> , not a themed widget.
<code>canvas</code>	The raw <code>tkinter.Canvas</code> providing the scrolling viewport.

Methods

Method	Description
<code>hook_scrollbar(scrollbar_widget)</code>	Connects a scrollbar to the canvas in both directions, and registers it as a scroll layer so the wheel keeps working while the pointer is over the bar itself.
<code>propagate_scroll_events(target_widget)</code>	Registers a widget outside <code>scroll_content</code> to receive scroll events, along with its descendants. Rarely needed now — see below.

Method	Description
<code>process_incoming_scroll(event)</code>	Compatibility shim. Scroll events are dispatched by the mixin directly; this remains only for external callers that bound this method themselves.

`propagate_scroll_events()` is no longer required for ordinary content. Anything placed inside `scroll_content` is bound automatically and re-bound whenever it changes, so the per-item call shown in older examples is redundant. It's still useful for widgets that sit *outside* that tree.

Its behavior also changed: registered widgets are now **remembered** and re-bound on every subsequent pass. The previous implementation bound once and forgot, so any later rebind lost them.

Wiring it up

```
scroll_view = sCTkScrollArea(container)
scroll_view.pack(fill="both", expand=True)

scrollbar = sCTkScrollbar(container, orientation="vertical")
scrollbar.pack(side="right", fill="y")

scroll_view.hook_scrollbar(scrollbar)

for i in range(25):
    sCTkLabelSecondary(scroll_view.scroll_content, text=f"Row {i}").pack(anchor="w")
```

No `propagate_scroll_events()` call is needed — the rows are inside `scroll_content`, so the content rebind picks them up.

Theming

This widget is not part of the theme system. It doesn't inherit `ThemeableWidget`, doesn't read `sCTkThemes.json`, and has no theme block. An earlier version of this page showed an `sCTkScrollArea` JSON block — nothing reads it, and adding it has no effect.

The canvas and content-frame backgrounds are hardcoded: `#FAFAFA` in light mode, `#1A1A1A` in dark, switched by the widget's own `_set_appearance_mode()` hook. `scroll_content` is a raw `tkinter.Frame`, which cannot render CustomTkinter's transparent pseudo-value or `(light, dark)` tuples, so it needs a literal color.

Bringing this widget into the theme system is an open item, tied to the Pygubu Designer integration work.

Example

```
#!/usr/bin/python3
import customtkinter as ctk
from scustomtkinter import (sCTk, sCTkFrame, sCTkButtonPrimary, sCTkLabelSecondary,
                           sCTkScrollbar, sCTkScrollArea)

if __name__ == "__main__":
    root = sCTk()
    root.geometry("480x480")
    root.title("sCTkScrollArea Validation Bench")
    root.configure(fg_color=("#F1F5F9", "#1C1C1C"))

    lower_tray = sCTkFrame(root, fg_color="transparent")
    lower_tray.pack(side="bottom", fill="x", padx=15, pady=(0, 15))

    main_layout = sCTkFrame(root, border_width=2)
    main_layout.pack(expand=True, fill="both", padx=15, pady=15)

    status_monitor = sCTkLabelSecondary(main_layout, text="STATUS: viewport online")
    status_monitor.pack(fill="x", padx=10, pady=(5, 10))

    def toggle_appearance_skin():
        ctk.set_appearance_mode("Light" if ctk.get_appearance_mode() == "Dark" else "Dark")

    btn_theme = sCTkButtonPrimary(lower_tray, text="Toggle Light/Dark", command=toggle_appearance_skin
)
    btn_theme.pack(fill="x", expand=True, padx=5)

    scrollbar = sCTkScrollbar(main_layout, orientation="vertical")
    scrollbar.pack(side="right", fill="y", padx=(5, 10), pady=10)

    content_chassis = sCTkFrame(main_layout, border_width=0, fg_color="transparent")
    content_chassis.pack(side="left", fill="both", expand=True, padx=(10, 0), pady=10)

    scroll_view = sCTkScrollArea(content_chassis)
    scroll_view.pack(fill="both", expand=True)

    for i in range(25):
        sCTkLabelSecondary(
            scroll_view.scroll_content,
            text=f"Transceiver channel {100 + i} [OK]"
        ).pack(anchor="w", padx=10, pady=4)

    scroll_view.hook_scrollbar(scrollbar)

    root.mainloop()
```

Known Limitations

- **Outside the theme system entirely** — see [Theming](#). It's the only widget in this library in that position.
- **No `_finalize_themeable_lifecycle()` handshake**, so Pygubu Designer gets no registration signal from it.
- **No disabled state**. Unlike `sCTkScrollableFrame` and `sCTkFileExplorer`, there's no `state()` here and no way to make it inert.
- **`scroll_content` is a raw `tkinter.Frame`**, so widgets placed in it don't inherit CustomTkinter background propagation. Themed `sCTk` children render correctly; plain `tk` children may need their `bg` set to match.
- **The debounced rebind also runs on genuine resizes** — see the [mixin page](#).

Behavior changed when this widget adopted the shared mixin. Three corrections, all bringing it in line with the rest of the library: Windows wheel travel halved (it previously doubled the `/120` delta), the two's-complement boundary at exactly 32768 was fixed, and the packed touchpad delta is now decoded by bit-shifting rather than reading `event.delta_y`. It also gained the nested-frame boundary guard and the automatic content rebind.

[Return to Table of Contents](#)

ScrollBindingMixin

The single shared implementation of cross-platform mouse wheel and macOS trackpad scroll handling for this library. Used by `sCtkScrollableFrame` (and therefore `sCtkTableview`), `sCtkFileExplorer`, and `sCtkScrollArea`.

This page is the reference for how scrolling works. The individual widget pages describe only their own hooks and link here.

Table of Contents

- [Why it exists](#)
 - [Platform behavior](#)
 - [Tuning constants](#)
 - [Activation and rebinding](#)
 - [Disabling scroll](#)
 - [Nested scroll regions](#)
 - [Host contract](#)
-

Why it exists

This logic previously existed as three independent copies, each adapted by hand from the first. They drifted, as duplicated code does:

- **The two's-complement sign correction disagreed.** `sCtkScrollableFrame` used `>= 0x8000`; `sCtkScrollArea` used `> 32768`. Those differ at exactly 32768 — the smallest *negative* value in a signed 16-bit field — which one read as `-32768` and the other as `+32768`, inverting direction at that value.
- **`sCtkScrollArea` decoded the packed touchpad delta differently again**, reading `event.delta_y` when present and applying a 16-bit correction to what may be a 32-bit packed value, rather than bit-shifting out the signed components.
- **Windows wheel scaling disagreed:** `/120` unscaled in `sCtkScrollableFrame`, `/120 * 2` in `sCtkScrollArea` — twice the travel per notch.
- **`sCtkFileExplorer` had no touchpad accumulator at all**, scrolling on every raw event instead of gating on an accumulated threshold. Trackpad scrolling there was markedly faster and coarser than everywhere else.
- **`sCtkFileExplorer` walked only one level** into its row frame, so a row's label or icon was never bound and the wheel did nothing over them.
- **The nested-scrollable boundary guard existed in exactly one of the three.**

Every fix had to be made three times, and none of them were. Where the copies disagreed, `sCtkScrollableFrame`'s version — the maintainer-verified reference, confirmed smooth in live testing on macOS with both an Apple mouse and a trackpad — is the one that won.

Platform behavior

Three genuinely different platform models are handled:

Platform	Mechanism
Windows	<code><MouseWheel></code> with a <code>/120</code> -scaled delta
Linux	Discrete <code><Button-4>/<Button-5></code> events — no continuous delta exists

Platform	Mechanism
macOS	Its own <MouseWheel> scaling, plus a separate higher-precision <TouchpadScroll> synthetic event

macOS <TouchpadScroll> packs a two-axis delta into a single 32-bit integer — X in the high 16 bits, Y in the low 16 — each an *unsigned* field that must be converted back to signed, or every upward scroll reads as a large positive number.

Trackpad events arrive far more frequently and with far finer deltas than wheel notches, so scrolling on each one is unusably fast. Deltas accumulate and move the view only once a threshold is crossed. The accumulator resets on a direction reversal, so reversing responds immediately instead of first cancelling out what had built up.

Tuning constants

Class attributes on the mixin, so they can be overridden per subclass or per instance without any additional machinery. **These are macOS-tuned**; other platforms may want different values.

Constant	Default	Meaning
MAC_SCROLL_SENSITIVITY	3	Amplification for macOS wheel deltas, which are much smaller than Windows' /120 steps
MAC_SCROLL_MAX_STEP	5	Ceiling on units travelled per macOS wheel event
TOUCHPAD_ACCUMULATION_THRESHOLD	12.0	Accumulated trackpad delta required before the view moves

MAC_SCROLL_MAX_STEP exists because macOS reports wildly different delta magnitudes depending on hardware. An Apple Magic Mouse sends fine-grained values near 1; a conventional wheel mouse sends a large value per detent — around 38 in live testing. Multiplying that by the sensitivity gave 114 units from a single wheel click, jumping a 100-row list end to end. The amplification is still correct for fine-grained hardware, so rather than dropping it, the result is clamped: small deltas scale normally, large ones saturate.

Resulting travel per event:

event.delta	Hardware	Units
0.4	Magic Mouse	3
1	Magic Mouse	3
2	Magic Mouse	5 (clamped)
38	Wheel detent	5 (clamped)

Setting `MAC_SCROLL_MAX_STEP` to 3 gives the conventional three-lines-per-notch that matches macOS defaults. Values below 3 slow the Magic Mouse too, since its fine deltas already scale to 3 before the clamp applies.

Whether these should move somewhere more discoverable than class attributes is an open question.

Activation and rebinding

Bindings are automatic and self-maintaining. No activation call is needed, and content added after a widget is placed is picked up on its own.

That reliability takes four mechanisms, each covering a gap the others don't. All are idempotent — bindings are always torn down before being rebuilt — so overlapping coverage costs nothing.

Mechanism	Covers
<Map> on the host	Later remaps, e.g. <code>pack_forget()</code> then re-placement
<Map> on <code>extra_map_widget</code>	The widget the geometry manager actually sees
<code>after_idle()</code> at construction	Initial activation, independent of mapping semantics
<Configure> on <code>content_widget</code> , debounced	Content added <i>after</i> activation

Why <Map> alone isn't enough. `CTkScrollableFrame` is not the widget that gets placed: it builds an internal `_parent_frame` plus a canvas, inserts *itself* into that canvas via `create_window()`, and overrides `pack()` / `grid()` / `place()` to operate on `_parent_frame`. The widget is therefore a canvas-window child and may never receive <Map> the way an ordinarily-managed widget does. `after_idle()` is what actually establishes bindings in practice.

Why the content rebind is needed. Activation happens once, at a moment when the container is usually still empty — callers construct, place, and *then* populate. Confirmed by live testing: an `sCtkTableview` bound at activation time collected 16 layers (frame, canvas, header cells) because `load_dataset()` hadn't run yet; the 32 data cells created afterwards were never bound, so it scrolled beside its rows but not over them.

<Configure> fires when children change the container's layout, so it catches every content-adding path. It's debounced through `after_idle` because building a table fires it once per cell — one rebind instead of 32, run after the burst rather than during it, so it sees the finished tree.

<Configure> can't distinguish "children were added" from "the window was dragged", so **resizing rebinds too**. Coalesced, but not free on a very large content tree.

Do not replace `tk.Misc.bind(self, ...)` with `self.bind(...)`. `CustomTkinter` overrides `CTkScrollableFrame.bind()` to forward every binding to `self._parent_canvas` instead of attaching it to the widget. An earlier version used `self.bind()`, and bindings never landed on the frame — scroll handling was silently never installed, and widgets only appeared to scroll where native `CustomTkinter`'s own global `bind_all` handler happened to cover for it. `tk.Misc.bind` called unbound reaches the real Tkinter implementation. This looks like a needless complication and is not.

Disabling scroll

When a host's `_scroll_permitted()` returns `False`, the mixin doesn't merely unbind — it installs **blocking** handlers. Two separate mechanisms are involved, because neither alone is sufficient.

Wheel and trackpad events. Unbinding is not enough: native `CTkScrollableFrame.__init__` installs its own application-global `bind_all("<MouseWheel>")` handler that survives any `unbind()`. Calling `unbind_all()` would disable scrolling for *every other* scrollable widget in the application. Instead a handler returning `"break"` is installed on each layer. Tk dispatches bindings by bindtag in order — widget, class, toplevel, then `all` — and `bind_all` lands on that final tag, so a widget-level `"break"` halts the chain before the global handler is reached. Confirmed by live testing: with two independent scrollable frames side by side, disabling one left the other scrolling normally with both a mouse wheel and a trackpad.

Scrollbar dragging. `unbind()` is actively dangerous here — Tk's `unbind()` removes every binding for an event on a widget, so calling it on a scrollbar's `<Button-1>` would destroy CustomTkinter's own drag handler permanently, with no way to restore it. Binding a blocker with `add="+"` doesn't work either: handlers fire in the order added, and CustomTkinter's was added during its own construction, so `"break"` at that point is too late. A private, per-instance `bindtag` is inserted at the **front** of the widget's tag list instead, so the blocker runs before CustomTkinter's bindings. Re-enabling removes the tag; CustomTkinter's bindings are never modified.

The tag name embeds `id(self)`, so disabling one host has no effect on any other in the same application.

The scrollbar stays visible when blocked, just inert. CustomTkinter's scrollbar has no native disabled state to lock, so there's no greyed-out appearance to switch to.

Nested scroll regions

The descendant walk stops at any nested `CTkScrollableFrame` boundary — covering `sCTkScrollableFrame` and anything built on it, such as `sCTkSelector` and `sCTkTableview`. Without this, an inner scrollable frame placed inside an outer one would have its canvas, scrollbar, and entire content tree bound to the *outer* host's handler as well as its own, and since bindings use `add="+"`, both fire on the same event and scroll both at once. Native CustomTkinter guards the same boundary in its own `_check_if_valid_scroll`.

The guard applies to descendants only, so a scrollable host still binds its own layers.

Not yet live-tested. The logic mirrors CustomTkinter's own guard and is straightforward, but an actual nested case hasn't been exercised against it.

Widgets that consume scroll themselves

A widget doesn't have to be a scrollable frame to have its own claim on the wheel. Setting `_CONSUMES_SCROLL = True` on a class ends the descendant walk there, exactly as the boundary check above does:

```
class sCTKdialBase(ctk.CTkFrame, ThemeableWidget):
    _CONSUMES_SCROLL = True
```

The dial family declares it. Without it, a dial placed inside an `sCTkScrollableFrame` had its canvas bound to the *frame's* handler as well as its own — and since the frame's binding is installed with `add="+"` and fires first, pointing at a VFO knob and turning the wheel scrolled the list instead of the knob. Confirmed at runtime, not only in the Designer.

The trade is deliberate: a dial becomes a dead zone for scrolling the container behind it. Pointing at a control and turning the wheel should operate that control, the way it does on a physical panel — and the alternative left the dial with no wheel gesture at all while it sat in a scrolling container.

A separate scrolling region built directly on a plain `Canvas` is still **not** guarded automatically — the `isinstance` check keys on `CTkScrollableFrame` specifically — but such a widget can now opt out for itself with `_CONSUMES_SCROLL`.

Host contract

A host class must implement two methods and may override two more:

Method	Required	Returns
<code>_scroll_target()</code>	yes	The widget to call <code>yview_scroll()</code> on, or <code>None</code> if scrolling isn't currently possible
<code>_scroll_layers()</code>	yes	The ordered, deduplicated list of widgets to bind
<code>_scroll_permitted()</code>	no	<code>False</code> to install blocking handlers instead of scroll handlers. Default <code>True</code>
<code>_scroll_drag_targets()</code>	no	Widgets whose click-drag should also be blocked when not permitted. Default <code>none</code>

A host may also set the class attribute `_CONSUMES_SCROLL = True` to stop an enclosing scroll region binding its descendants — see [Nested scroll regions](#).

Hosts call two setup methods from `__init__`:

```

self._init_scroll_state()                # must run before any binding
self._install_scroll_activation(
    extra_map_widget=...,                # optional
    content_widget=...,                  # optional
)

```

content_widget matters. It defaults to `self`, which is correct only when content is added directly to the host. A host that puts content in a separate inner frame **must** pass that frame — adding rows to an inner frame doesn't resize the outer widget, so `<Configure>` on `self` would never fire and the content rebind would silently never happen.

Hosts may also define `_USE_CUSTOM_SCROLL_BINDING = False` as a kill switch, falling back to whatever native CustomTkinter provides. It's checked inside `_toggle_scroll_bindings()` rather than at the call sites, so it can't be bypassed by reaching that method through a different entry point — the exact bug that made the toggle ineffective in an earlier revision.

Host implementations

Host	<code>_scroll_target()</code>	Notes
<code>sTkScrollableFrame</code>	Parent canvas via <code>winfo_parent()</code>	Wrapped by a native <code>CTkScrollableFrame</code> that owns the canvas. Passes <code>_parent_frame</code> as <code>extra_map_widget</code> . Disabling stops scrolling.
<code>sTkFileExplorer</code>	<code>self.canvas</code>	Builds its own canvas, so no lookup needed. Passes <code>explorer_frame</code> as <code>content_widget</code> . Disabling stops scrolling.
<code>sTkScrollArea</code>	<code>self.canvas</code>	Builds its own canvas. Passes <code>scroll_content</code> as <code>content_widget</code> . No disabled state.

[Return to Table of Contents](#)

sCTkSelector

(Derived from Selector class by Fastattack, 2024. This widget was made available to the community via the MIT License. Source Repository: [MoreCustomTkinterWidgets](#))

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkSelector is a theme-compliant, scrollable multi-select (or single-select) list of checkboxes, with an optional live-filtering search field. It's built by composing a themed frame, an sCTkScrollableFrame for the checkbox list, and one sCTkCheckBox per item — not by subclassing a single native CustomTkinter widget.



This widget inherits sCTkFrame directly (rather than raw ctk.CTkFrame) — a composition pattern that previously carried a real risk of ThemeableWidget.__init__ running twice per instance and silently corrupting this widget's own resolved theme data. That risk is now fully closed: ThemeableWidget has a run-once guard preventing the double-init, and sCTkFrame itself filters its inbound kwargs down to only what native CTkFrame actually accepts before its own constructor call. Neither fix required any change to this widget.

Constructor

```
sCTkSelector(master, items=None, multiple_choices=True, searchBox=True, **kwargs)
```

Parameter	Type	Default	Description
master	widget	—	Parent container.

Parameter	Type	Default	Description
items	list[str] or str	None	Initial list of checkbox labels. Accepts ["A", "B"], a bare comma-separated string, or a real list — see List Properties . Must not contain duplicates — raises ValueError if it does.
multiple_choices	bool	True	If False, selecting one item automatically deselects any other currently-selected item.
searchBox	bool	True	Whether the live-filtering search field is shown above the checkbox list.
**kwargs	—	—	Any native CTkFrame argument, or an override for one of the theme keys listed under Theming .

```
channel_selector = sCTkSelector(control_panel, items=["Ch 1", "Ch 2", "Ch 3"], multiple_choices=False)
channel_selector.pack(expand=True, fill="both", padx=20, pady=20)
```

Methods

Method	Returns	Description
get_all_items()	list[str]	Every checkbox's label text, regardless of current search filter or selection state.
state(mode=None)	str	Gets or sets the widget's visual state. "disabled" locks and dims every checkbox and the search field (routed to the search field's own "readonly", not "disabled" — see Known Limitations); anything in ("normal", "enabled", "active") re-enables both.
get_state()	str	Equivalent to calling state() with no argument.
configure(**kwargs) / config(**kwargs)	varies	Standard widget configuration, plus items, multiple_choices, searchBox, pack_propagate, grid_propagate, and state are all handled as first-class properties, matching the constructor. Calling configure("propname") with a single property name returns a Tkinter-style query tuple for state, multiple_choices, searchBox, items, pack_propagate, grid_propagate, and fg_color/border_color/text_color.

Theming (sCTkThemes.json)

- **Applied once, at construction** — `fg_color` and `corner_radius` control the outer frame; the checkbox-related keys and `border_color` are validated and applied once at construction time (not repeatedly on every state change).

- **Re-applied on every `state()` change** — the checkbox colors below are recomputed from normal values or `disabled_map` every time you call `state()`.

```
{
  "sCTkSelector": {
    "fg_color": "transparent",
    "corner_radius": 6,
    "text_color": ["#1F2937", "#F9FAFB"],
    "checkbox_fg_color": ["#1A4375", "#1F6AA5"],
    "checkbox_hover_color": ["#112A4B", "#1A5885"],
    "border_color": ["#94A3B8", "#4B5563"],
    "checkmark_color": ["#FFFFFF", "#FFFFFF"],
    "disabled_map": {
      "text_color": ["#808080", "#666666"],
      "checkbox_fg_color": ["#CBD5E1", "#475569"],
      "border_color": ["#CBD5E1", "#334155"],
      "checkmark_color": ["#F1F5F9", "#94A3B8"]
    }
  }
}
```

checkbox_fg_color / checkbox_hover_color are dedicated keys, not reused from fg_color. An earlier version derived each checkbox's accent color from this widget's own `fg_color` — the same key that controls the outer frame's background — falling back to a hardcoded generic blue whenever `fg_color` was "transparent" (a common, legitimate choice for a frame, not a theme gap). Reusing one key for two different visual purposes didn't work well; dedicated keys fix that cleanly. All five top-level keys (`text_color`, `checkbox_fg_color`, `checkbox_hover_color`, `border_color`, `checkmark_color`) and four `disabled_map` keys (all but `checkbox_hover_color` — disabled checkboxes reuse `checkbox_fg_color` for hover too, since hover can't meaningfully trigger while disabled) are required; missing any raises immediately at construction.

border_color is also shared with this widget's two internal sub-widgets (the search field and the checkbox-list frame), passed in once at construction so their *normal*-state border visually matches this widget's own border — confirmed by direct testing that these two sub-widgets' own independent default themes can otherwise visibly mismatch, especially in dark mode. This only establishes the shared normal-state value; each sub-widget's own state-driven color changes (the search field's readonly/disabled coloring in particular) are left completely untouched afterward.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

Every color is passed through as a raw (`light`, `dark`) tuple, letting CustomTkinter's native appearance-mode tracking handle repaints automatically, consistent with the approach used throughout this project.

Example

```
from scustomtkinter import sCTk, sCTkFrame, sCTkSelector, sCTkButtonPrimary, sCTkLabelPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x420")
    root.title("Selector Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    selector = sCTkSelector(base, items=[f"Item {i}" for i in range(1, 21)])
    selector.pack(expand=True, fill="both", pady=10)

    status = sCTkLabelPrimary(base, text=f"state: {selector.get_state()}")
    status.pack(pady=5)

    def toggle_disabled():
        target = "disabled" if selector.get_state() == "normal" else "normal"
        selector.state(target)
        status.configure(text=f"state: {selector.get_state()}")
        toggle_btn.configure(text="Enable" if target == "disabled" else "Disable")

    toggle_btn = sCTkButtonPrimary(base, text="Disable", command=toggle_disabled)
    toggle_btn.pack(pady=10)

    root.mainloop()
```

Pygubu Designer

grid_propagate has been removed. `pack_propagate()` and `grid_propagate()` control whether a container resizes to fit its **children**, and which applies depends on how those children are managed — not on how this widget is managed by its own parent. This widget packs its children, so `pack_propagate` is the meaningful call and `grid_propagate` could never have an effect. It was offered anyway, which made it look like a knob that did nothing while `pack_propagate` appeared to work "regardless of geometry management" — which is simply what it does.

Passing it still works and is ignored, so existing code does not raise.

Clearing items restores ["Item 1", "Item 2"] rather than emptying the list. The default slot previously reported `[]`, so blanking the field deleted every checkbox — the widget did exactly what it was told, but the answer was wrong.

Known Limitations

- **Disabling this widget routes the search field to "readonly", not "disabled"** — deliberate, so its text remains selectable/copyable, but worth knowing if you expected a uniform "disabled" state across every sub-component.
- **Fixed:** single-argument queries used to return `str(value)` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair, so the Designer reads a usable value.
- **.config() previously bypassed this widget entirely.** Tkinter binds `.config` to `.configure` as a separate class attribute rather than tracking a subclass's override, and this class had no `config = configure` line — so `.config(...)` skipped the `items/searchBox/multiple_choices/state` handling and landed on `sCTkFrame`'s `configure()` instead. Fixed. Note this widget uses the older `(self, cnf=None, **kwargs)` signature rather than `*args`; that's correct here and is *not* the shape that caused the tuple-comparison bugs found elsewhere in the library, since `cnf` is a real parameter holding the value itself.

- `items` must not contain duplicate labels — `configure(items=[...])` raises `ValueError` if it does, since selection tracking is index-based and duplicate labels would make search filtering ambiguous.

[Return to Table of Contents](#)

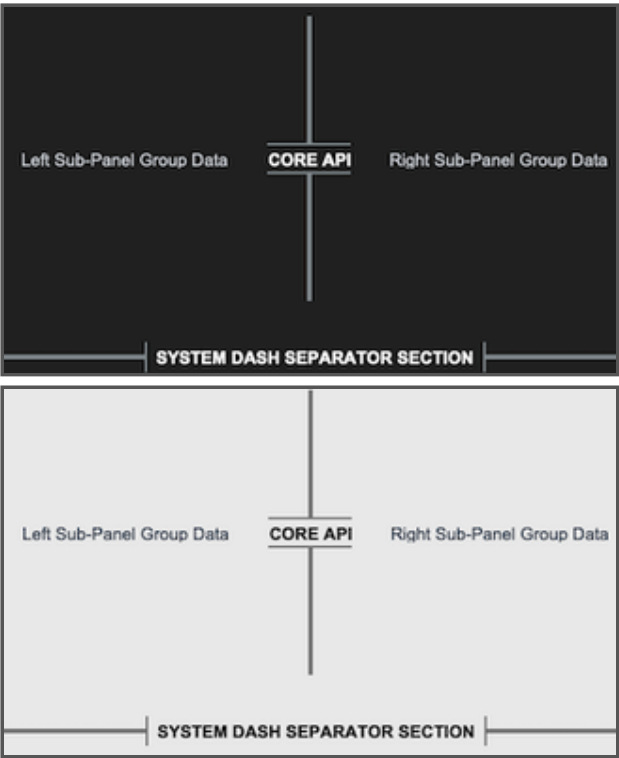
sCTkSeparator

(Derived from Separator class by Fastattack, 2024. This widget was made available to the community via the MIT License. Source Repository: [MoreCustomTkinterWidgets](#))

Table of Contents

- [Overview](#)
- [Constructor](#)
- [Changing properties at runtime](#)
- [State](#)
- [Theming](#)
- [Layout Manager Integration](#)
- [Pygubu Designer](#)
- [Event Binding](#)
- [Example](#)

The *sCTkSeparator* is an advanced, themeable divider widget for CustomTkinter. It provides dynamic scaling via layout managers, vector-drawn customizable corner radiuses, dashed/dotted line styles, and automated line-splitting centered section text headers with bounding capsule brackets.



Overview

The component functions as a structural vector drawing lane subclassed from `ctk.CTkBaseClass`. Rather than forcing a static line width or texture file, it wraps a native Tkinter canvas object to paint partitions programmatically.

The visual update matrix implements two important enhancements:

1. **Dynamic Layout Dimension Adapters:** To prevent text characters from clipping, the instantiation block monitors initialization properties. If text section banners are provided, the widget automatically stretches its bounding frame vertical or horizontal thickness out to 28px to give text canvas regions clear physical space while leaving the split line itself perfectly thin.
2. **Skin Preference Broadcaster Interceptor:** Features an explicit `_set_appearance_mode` connection loop. This forces text header strings, dashed patterns, and line fills to recalculate active palettes instantly during global theme swaps without any pixel lag.

Constructor

Property Name	Data Type	Default Value	Description
master	any	<i>Required</i>	Parent container instance (e.g., sCTkFrame or ctk.CTk).
length	int	100	The total span length of the line track in pixels (corresponds to widget height if vertical, width if horizontal).
width	float	4	The visual thickness profile of the divider line in pixels.
corner_radius	int or None	6 (from theme)	Defines roundness sharpness of divider line endpoints (defaults to stylesheet configuration).
orientation	str	"vertical"	Sets spatial directional positioning alignment. Accepts "vertical" or "horizontal".
text	str	""	Appends a centered section header label text directly inside a computed line split zone.
font	tuple or CtkFont	("Arial", 11, "bold")	Text font profile style parameters for the embedded header tag.
text_color	str or Tuple[str, str]	Central theme default	Font hex palette token string mapping. Supports appearance mode tuples.
dash	tuple or None	None	Integer stroke sequence array tuple mapping out dashed/dotted rendering rules (e.g., (5, 5)).

Changing properties at runtime

`orientation`, `length`, `text`, `dash` and `state` can all be changed after construction through `configure()`.

orientation swaps the widget's width and height. A vertical separator is built with `height = length`, a horizontal one with `width = length`, so flipping the orientation without swapping the dimensions would leave a horizontal separator tall and thin. `configure()` handles that.

`length` is a construction-time alias for whichever dimension the current orientation makes the long one, and is translated the same way.

Both previously fell straight through to native `CTkBaseClass.configure()`, which raised `ValueError: ['orientation'] are not supported arguments` — so they worked at construction and failed the moment you tried to change them, which is exactly what Pygubu Designer does when you edit the dropdown.

State

Method	Description
<code>state(mode=None)</code>	Getter with no argument; setter with "normal" or "disabled".
<code>get_state()</code>	Equivalent to <code>state()</code> with no argument.
<code>configure(state=...)</code>	Same effect. Both routes are supported.
<code>cget("state")</code>	Reads the current state.
<code>configure("state")</code>	Pygubu-style single-argument query.

A separator has nothing to interact with, so disabling only repaints it from `disabled_map` — the line and any header text dim together. It exists so a panel can disable every widget it contains uniformly.

Theming (sCTkThemes.json)

The component queries your centralized theme sheet profile matrix using standard `self._resolve_color()` lookup calls, ensuring that indicator dots and canvas borders translate colors smoothly across appearance updates.

To satisfy the framework configuration guidelines, ensure your theme matrix includes this structured asset block:

```
{
  "sCTkSeparator": {
    "fg_color": ["#808080", "#8A9296"],
    "bg_color": "transparent",
    "corner_radius": 6,
    "font": ["Arial", 11, "bold"],
    "text_color": ["#1A1A1A", "FFFFFF"],
    "disabled_map": {
      "fg_color": ["#CBD5E1", "#475569"],
      "text_color": ["#94A3B8", "#64748B"]
    }
  }
}
```

Every key above is required, including `disabled_map`. Construction raises `KeyError` naming the missing key and whether it belongs at the top level or in `disabled_map`.

This matters because the disabled colours were previously unreachable. `_draw()` read them with hardcoded fallbacks — `.get("fg_color", ["#CBD5E1", "#475569"])` and `.get("text_color", ["#94A3B8", "gray50"])` — and since the theme block had no `disabled_map` at all, those fallbacks were **always** taken. A disabled separator

never used the configured theme. The values shown above are those same fallbacks promoted into the theme file, so the appearance is unchanged; the one exception is dark-mode disabled text, where the Tk colour name `gray50` is replaced by `#64748B`, matching the disabled text colour used across the rest of the library.

`font` and `corner_radius` lost their fallbacks for the same reason. `text_color` additionally used to fall back to `ctk.ThemeManager.theme["CTkLabel"]["text_color"]` — borrowing another widget class's colour, which would now only mask a theme gap.

Structural parameters are not required in the theme. `orientation`, `length`, `width`, `text` and `dash` are read from the resolved keywords, so the theme *can* supply them, but they are constructor arguments with sensible defaults and requiring them would push layout decisions into the stylesheet.

Layout Manager Integration

Mixing layout manager tracking loops within the same immediate frame layer is completely blocked. When handling automated expansion parameters across scaling monitor resolutions, enforce the following geometry behaviors:

Grid Configurations (`.grid()`)

- **Horizontal Mode Line:** Must use `sticky="ew"` to allow the vector path to grow horizontally.
- **Vertical Mode Line:** Must use `sticky="nswe"` to stretch across columns and rows evenly without crushing string lines.
- **Parent Frame Setup:** The container frame track columns/rows **must** have their weights configured to let the engine allocate expanding window real estate:

```
# Column 0 and Column 2 hold widgets and expand; Column 1 isolates the separator line track
grid_Frame.grid_columnconfigure(0, weight=1)
grid_Frame.grid_columnconfigure(1, weight=1)
grid_Frame.grid_columnconfigure(2, weight=1)
```

Pack Configurations (`.pack()`)

- **Horizontal Mode Line:** Must use `fill="x"` alongside `expand=False` so it hugs adjacent frames tightly instead of expanding into empty background rows.
- **Vertical Mode Line:** Must use `fill="y"` inside layout columns.

Pygubu Designer

When configuring layouts visually within the Pygubu Designer editing workspace panel strip, observe these property formatting rules:

1. **orientation**: Select `vertical` or `horizontal` from the choice dropdown list pane. The preview canvas will immediately adjust orientations without flattening.
2. **text**: Type any section title banner sequence string directly into the entry field (e.g., `AUDIO CONTROLS`). The line will cleanly break around the text boundaries.
3. **dash**: Enter raw comma-separated lists of numerical values directly into the input strip **without using quote symbols or brackets**.
 - Type `5,5` for standard clean dash blocks.
 - Type `2,6` for clean dotted layout maps.
 - Leave blank or type `None` to restore solid rounded vector shapes.

Event Binding

`bind()` and `unbind()` are overridden to route to the internal canvas, which is what actually receives events — `CTkBaseClass` filters direct binds on the widget itself.

Both previously discarded an argument, and both were fixed:

- **`bind()` accepted `add` and ignored it**, hardcoding `add=True` in the forwarded call. A caller passing `add=False` to *replace* an existing binding would silently accumulate one instead.
- **`unbind()` accepted `funcid` and discarded it**, so it removed *every* binding for that sequence rather than the one identified. This is the same destructive behaviour that made `unbind()` unusable for blocking scrollbar drags in `sCTkScrollableFrame`: Tk's `unbind()` with no `funcid` wipes bindings this widget never installed, with no way to restore them.

If existing code depended on the old behaviour it will change — though neither method did what its signature promised.

Example

Below is a complete, self-contained test execution script demonstrating how to layout horizontal, vertical, and dashed separators inside an interactive telemetry deck panel while exercising lock states and skin sweeps.

```

#!/usr/bin/python3
# =====
# TESTING HARNESS IMPORTS & SETUP for Separator
# =====

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTkLabelSecondary, sCTk, sCTkSeparator

if __name__ == "__main__":
    root = sCTk()
    root.title("sCTkSeparator Production Test Environment")
    root.geometry("600x450")

    grid_Frame = sCTkFrame(root)
    grid_Frame.pack(side="top", fill="both", expand=True, padx=20, pady=15)
    grid_Frame.grid_columnconfigure(0, weight=1); grid_Frame.grid_columnconfigure(1, weight=1);
    grid_Frame.grid_columnconfigure(2, weight=1); grid_Frame.grid_rowconfigure(0, weight=1)

    lbl_left = sCTkLabelSecondary(grid_Frame, text="Left Sub-Panel Group Data")
    lbl_left.grid(row=0, column=0, sticky="nswe")

    sep_vertical_text = sCTkSeparator(grid_Frame, orientation="vertical", text="CORE API", width=4)
    sep_vertical_text.grid(row=0, column=1, sticky="nswe", padx=10, pady=10)

    lbl_right = sCTkLabelSecondary(grid_Frame, text="Right Sub-Panel Group Data")
    lbl_right.grid(row=0, column=2, sticky="nswe")

    sep_horizontal_text = sCTkSeparator(root, orientation="horizontal", text="SYSTEM DASH SEPARATOR
SECTION", width=4)
    sep_horizontal_text.pack(side="top", fill="x", padx=20, pady=10)

    def toggle_separator_lock():
        target = "disabled" if sep_vertical_text.get_state() == "normal" else "normal"
        sep_vertical_text.configure(state=target)
        sep_horizontal_text.configure(state=target)
        btn_lock.configure(text="Lock Separators" if target == "normal" else "Unlock Separators")

    def toggle_skin_mode():
        current_skin = ctk.get_appearance_mode()
        ctk.set_appearance_mode("Light" if current_skin == "Dark" else "Dark")

    btn_lock = sCTkButtonPrimary(root, text="Lock Separators", command=toggle_separator_lock)
    btn_lock.pack(pady=5)
    btn_theme = sCTkButtonPrimary(root, text="Simulate Global Theme Shift", command=toggle_skin_mode)
    btn_theme.pack(pady=(5, 20))

    root.mainloop()

```

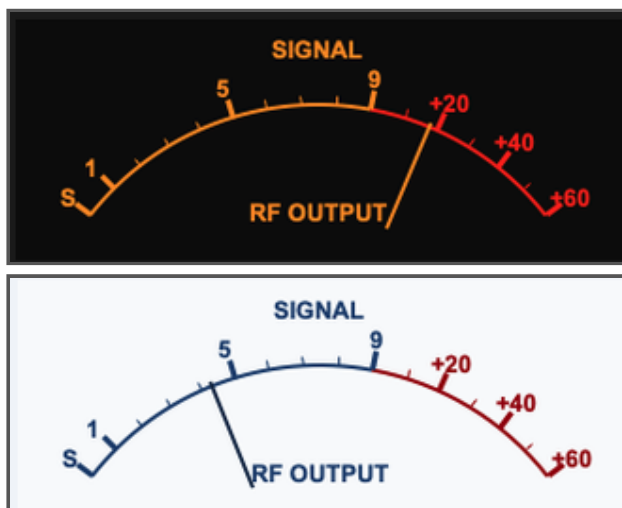
[Return to Table of Contents](#)

sCTkSMeter

Table of Contents

- Overview
- Constructor
- Methods
- State
- Fonts and Label Placement
- Centralized Stylesheet Integration
- Example

The `sCTkSMeter` is a standalone, theme-adaptive analog S-Meter/Power Output gauge instrument designed specifically for ham radio transceiver desktop interfaces. Natively inheriting container footprints from `customtkinter.CTkFrame`, it delivers smooth telemetry tracking sweeps without the overhead of extraneous nesting modules.



Overview

The instrument face is split mathematically to mirror classic analog transceiver gauge divisions perfectly:

- **The S-Unit Scale (Ticks 0–9):** Maps incoming telemetry values from `0.0` to `9.0` linearly across the first 60% of the visual arc container, rendered in your high-contrast brand or amber theme palettes.
- **The Decibel Over S9 Scale (Ticks 9–15):** Maps advanced signal parameters from `9.0` up to `69.0` across the remaining 40% of the dial arc track (where `+20dB` sits at coordinate 29, `+40dB` at 49, and `+60dB` at 69). This region is permanently framed by your crimson/redline alert warning colors.
- **Unified Pivot Axis Integration:** The inner rendering engines calculate lines, arcs, labels, and needle sweeps using a singular synchronized mathematical pivot point (`center_x = width * 0.48`). This entirely eliminates off-axis tracking drift or floating pointer artifacts when live data streams update.

Constructor

```
sCTkSMeter(master=None, width=250, height=130, state="normal", **kw)
```

Parameter Name	Data Type	Default Value	Description
master	any	None	Reference pointer tracking your root window or parent sCTkFrame container layout layer.
width	int	250	Panel width in pixels. Supports Pygubu geometry-default reset queries.
height	int	130	Panel height in pixels. Supports Pygubu geometry-default reset queries.
state	str	"normal"	"normal" or "disabled". See State below.

Methods

To drive the needle tracking sweep fluidly inside background receiver threads, automatic VFO frequency scanning loops, or telemetry data parsing hooks, utilize this direct public setter:

Update Instrument Needle Position

```
# Updates pointer positioning dynamically. Expects a float value clamped between 0.0 and 69.0.
smeter.set(value)
```

State

Method	Description
state(mode=None)	Getter with no argument; setter with "normal" or "disabled".
get_state()	Equivalent to state() with no argument.
configure(state=...)	Same effect. Both routes are supported.
cget("state")	Reads the current state.
configure("state")	Pygubu-style single-argument query.

Disabling dims, it does not freeze. `state("disabled")` changes only the palette. `set()` continues to update the needle, and the gauge keeps tracking live values in the dimmed colours. This is deliberate for an output-only instrument: a meter that held its last reading while greyed out would be indistinguishable from one showing a current value, which on a radio panel is actively misleading. There is no input to lock out — the state exists so a panel can disable every widget it contains uniformly.

The background is deliberately **not** dimmed; the face and needle carry the signal, matching `sCTkScrollableFrame` and the dial family.

Fonts and Label Placement

`font` and `scale_font` are **per-instance properties** as well as theme keys. Set either in the constructor, through `configure()`, or in the Designer's inspector; leave it blank and the theme's value applies.

Property	Applies to
font	The "SIGNAL" and "RF OUTPUT" captions
scale_font	The numeric scale tick labels

```
meter.configure(scale_font=("Arial", 12, "bold"))
meter.configure(scale_font="") # back to the theme's scale_font
```

Label positions are derived from the font, not hardcoded. Every gap between a label and the scale it marks used to be a fixed pixel count tuned for the default size, so a larger font grew across it and overlapped the thing it was labelling. Those offsets now come from the font's own line height, so text stays clear at any reasonable size.

The scale labels are placed by angle around the arc, so an oversized `scale_font` crowds them against one another rather than clipping at the canvas edge. There is no automatic spacing: if the labels start to touch, use a smaller `scale_font` or a larger meter.

Theming (sCTkThemes.json)

```
{
  "sCTkSMeter": {
    "fg_color": ["#F4F7FA", "#0A0A0A"],
    "text_color": ["#1A4375", "#FF9100"],
    "alarm_color": ["#990000", "#FF2200"],
    "needle_color": ["#112A4B", "#FF9100"],
    "font": ["Arial", 10, "bold"],
    "scale_font": ["Arial", 10, "bold"],
    "disabled_map": {
      "text_color": ["#94A3B8", "#4B5563"],
      "alarm_color": ["#CBD5E1", "#4B5563"],
      "needle_color": ["#94A3B8", "#4B5563"]
    }
  }
}
```

Every key above is required. Construction raises `KeyError` naming the missing key and whether it belongs at the top level or in `disabled_map`. This replaced a pattern of `.get(key, ("#hex", "#hex"))` throughout the draw code, which silently substituted a plausible guess and made an incomplete theme block look merely slightly-off rather than broken.

`font` is used for the "SIGNAL" and "RF OUTPUT" captions; `scale_font` for the numeric tick labels and the "S" marker. They're separate keys because the widget makes that distinction, even though the default values happen to match.

Superseded. This page used to warn that label positions were fixed pixel offsets tuned for 10pt text and that a larger font would overlap the scale. They are now derived from the font's own metrics — see [Fonts and Label Placement](#) for what still constrains a large font.

`fg_color` is a property as well as a theme key, and it paints the **canvas** rather than the frame behind it — the canvas covers the whole widget, so the frame is never visible. Set it in the inspector or through `configure()`; blank returns to the theme's value.

The other frame properties are deliberately **not** offered. `border_color`, `border_width` and `corner_radius` were added to the inspector at one point and taken back out: they appeared and did nothing at all, for the same reason. A property that cannot work is worse than an absent one.

`corner_radius` is the real loss. Rounding the meter would mean rounding the canvas, which Tk cannot do — it would have to be drawn, as a rounded rectangle filling the corners in the parent's colour. Possible, not free, and not done.

A colour set at runtime survives a state change, and clearing it returns to the theme rather than to the previous value. See [Theming](#).

Fixed: the configured `fg_color` never actually rendered. It was popped out of the resolved defaults in the constructor (correctly — the native frame takes it separately) and then read back afterwards from the dictionary it had been removed from, so the background always fell through to a hardcoded value. Light mode is where this was visible.

Example

Below is a complete, self-contained interactive test execution script demonstrating how to use `sCTkSMeter`.

```

#!/usr/bin/python3
# =====
# TESTING HARNESS IMPORTS & SETUP for S Meter
# =====

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkSMeter
import random

if __name__ == "__main__":

    root = sCTk()
    root.title("sCTk Standalone Analog Gauge")
    root.geometry("450x260")
    root.configure(fg_color=("#F1F5F9", "#1C1C1C"))

    dashboard_frame = sCTkFrame(root, fg_color="transparent", border_width=0)
    dashboard_frame.pack(padx=20, pady=20)

    smeter = sCTkSMeter(dashboard_frame, width=340, height=130)
    smeter.pack(padx=10, pady=10)

    class SignalSimulator:
        def __init__(self, root_win, meter):
            self.root, self.meter = root_win, meter
            self.target, self.needle = 6.0, 0.0

        def shift_vfo(self):
            self.target = random.uniform(1.5, 65.0)
            self.root.after(random.randint(2500, 5000), self.shift_vfo)

        def physics_loop(self):
            jitter = random.uniform(-1.5, 1.5)
            sig = max(0.0, min(69.0, self.target + jitter))
            self.needle += (sig - self.needle) * 0.25
            self.meter.set(self.needle)
            self.root.after(25, self.physics_loop)

    sim = SignalSimulator(root, smeter)
    sim.physics_loop()
    sim.shift_vfo()

    def toggle_theme():
        ctk.set_appearance_mode("Light" if ctk.get_appearance_mode() == "Dark" else "Dark")

    sCTkButtonPrimary(root, text="Toggle Theme mode", command=toggle_theme).pack(pady=5)
    root.mainloop()

```

[Return to Table of Contents](#)

sCTkSMeterBar

Table of Contents

- Overview
- Constructor
- Methods
- State
- Fonts and Label Placement
- Centralized Stylesheet Integration
- Example

The `sCTkSMeterBar` is a standalone, low-profile horizontal discrete 30-segment LED bar instrumentation widget displaying independent telemetry tracks for incoming receiver S-Units, transmitter SWR ratio levels, and forward RF Power output percentage. Like all sCTk widgets, it is fully theme-adaptive.



Overview

The discrete LED matrix map shifts automatically based on the device operational path constraints:

- **The S-Meter Track (Top Row):** Maps incoming telemetry values across 30 linear segments. Signals from `0.0` to `9.0` utilize the first 60% of the bar, while advanced signal ranges up to `+60dB` expand into the remaining 40% redline warning zone.
- **The Transmitter Track (Split Bottom Row):** Splices the lower segment path down the center into two separate monitoring zones. The left half maps a logarithmic SWR reflection track up to your custom `swr_max_value`, while the right half tracks forward RF power from `0%` to `100%`.
- **Post-Boot Geometry Flattening:** Overrides native internal grid layout constraints programmatically to force all 30 LED rectangles to sit perfectly flush. This completely removes horizontal spacing holes, keeping your panel elements locked into a solid hardware console bar.

Constructor

```
sCTkSMeterBar(master=None, swr_max_value=5.0, swr_visible=True, pwr_visible=True,
               hide_lower_row=False, width=320, height=110, state="normal", **kw)
```

Parameter Name	Data Type	Default Value	Description
master	any	None	Reference pointer tracking your root window or parent sCTkFrame container layout layer.
swr_max_value	float	5.0	The explicit maximum scale boundary representing the far right edge limit tracking your transmitter's SWR track.
swr_visible	bool	True	Visibility flag for the SWR cluster. False greys its text, ticks and LEDs to inactive_color. Distinct from widget state — see State .
pwr_visible	bool	True	Visibility flag for the PWR cluster. False greys its text, ticks and LEDs to inactive_color. Distinct from widget state — see State .
hide_lower_row	bool	False	Layout override command. When True, the entire lower instrumentation cluster collapses and vanishes, pushing the SIG bar to the true vertical center of the card footprint.
width	int	320	Panel width in pixels. Supports Pygubu geometry-default reset queries.
height	int	110	Panel height in pixels. Supports Pygubu geometry-default reset queries.
state	str	"normal"	"normal" or "disabled". See State below.

Methods

Update Instrument Telemetry Channels

```
# Pass parameters to update any of the 3 telemetry rows independently on the fly.
# Expects floats matching your radio data streams.
led_bar_gauge.set(s_value=9.2, swr_value=1.4, pwr_value=45.0)
```

Live Layout Configuration Modifier

```
# Updates layout presentation properties on the fly without reconstruction overhead.
led_bar_gauge.configure_visibility(swr_visible=False, pwr_visible=True, hide_lower_row=False)
```

The same three flags also go through the standard `configure()` call, which is what the Designer's inspector uses:

```
led_bar_gauge.configure(swr_visible=False, hide_lower_row=True)
led_bar_gauge.configure(swr_visible="") # back to the constructor default
```

They were constructor arguments with `cget()` support but **no `configure()` branch**, so setting one in the inspector reached native `CTkFrame` and raised `['pwr_visible'] are not supported arguments`. `swr_max_value` had a branch, which is why only these three failed.

State

Method	Description
<code>state(mode=None)</code>	Getter with no argument; setter with "normal" or "disabled".
<code>get_state()</code>	Equivalent to <code>state()</code> with no argument.
<code>configure(state=...)</code>	Same effect. Both routes are supported.
<code>cget("state")</code>	Reads the current state.
<code>configure("state")</code>	Pygubu-style single-argument query.

Disabling dims, it does not freeze. `state("disabled")` changes only the palette. `set()` continues to update all three telemetry rows, and the bar keeps tracking live values in the dimmed colours. This is deliberate for an output-only instrument: a meter that held its last reading while greyed out would be indistinguishable from one showing a current value, which on a radio panel is actively misleading. There is no input to lock out — the state exists so a panel can disable every widget it contains uniformly.

State and row visibility are independent. `state("disabled")` dims the whole widget via `disabled_map`. `configure_visibility(swr_visible=False)` greys just that cluster to `inactive_color`. A row can be hidden on an enabled widget, and a disabled widget still shows whichever rows are visible. Both can apply at once.

The background is deliberately **not** dimmed; the LEDs and labels carry the signal.

Fonts and Label Placement

`font` and `scale_font` are **per-instance properties** as well as theme keys. Set either in the constructor, through `configure()`, or in the Designer's inspector; leave it blank and the theme's value applies.

Property	Applies to
<code>font</code>	The "SIG", "SWR" and "PWR" captions
<code>scale_font</code>	The numeric scale tick labels

```
meter.configure(scale_font=("Arial", 12, "bold"))
meter.configure(scale_font="") # back to the theme's scale_font
```

Label positions are derived from the font, not hardcoded. Every gap between a label and the scale it marks used to be a fixed pixel count tuned for the default size, so a larger font grew across it and overlapped the thing it was labelling. Those offsets now come from the font's own line height, so text stays clear at any reasonable size.

The right-hand margin is sized for the widest scale label, `" +60 dB"`, which is centred on the end of the bar so half of it extends beyond. A larger `scale_font` therefore leaves proportionally less room for the bar itself — noticeable above about 18pt on a default-width meter. Widen the meter to compensate.

Theming (sCTkThemes.json)

```
{
  "sCTkSMeterBar": {
    "fg_color": ["#FFFFFF", "#0A0A0A"],
    "text_color": ["#1A4375", "#FF9100"],
    "alarm_color": ["#DC2626", "#FF2200"],
    "led_on_color": ["#2471A3", "#FF9100"],
    "led_off_color": ["#E2E8F0", "#1A1D20"],
    "inactive_color": ["#94A3B8", "#334155"],
    "font": ["Arial", 10, "bold"],
    "scale_font": ["Arial", 9, "bold"],
    "disabled_map": {
      "text_color": ["#94A3B8", "#4B5563"],
      "alarm_color": ["#CBD5E1", "#4B5563"],
      "led_on_color": ["#CBD5E1", "#374151"],
      "led_off_color": ["#F1F5F9", "#1A1D20"]
    }
  }
}
```

Every key above is required. Construction raises `KeyError` naming the missing key and whether it belongs at the top level or in `disabled_map`. This replaced a pattern of `.get(key, ("#hex", "#hex"))` throughout the draw code, which silently substituted a plausible guess and made an incomplete theme block look merely slightly-off rather than broken.

`inactive_color` greys the SWR or PWR cluster when that row is switched off via `configure_visibility()`. It has no `disabled_map` entry because it is not a state colour — see [State](#). This value was previously hardcoded in the draw routine with no theme lookup at all, the only colour in this widget the theme could not reach.

`font` is used for the "SIG", "SWR" and "PWR" section labels; `scale_font` for the numeric scale markings — S units, SWR values and power percentages. These were previously hardcoded across eight separate `create_text` calls and never consulted the theme.

Superseded. This page used to warn that label positions were fixed pixel offsets tuned for 9pt and 10pt text and that a larger font would overlap the scale. They are now derived from the font's own metrics — see [Fonts and Label Placement](#) for what still constrains a large font.

`fg_color` is a property as well as a theme key, and it paints the **canvas** rather than the frame behind it — the canvas covers the whole widget, so the frame is never visible. Set it in the inspector or through `configure()`; blank returns to the theme's value.

The other frame properties are deliberately **not** offered. `border_color`, `border_width` and `corner_radius` were added to the inspector at one point and taken back out: they appeared and did nothing at all, for the same reason. A property that cannot work is worse than an absent one.

`corner_radius` is the real loss. Rounding the meter would mean rounding the canvas, which Tk cannot do — it would have to be drawn, as a rounded rectangle filling the corners in the parent's colour. Possible, not free, and not done.

A colour set at runtime survives a state change, and clearing it returns to the theme rather than to the previous value. See [Theming](#).

Fixed: the configured `fg_color` never actually rendered. It was popped out of the resolved defaults in the constructor (correctly — the native frame takes it separately) and then read back afterwards from the dictionary it had been removed from, so the background always fell through to a hardcoded value. Light mode is where this was visible.

Example

Below is a complete, self-contained interactive test execution script demonstrating how to use `sCTkSMeterBar`.

```

#!/usr/bin/python3
# =====
# TESTING HARNESS IMPORTS & SETUP for S Meter Bar
# =====

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkSMeterBar
import random

if __name__ == "__main__":

    app = sCTk()
    app.title("sCTk Bar Instrument Test Harness")
    app.geometry("440x240")
    app.configure(fg_color=("#F1F5F9", "#1C1C1C"))

    panel_container = sCTkFrame(app, fg_color="transparent", border_width=0)
    panel_container.pack(padx=20, pady=15, fill="both", expand=True)

    led_bar_gauge = sCTkSMeterBar(panel_container, width=340, height=110)
    led_bar_gauge.pack()

    class HarnessSimulator:
        def __init__(self, root_win, bar):
            self.root, self.bar = root_win, bar
            self.s_target, self.s_curr = 4.0, 0.0
            self.swr_target, self.pwr_target = 1.0, 0.0
            self.swr_curr, self.pwr_curr = 1.0, 0.0
            self.tx_active = False

        def tuning_cycle(self):
            self.s_target = random.uniform(0.5, 13.5)
            if not self.tx_active and random.random() > 0.4:
                self.tx_active = True
                self.swr_target = random.uniform(1.1, 4.5)
                self.pwr_target = random.uniform(35.0, 95.0)
                self.root.after(random.randint(1500, 3000), self._release)
            self.root.after(random.randint(4000, 8000), self.tuning_cycle)

        def _release(self):
            self.tx_active = False
            self.swr_target, self.pwr_target = 1.0, 0.0

        def physics_tick(self):
            self.s_curr += (((max(0.0, min(15.0, self.s_target + random.uniform(-1.2, 1.2)))) - self.s_
curr) * 0.35
            self.swr_curr += (((max(1.0, min(5.0, self.swr_target + random.uniform(-0.15, 0.15)))) if s
elf.tx_active else 1.0)) - self.swr_curr) * 0.20
            self.pwr_curr += (((max(0.0, min(100.0, self.pwr_target + random.uniform(-2.5, 2.5)))) if s
elf.tx_active else 0.0)) - self.pwr_curr) * 0.20
            self.bar.set(s_value=self.s_curr, swr_value=self.swr_curr, pwr_value=self.pwr_curr)
            self.root.after(20, self.physics_tick)

    sim = HarnessSimulator(app, led_bar_gauge)
    sim.physics_tick()
    sim.tuning_cycle()

    def toggle_theme():
        ctk.set_appearance_mode("Light" if ctk.get_appearance_mode() == "Dark" else "Dark")

    sCTkButtonPrimary(app, text="Toggle Theme", command=toggle_theme).pack(pady=5)
    app.mainloop()

```

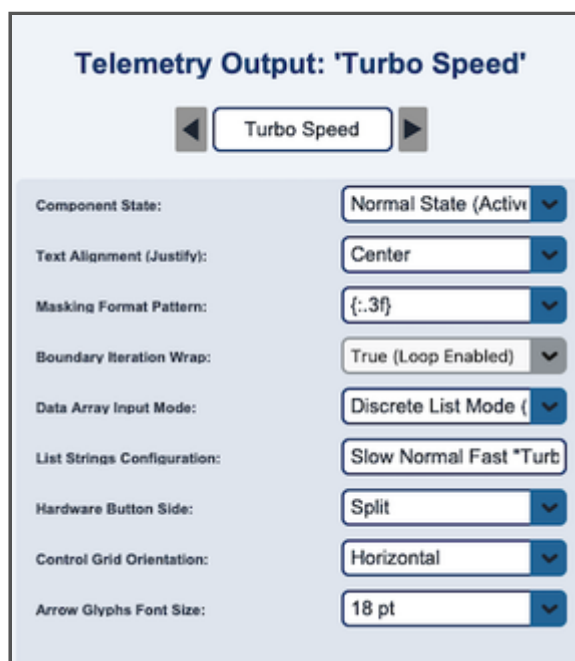
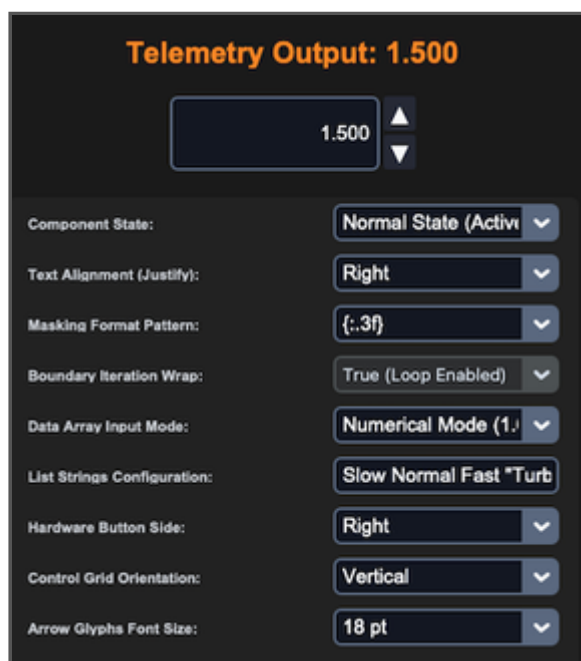
sCTkSpinbox

Table of Contents

- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

`sCTkSpinbox` is a theme-compliant subclass of `customtkinter.CTkFrame` that composes an internal `sCTkEntryPrimary` alongside two increment/decrement buttons. It supports two operating modes: stepping a numeric value between `from_` and `to` in increments of `step_size`, or cycling through a fixed list of string values.



Like `sCTkEntryPrimary`, this widget has a genuine three-state model — normal, readonly, and disabled — matching real `ttk.Spinbox` semantics. The key distinction: in readonly mode, the entry can't be typed into directly, but the increment/decrement buttons stay fully clickable, since they're the intended alternative way to change the value. The buttons themselves only ever report "normal" or "disabled" — "readonly" isn't a real native `CTkButton` concept, and there's no reason for the buttons to look any different in readonly mode, since nothing about their own behavior changes.

Every property this widget forwards to its internal entry (`width`, `height`, `justify`, `show`, `exportselection`, `placeholder_text`, `fg_color`, `border_color`, `text_color`, `state`) has been confirmed valid against CustomTkinter's own real `CTkEntry` source — both its explicit named constructor/configure parameters and its `_valid_tk_entry_attributes` whitelist. There's no risk of this widget sending an unrecognized property to its own entry.

Constructor

```
sCTkSpinbox(master=None, from_=0.0, to=100.0, step_size=1.0, command=None,
             state="normal", wrap=False, justify="left", show=None,
             placeholder_text=None, exportselection=True, width=140, height=32, **kw)
```

Parameter	Type	Default	Description
master	widget	None	Parent container.
from_	float	0.0	Lower bound of the numeric range. Ignored in discrete-value mode (see values below).
to	float	100.0	Upper bound of the numeric range. Ignored in discrete-value mode.
step_size	float	1.0	Amount added or subtracted per click of the up/down buttons.
command	callable	None	Called with the new value whenever it changes, via either button click or set().
state	str	"normal"	Initial state: "normal", "readonly", or "disabled". Any other value is treated as "normal".
wrap	bool	False	If True, stepping past either bound (or past either end of a discrete value list) wraps around instead of stopping.
justify	str	"left"	Text alignment inside the entry: "left", "center", or "right".
show	str	None	Character-masking string, e.g. "*" for password-style display.
placeholder_text	str	None	Placeholder shown when the entry is empty.
exportselection	bool	True	Standard Tkinter selection-to-clipboard behavior.
width / height	int	140 / 32	Overall widget dimensions in pixels.
**kw	—	—	button_width, button_height, button_side ("left"/"right"/"split"), orientation ("vertical"/"horizontal"), arrow_font_size, format, values — see Theming for where these come from — plus any theme-key override.

```
freq_spinbox = sCTkSpinbox(control_panel, from_=88.0, to=108.0, step_size=0.1, placeholder_text="MHz")
freq_spinbox.pack(pady=10)
```

Methods

Method	Returns	Description
<code>get()</code>	<code>str</code>	Current entry text.
<code>set(value)</code>	<code>None</code>	Sets the displayed value (a number, or a string matching one of values in discrete mode), and calls <code>command</code> if one was given. Temporarily re-enables the entry to update its text if it isn't currently "normal", then restores whatever state it was actually in — including "readonly", not just "normal"/"disabled".
<code>set_values(list_of_strings)</code>	<code>None</code>	Switches to discrete-value mode with the given list, or back to numeric mode if given an empty list. Accepts any of the forms described in List Properties .
<code>state(mode=None)</code>	<code>str</code>	Gets or sets the widget's normal/readonly/disabled state. The entry receives the full three-way state (routed through its own <code>state()</code>); the up/down buttons only ever receive "normal" or "disabled".
<code>get_state()</code>	<code>str</code>	Equivalent to calling <code>state()</code> with no argument.
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard configuration, plus all the constructor's custom keywords (<code>button_width</code> , <code>orientation</code> , <code>format</code> , <code>values</code> , etc.) can be changed at runtime the same way.
<code>cget(attribute_name)</code>	varies	Supports querying <code>state</code> , <code>from_</code> , <code>to</code> , <code>step_size</code> , <code>button_width</code> , <code>button_height</code> , <code>button_side</code> , <code>orientation</code> , <code>arrow_font_size</code> , <code>format</code> , <code>wrap</code> , and <code>values</code> directly, in addition to standard native properties.

Theming (sCTkThemes.json)

```
{
  "sCTkSpinbox": {
    "font": ["Arial", 15, "normal"],
    "arrow_font": ["Arial", 8, "normal"],
    "arrow_up_char": "▲",
    "arrow_down_char": "▼",
    "arrow_right_char": "▶",
    "arrow_left_char": "◀",
    "border_width": 1.5,
    "corner_radius": 6,
    "entry_color": ["#FFFFFF", "#111827"],
    "border_color": ["#1A4375", "#64748B"],
    "text_color": ["#1F2937", "#F9FAFB"],
    "placeholder_text_color": ["#5A6E7F", "#526071"],
    "button_color": ["#9E9E9E", "#2A2F3D"],
    "button_hover_color": ["#7D7D7D", "#374151"],
    "disabled_map": {
      "entry_color": ["#F3F4F6", "#1F2937"],
      "border_color": ["#CBD5E1", "#475569"],
      "text_color": ["#94A3B8", "#64748B"],
      "button_color": ["#CBD5E1", "#334155"]
    },
    "readonly_map": {
      "entry_color": ["#F8FAFC", "#1F2937"],
      "border_color": ["#64748B", "#94A3B8"],
      "text_color": ["#1F2937", "#F9FAFB"]
    }
  }
}
```

`entry_color` / `border_color` / `text_color` override the internal entry's own colors for all three states, a deliberate design choice: this widget controls its entry's look via its own theme keys rather than the entry's independent defaults.

text_color reaches the entry in every state. It used to be applied only for readonly, with normal and disabled delegated to `sCTkEntryPrimary`'s own block — so setting `disabled_map.text_color` here coloured the **arrows** and left the number in the box using the entry's colour. One key appearing to affect the wrong part of the widget. Note this can shift the normal appearance too, wherever the two blocks disagree; that is the point, since the spinbox block is the source of truth for a spinbox. `readonly_map` requires `entry_color`, `border_color`, and `text_color` whenever readonly is actually requested — missing any raises immediately. No readonly-specific `button_color` exists or is needed, since buttons always use normal `button_color` / `button_hover_color` whenever they aren't disabled — they're meant to look completely ordinary in readonly mode.

A colour set at runtime survives a state change, and clearing it returns to the theme's value rather than to whatever was set before. See [Theming](#) for the general rule.

values accepts the library's standard list formats — `["Porsche", "VW", "Tesla"]`, a bare comma-separated string, or a real Python list. See [List Properties](#).

Note that **space is no longer a separator**. An earlier version used `shlex.split()` whenever the input contained no comma, which made `Meat Loaf` two values here and one value in every other widget. Quote a value if it needs to contain a comma.

format accepts three forms, all equivalent:

Written as	Meaning
<code>:.2f</code>	Shorthand — braces are added for you

Written as	Meaning
{:.2f}	Full Python brace form
%.2f	printf style

Leave it blank for no formatting, in which case the decimal places follow `step_size`.

`arrow_font` is read in full — family, size, and weight — and applied to both increment/decrement buttons. It can also be overridden at runtime via `configure(arrow_font=(...))`, or `configure(arrow_font_size=...)` to change just the size without respecifying the full tuple.

Example

```
from scustomtkinter import sCTk, sCTkFrame, sCTkSpinbox, sCTkButtonPrimary, sCTkLabelPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("Spinbox Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    freq_spinbox = sCTkSpinbox(base, from_=88.0, to=108.0, step_size=0.1, placeholder_text="MHz")
    freq_spinbox.pack(pady=10)

    status = sCTkLabelPrimary(base, text=f"state: {freq_spinbox.get_state()}")
    status.pack(pady=5)

    def cycle_state():
        order = ["normal", "readonly", "disabled"]
        current = freq_spinbox.get_state()
        next_state = order[(order.index(current) + 1) % len(order)]
        freq_spinbox.state(next_state)
        status.configure(text=f"state: {freq_spinbox.get_state()}")

    cycle_btn = sCTkButtonPrimary(base, text="Cycle State", command=cycle_state)
    cycle_btn.pack(pady=10)

    root.mainloop()
```

Known Limitations

- **Setting `placeholder_text` after construction only clears the field if it still holds the initial value.** That is intentional — a placeholder must not wipe a value the user has typed. An earlier version compared the entry text against `str(from_)` while the entry actually held the *formatted* initial value, so with a `format` set the two never matched and the placeholder never appeared.
- **The disable/enable-cycle cursor-position fix is not independently confirmed for readonly transitions** — the underlying entry inherits this caveat from `sCTkEntryPrimary`; see that widget's docs for the full explanation.
- **readonly mode's placeholder behavior follows `sCTkEntryPrimary`'s** — a readonly field showing placeholder text never clears it on focus, since native CustomTkinter deliberately never deactivates a placeholder while `state` is "readonly".
- **Fixed:** single-argument queries used to report `str()` of a `(light, dark)` tuple rather than a resolved colour. The shared query helper now resolves the pair.

- **Fixed:** building a spinbox already disabled raised `ValueError: ['state'] are not supported arguments`. The constructor called `super().configure(state="disabled")`, and this widget is a `CTkFrame` subclass with no such option. It only fired when the state came from the constructor rather than a later call — which is what a Designer preview does.

[Return to Table of Contents](#)

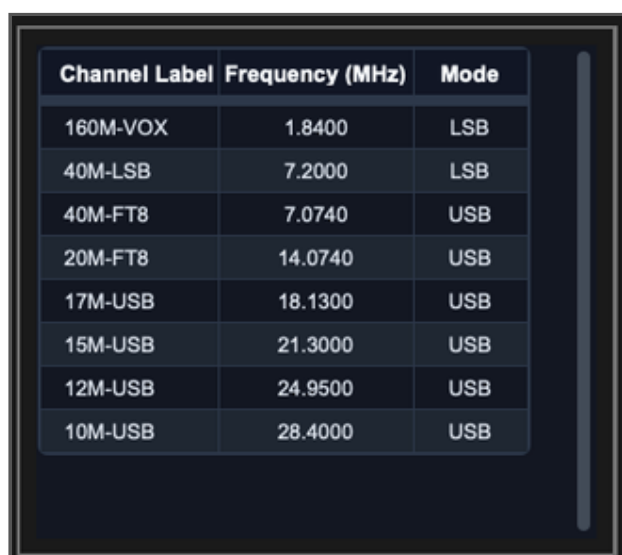
sCTkTableview

Table of Contents

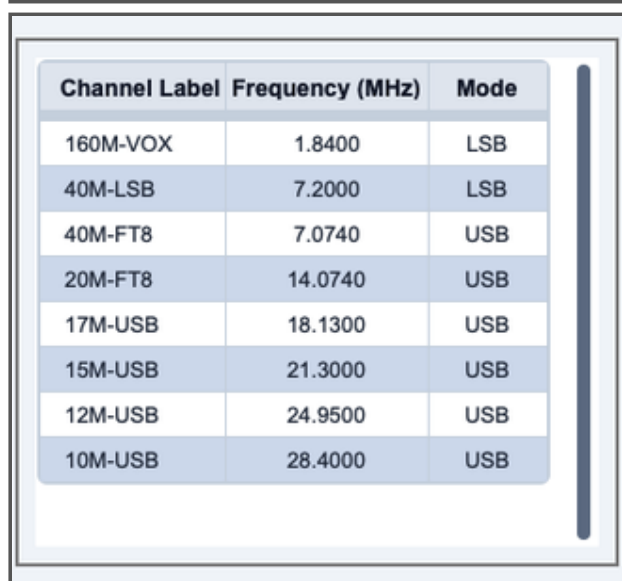
- Overview
- Constructor
- Methods
- Theming (sCTkThemes.json)
- Example
- Known Limitations

Overview

sCTkTableview is a theme-compliant, scrollable grid of labeled cells — a simple spreadsheet-like table, with optional zebra-striped rows, click and edit callbacks, and in-place cell editing. It's built by inheriting sCTkScrollableFrame directly, using its scrolling and label feature, then laying out its own header row and cell grid on top.



Channel Label	Frequency (MHz)	Mode
160M-VOX	1.8400	LSB
40M-LSB	7.2000	LSB
40M-FT8	7.0740	USB
20M-FT8	14.0740	USB
17M-USB	18.1300	USB
15M-USB	21.3000	USB
12M-USB	24.9500	USB
10M-USB	28.4000	USB



Channel Label	Frequency (MHz)	Mode
160M-VOX	1.8400	LSB
40M-LSB	7.2000	LSB
40M-FT8	7.0740	USB
20M-FT8	14.0740	USB
17M-USB	18.1300	USB
15M-USB	21.3000	USB
12M-USB	24.9500	USB
10M-USB	28.4000	USB

This widget inherits `sCTkScrollableFrame` directly — the same composition pattern used by `sCTkSelector` — and previously needed a fragile workaround for it: temporarily overwriting its own `self.__class__.__name__` during construction, to trick `sCTkScrollableFrame`'s internal `ThemeableWidget.__init__` call into reading a harmless theme block instead of corrupting this widget's own. That workaround has been removed entirely. `ThemeableWidget`'s run-once guard now prevents the double-init outright, and `sCTkScrollableFrame` itself filters its inbound kwargs down to only what native `CTkScrollableFrame` actually accepts — confirmed directly against `CustomTkinter`'s source to have no `**kwargs` catch-all at all, so this filtering matters more here than for almost any other widget in this project.

Constructor

```
sCTkTableview(master, columns=None, width=500, height=300, grid_mode="zebra",
               header_line_width=2, outline_width=1.0, outline_radius=4,
               state="normal", num_columns=3, num_rows=1, show_headers=True,
               cell_bg_color=None, cell_alt_bg_color=None, *args, **kwargs)
```

Parameter	Type	Default	Description
<code>master</code>	widget	—	Parent container.
<code>columns</code>	<code>list[str]</code> or <code>str</code>	None	Column header labels. Accepts ["Time", "Freq"], a bare comma-separated string, or a real list — see List Properties . Setting this also sets <code>num_columns</code> to match.
<code>width / height</code>	<code>int</code>	500 / 300	Overall widget dimensions in pixels.
<code>grid_mode</code>	"zebra" / "grid" / "none"	"zebra"	Row background styling.
<code>header_line_width</code>	<code>int</code>	2	Header row's bottom border thickness.
<code>outline_width / outline_radius</code>	<code>float / int</code>	1.0 / 4	Outer table border thickness and corner rounding.
<code>state</code>	"normal" / "disabled"	"normal"	Initial state.
<code>num_columns / num_rows</code>	<code>int</code>	3 / 1	Initial grid size when columns isn't given.
<code>show_headers</code>	<code>bool</code>	True	Whether the header row is shown.
<code>cell_bg_color / cell_alt_bg_color</code>	<code>color</code>	None	Overrides the theme's cell background colors for this instance specifically — see Theming for how this interacts with the theme file.
<code>**kwargs</code>	—	—	Any native <code>CTkScrollableFrame</code> argument, or an override for one of the other theme keys listed under Theming .

```
readings_table = sCTkTableview(control_panel, columns=["Time", "Frequency", "Signal"], num_rows=8)
readings_table.pack(expand=True, fill="both", padx=20, pady=20)
```

Methods

Method	Returns	Description
<code>state(mode=None) / get_state()</code>	<code>str</code>	Gets or sets "normal"/"disabled".
<code>configure(**kwargs) / config(**kwargs)</code>	varies	Standard configuration, plus <code>state=...</code> triggers a full color/font re-application across every header and cell. <code>columns=...</code> rebuilds the header row and resizes the grid to match.

Theming (sCTkThemes.json)

- **Applied once, at construction** — every key below, plus `cell_bg_color / cell_alt_bg_color` (which can also come from the constructor, see below).
- **Re-applied on every `state()` change.**

```
{
  "sCTkTableview": {
    "header_bg_color": ["#E2E8F0", "#0F172A"],
    "header_text_color": ["#0F172A", "#F8FAFC"],
    "header_font": ["Arial", 14, "bold"],
    "cell_bg_color": ["#FFFFFF", "#111827"],
    "cell_alt_bg_color": ["#D1DCEE", "#222C3A"],
    "cell_text_color": ["#1E293B", "#E2E8F0"],
    "cell_font": ["Arial", 13, "normal"],
    "grid_line_color": ["#CBD5E1", "#334155"],
    "disabled_map": {
      "header_bg_color": ["#CBD5E1", "#1E293B"],
      "header_text_color": ["#94A3B8", "#64748B"],
      "cell_bg_color": ["#F1F5F9", "#1F2937"],
      "cell_alt_bg_color": ["#E2E8F0", "#263241"],
      "cell_text_color": ["#94A3B8", "#64748B"],
      "grid_line_color": ["#E2E8F0", "#293548"]
    }
  }
}
```

All six colors are required both at the top level and in `disabled_map` — missing any raises immediately at construction, naming the exact key. `header_font / cell_font` are required only at the top level; no widget in this project uses a disabled-state font variant.

`cell_bg_color / cell_alt_bg_color` are the two exceptions — they can come from either the theme block or the constructor kwarg of the same name, so it's only a hard failure if *neither* provides a value. Whichever one this instance resolves to at construction is remembered and correctly restored on every return to "normal" — an earlier version always reverted to the theme's value on re-enable, silently discarding a constructor override after a disable/enable cycle.

Colors are passed through as raw `(light, dark)` tuples, letting CustomTkinter's native appearance-mode tracking handle repaints — an earlier version resolved disabled-state colors to a single fixed string while leaving enabled-state colors as tuples, meaning a disabled table would stop following light/dark mode changes while an enabled one kept working correctly. Both branches are now consistent.

Example

```
from scustomtkinter import sCTk, sCTkFrame, sCTkTableview, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("Tableview Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    table = sCTkTableview(base, columns=["Time", "Frequency", "Signal"], num_rows=6)
    table.pack(expand=True, fill="both", pady=10)

    def toggle_disabled():
        target = "disabled" if table.get_state() == "normal" else "normal"
        table.configure(state=target)
        toggle_btn.configure(text="Enable Table" if target == "disabled" else "Disable Table")

    toggle_btn = sCTkButtonPrimary(base, text="Disable Table", command=toggle_disabled)
    toggle_btn.pack(pady=10)

    root.mainloop()
```

Known Limitations

- **Changing columns clears the table.** The rebuild reloads with empty rows, the same as changing `num_columns`. Expected at design time; reload your data afterwards at runtime.
- **The edit callback fires only when a value actually changes.** Retyping the same value, or leaving an editor without altering anything, is silent — as is an edit the validation callback rejects. An earlier version compared the cell against the value it had just written to that same cell, a condition that was always true, so the callback fired on every save regardless.
- Missing a required theme key raises `KeyError` at construction, naming exactly which key and whether it's needed at the top level or in `disabled_map` — check the exact message if construction fails after a theme file change.
- Calling `configure("propname")` for most single-argument property queries falls through to the native widget's `configure()`, which doesn't support arbitrary single-argument queries — the same known gap as elsewhere in this project's Pygubu-query investigation.

[Return to Table of Contents](#)